



How does a storm turn into a typhoon

A storm becomes a typhoon when it is a well-organised tropical cyclone over the Northwest Pacific and its sustained winds reach at least about 74 mph (119 km/h).^{[1] [2]}

Step-by-step: from storm to typhoon

1. Warm ocean and rising air

- It starts over very warm tropical ocean water, usually at least about 26–27 °C.^{[1] [3]}
- Warm, moist air rises from the sea surface, creating a region of low pressure that pulls in more air at the surface.^{[4] [3]}

2. Tropical disturbance to depression

- Groups of thunderstorms over the ocean begin to cluster and organise, forming a tropical disturbance.^{[3] [1]}
- As circulation tightens and a clear low-pressure centre forms, it is called a tropical depression, with relatively weak winds.^{[2] [1] [3]}

3. Tropical storm stage

- If the system strengthens and the sustained winds reach 39 mph (63 km/h), it is upgraded to a tropical storm and given a name.^{[1] [2]}
- The storm now has a more defined spiral shape and stronger rain bands feeding into the centre.^[4]

4. Becoming a typhoon

- As long as it stays over warm water, with lots of moist air, low wind shear (winds not tearing it apart with height), and enough Coriolis effect to spin, the storm can intensify further.^{[4] [3]}
- Once maximum sustained winds reach 74 mph (119 km/h) or more, it is classified as a typhoon if it is in the Northwest Pacific; the same thing in the Atlantic is called a hurricane.^{[1] [2] [5]}

Why location and name matter

- “Typhoon” is just the regional name for a strong tropical cyclone in the Northwest Pacific; the physics are the same as hurricanes and cyclones elsewhere.^{[1] [2] [3]}
- All of them are rotating storms with a low-pressure centre, strong winds and heavy rain; differences in name are about geography, not how they form.^{[2] [6]}

What can stop or weaken it

- Moving over cooler water, hitting land, or encountering strong wind shear cuts off the warm, moist energy supply, so the typhoon weakens.^[4] ^[3]
- As it decays, winds drop below typhoon strength, and it downgrades again to a tropical storm or depression, then eventually just a remnant low.

Would it help if I explained this again but aimed at, say, a primary-school level, or are you looking for more of the full meteorology detail?



What is the formula that links temperature wind speed and precipitation for a tc

There is no single simple formula that directly links temperature, wind speed and precipitation for a tropical cyclone; they are related through more complex physics and models.^[11] ^[12]

Why there is no single formula

- Tropical cyclones are governed by the full equations of fluid dynamics and thermodynamics (Navier–Stokes, heat and moisture budgets), not one neat algebraic formula.^[12]
- Temperature, wind and rainfall influence each other indirectly via sea-surface temperature, humidity, pressure gradients, and vertical motion in the storm.^[11] ^[12]

Common relationships that are used

Instead of one “linking formula”, people use several approximate relationships:

- **Intensity vs temperature (potential intensity)**
A widely used theory (Emanuel potential intensity) links maximum wind speed $V_{\max}$ to sea-surface temperature and outflow (upper-troposphere) temperature, plus exchange coefficients.^[13]
In words: warmer sea surface and colder upper atmosphere allow stronger maximum winds, but this does not directly give you rainfall.^[13]
- **Rainfall vs wind speed and temperature**
Studies show that the size and amount of tropical-cyclone rainfall (its “precipitation footprint”) depend strongly on wind speed, sea-surface temperature and environmental moisture, often captured with statistical or machine-learning models rather than a closed-form equation.^[14]
These models look like $P = f(V, \text{SST}, \text{humidity}, \text{location}, \text{shear}, \dots)$, where P is precipitation and V is wind speed, but the function f is learned, not a simple analytic formula.^[14]
- **Thermodynamic constraints**
Moisture-convergence and energy-budget arguments can relate average precipitation to

how much moist air flows into the storm and how much latent heat is released, but again this is usually expressed as a set of coupled equations, not a single neat expression in T, V, P .^{[12] [11]}

If you are after something specific

If you tell me whether you want:

- a simple teaching-level approximation,
- or a research-level expression like the Emanuel potential intensity formula,

I can write out one or two concrete equations and walk through what each term means.



I want to build a cyclone simulation

Yes — a cyclone simulation is very doable, but the first decision is what kind of simulation you want: a simple educational model, a game-like browser simulator, or a more scientific track/intensity model. Research examples show all three exist, from classroom formation demos to browser hurricane simulators and research models for track and wind fields.^{[18] [19] [20] [21]}

Good starting options

For a first build, the best approach is usually a simplified browser simulation with a storm moving over warm water, strengthening or weakening based on a few variables such as sea-surface temperature, humidity, wind shear and land interaction. Educational resources consistently describe those factors as the core ingredients for tropical cyclone formation and evolution, which makes them a solid basis for a playable or explorable model.^{[22] [23] [24]}

A practical model could update these state variables each time step:

- Position (x, y)
- Central pressure
- Maximum wind speed
- Sea-surface temperature
- Environmental humidity
- Vertical wind shear
- Ocean/land flag
- Storm stage: disturbance, depression, storm, cyclone/typhoon^{[19] [21] [23] [24]}

Recommended scope

I'd suggest building a 2D educational web app first, not a full physics solver. Full tropical cyclone models used in research are far more complex and rely on large-scale atmospheric modelling, while simple tropical cyclone model families and wind-profile methods show that reduced models are much more realistic for a first implementation.^{[19] [25] [21]}

A sensible first version would include:

1. A map or ocean grid with SST and land.
2. A storm seed that intensifies over warm water and weakens over land or in high shear.
3. A track rule that drifts westward then curves poleward.
4. Charts or panels showing wind speed, pressure and rainfall proxy over time.^{[21] [23] [24] [19]}

Simulation rules

You do not need a single exact formula linking temperature, wind and precipitation; a rules-based model is more appropriate for a first simulator because the real relationships are coupled and nonlinear. Research and educational sources both support using thresholds such as warm water around 26.5–27 °C, low shear, moist air, and progressive storm stages as simplified controls.^{[19] [23] [24]}

A good first-pass rule set is:

- If SST is below about 26.5 °C, intensity decays.^{[23] [24]}
- If shear is high, organisation drops and strengthening slows.^{[22] [24]}
- Over land, moisture supply is cut off and decay accelerates.^[24]
- Higher wind speed gives higher rainfall potential, using a simple proxy rather than a full precipitation model.^{[21] [26]}

Build choice

The most useful next step is for me to build you a single-file interactive web app that runs in the browser and lets you tune SST, shear, humidity and steering flow, then watch the cyclone form, intensify and decay. That fits well with the kind of browser-based hurricane simulators and educational storm tools already in use online.^{[20] [27]}

Choose one:

- **Educational simulator** — clean 2D model, sliders, charts, easy to understand.
- **Game-style simulator** — more visual, score/impacts, less scientific.
- **Quant-style model prototype** — simplified track and intensity equations with parameters exposed.

Which of those three do you want me to build first?

Scientific for insurance what about oasis?

For an insurance-grade, scientific cyclone simulation, you'd normally build (or plug into) a catastrophe model and then run it on a platform like Oasis rather than roll your own end-to-end stack from scratch.^{[34] [35] [36]}

What “scientific for insurance” implies

In a (re)insurance context a tropical-cyclone model is expected to:

- Simulate large event sets (thousands of synthetic storms) with realistic tracks, intensities and wind (and often surge/precip) fields.^{[37] [38]}
- Combine those hazards with exposure and vulnerability to produce event loss tables (ELTs) and year loss tables (YLTs) for pricing, capital and risk transfer.^{[39] [38]}

The TC piece itself typically has:

- A basin-wide event generator (genesis, track, intensity evolution).^[38]
- A wind-field / footprint model to get surface gusts at each location.^{[37] [38]}
- Sometimes coupling to rainfall and surge, especially for flood and parametric covers.^{[40] [38]}

Where Oasis fits

Oasis LMF is an open-source catastrophe modelling platform that gives you the plumbing: data standards, APIs, and engines for hazard, vulnerability and financial calculation.^{[35] [36] [34]}

Key points for your use case:

- It is designed specifically for insurance and reinsurance workflows (underwriting, portfolio analytics, capital).^{[41] [35]}
- It provides a model execution framework (“ktools”) and a financial module to apply policy terms and produce ELTs/YLTs, with open data standards for hazard and exposure.^{[36] [42] [39]}
- For climate-risk use cases, Oasis has already hosted typhoon models (for example, a Bangladesh typhoon model and flood/typhoon models for Asia) that run on the same platform.^[40]

So you'd either:

- Develop your own TC hazard module (track + footprint) and wrap it in the Oasis model format, then let Oasis handle exposure, vulnerability and financial calculations.^{[35] [36] [38]}
- Or adopt an existing TC model built for Oasis (e.g. from the Oasis model library / Oasis Hub) and focus on configuring, validating and integrating it with your portfolios.^{[43] [39] [40]}

TC modules and Oasis

Oasis itself is hazard-agnostic: you can plug in any tropical-cyclone model that can produce events and footprints in its standard formats.^{[36] [38] [35]}

Typical TC-in-Oasis architecture:

- Event set: simulated storms with IDs, times, and parameters like track, central pressure and max wind.^[38]
- Footprint: for each event, gridded or site-based wind (and possibly rainfall/surge), output in Oasis hazard formats.^{[37] [38]}
- Vulnerability: curves mapping hazard intensity (e.g. peak gust) to damage ratios by construction class/occupancy.^{[39] [38]}
- Financial engine: policy terms, layers and aggregates applied to damage to get insured losses.^{[42] [39] [36]}

The Oasis “Tropical Cyclone Modelling” hazard documentation explicitly describes basin-wide TC catastrophe models built in this way and run through Oasis.^[38]

Practical path for you

Given your background, a realistic “insurance-grade” path is:

- Use or adapt a physics-based or statistical TC model (or published open models) that can generate large sets of storms under different climates.^{[44] [37] [38]}
- Turn its outputs into Oasis-compliant hazard and event files using the Oasis Model Development Kit.^{[35] [36]}
- Run full portfolio loss simulations via Oasis (ktools / Platform), getting ELTs, YLTs, PMLs and AALs that fit standard market practice.^{[42] [39] [36]}

If you tell me whether you want to (a) build a new TC hazard module for Oasis, or (b) stand up an Oasis stack and plug in existing cyclone models, I can outline concrete next steps and data structures for that specific route.

✱

I am building a Monte Carlo simulator so what are the starting conditions

For a Monte Carlo tropical-cyclone simulator, the starting conditions are usually not one fixed state but a sampled **genesis state**: where and when the storm is born, plus the local environment that controls whether it intensifies or dies.^{[49] [50] [51]}

Core initial state

At minimum, each simulated storm should start with these variables:

- Genesis location: latitude and longitude, sampled from a basin- and season-specific genesis distribution. [\[52\]](#) [\[53\]](#) [\[49\]](#)
- Genesis time: month or day-of-year, because seasonal effects strongly affect genesis frequency and tracks. [\[53\]](#) [\[49\]](#)
- Initial intensity: central pressure or maximum sustained wind, usually as a weak disturbance / depression near the lower bound of tropical cyclone formation. [\[54\]](#) [\[53\]](#)
- Initial motion: translation speed and heading, often sampled conditionally on genesis location and season. [\[52\]](#) [\[53\]](#)

These are the storm-state variables; they define the seed you propagate forward in time. [\[53\]](#) [\[52\]](#)

Environmental conditions

You also need environmental covariates at the genesis point, because genesis and early intensification depend heavily on them:

- Sea-surface temperature or potential intensity proxy. [\[51\]](#) [\[54\]](#)
- Mid-level or low-level relative humidity. [\[55\]](#) [\[51\]](#)
- Vertical wind shear. [\[56\]](#) [\[57\]](#) [\[51\]](#)
- Coriolis parameter or latitude, since genesis does not occur too near the equator. [\[58\]](#) [\[59\]](#)
- Low-level vorticity / pre-existing disturbance indicator if you want a more physical genesis filter. [\[56\]](#) [\[58\]](#)

A common probabilistic genesis setup uses environmental predictors such as potential intensity, humidity, wind shear and vorticity in a Poisson or logistic model for storm occurrence. [\[59\]](#) [\[51\]](#)

Practical initialization ranges

For a first insurance-oriented simulator, a practical starting rule set is:

- Latitude: sample only outside the near-equatorial belt, typically not within about 5 degrees of the equator because Coriolis forcing is too weak there. [\[58\]](#)
- SST: require warm water, commonly around 26.5 °C or above. [\[54\]](#) [\[58\]](#)
- Relative humidity: exclude very dry environments; one Southern Hemisphere study found genesis events were not observed below 63% at 1000 mb or below 37% at 850 mb. [\[55\]](#)
- Wind shear: favour weak-to-moderate shear, because strong shear suppresses organisation. [\[57\]](#) [\[51\]](#) [\[56\]](#)
- Initial pressure/wind: start near tropical depression strength, not at mature cyclone intensity. [\[53\]](#) [\[54\]](#)

These are best treated as sampled constraints, not hard deterministic cutoffs, unless you want a very simple model.^{[50] [51]}

Insurance model structure

For insurance use, I would initialize each event in four layers:

1. Event birth: basin, season, genesis point, genesis time.^{[49] [53]}
2. Local environment: SST/potential intensity, humidity, shear, vorticity.^{[51] [56]}
3. Storm seed: initial pressure, wind, radius, heading, forward speed.^{[52] [53]}
4. Exposure interface: once the event evolves, calculate hazard footprints at insured locations for vulnerability and financial loss modules.^{[60] [61]}

That is closer to cat-model practice than trying to start from only temperature, wind and precipitation.^{[61] [60] [51]}

Suggested starter vector

A good sampled initial vector for each Monte Carlo event is:

$$X_0 = (\text{basin}, t_0, \phi_0, \lambda_0, V_{max,0}, P_{c,0}, u_0, \theta_0, SST_0, RH_0, Shear_0, Vor_0)$$

where t_0 is genesis time, ϕ_0, λ_0 are latitude/longitude, $V_{max,0}$ is initial max wind, $P_{c,0}$ is central pressure, u_0 is forward speed, θ_0 is heading, and the remaining terms are environmental controls.^{[51] [52]}

If you want, the next step is for me to turn this into a concrete Monte Carlo schema with distributions for each variable and a step-by-step event evolution algorithm.



can you check the attached model coding, I believe this is a typical storm intensity. can you review first and let me know what is missing.

Yes — this looks like a storm intensity marginal distribution model, not yet a full tropical-cyclone intensity process model. It is a reasonable first component for severity sampling, but several scientific and coding elements are still missing if you want it to represent “typical storm intensity” in an insurance Monte Carlo setting.^{[66] [67] [68]}

What the code currently is

The core design is a hybrid severity distribution: truncated normal for the body and a Pareto tail beyond a threshold, with scenario families changing the mean, volatility, threshold and tail index.^{[67] [68]}

That means the model currently generates independent intensity draws and exceedance/return-period estimates from a fitted marginal distribution, rather than

simulating genesis, track, lifecycle evolution, spatial footprint, or landfall-conditioned intensity.^[66] ^[67]

In other words, this is closer to a severity layer you might use inside a cat model than a storm physics model.^[67] ^[66]

Scientific pieces missing

For insurance-grade TC modelling, the biggest missing items are the state variables and conditioning structure. Right now intensity appears unconditional apart from scenario family choice, so the model omits:

- Basin and season dependence, e.g. Atlantic vs WNP and month-of-year.^[68] ^[67]
- Genesis and lifecycle state, e.g. formation, intensification, peak, decay.^[67]
- Environmental conditioning, e.g. SST, shear, humidity, land interaction, latitude.^[68] ^[67]
- Track and landfall dependence, because intensity at peak ocean state is not the same as intensity at insured exposure locations.^[67]
- Size / radius metrics such as RMW or wind radii, which are essential for hazard footprint and loss, not just scalar intensity.^[67]
- Event frequency model, since return period is currently derived from a fixed storms-per-year constant rather than a stochastic annual occurrence process.^[67]

So if your belief is “this is a typical storm intensity model,” I’d say: yes, as a toy or first-pass marginal severity sampler; no, as a scientific TC event model for insurance.^[66] ^[67]

Statistical pieces missing

There are also a few modelling gaps in the probabilistic structure:

- No calibration layer: I do not see likelihood fitting, parameter estimation, or validation against observed historical intensities.^[68] ^[67]
- No explicit mixing weight beyond the implied normal exceedance at the threshold, which is elegant, but you still need empirical validation that this body-tail splice is appropriate for your basin and metric.^[67]
- No non-stationarity framework, apart from manually defined scenario families; there is no covariate-driven parameterization over climate state or time.^[68]
- No uncertainty decomposition between aleatory variability and parameter uncertainty.^[68] ^[67]
- No dependence modelling across events or years, e.g. ENSO/state-conditioned clustering.^[67]

For cat risk use, I would expect at least fitted parameters by basin/season and a validation pack covering exceedance curves, QQ plots, tail diagnostics and return-level stability.^[68] ^[67]

Coding issues I can already see

There appear to be several syntax or structural problems in the pasted files, unless formatting was lost during attachment parsing:

- In `distribution.py`, the `DistributionParameters(...)` call in `__init__` appears not to close properly before the seed logic begins.^[67]
- Multiple dictionaries and lists appear truncated or missing closing brackets / braces, including `to_dict()` and `summary_statistics()`.^[67]
- In `cli.py`, the formatted string appended in `compare_scenarios()` appears not to close properly, and the custom `IntensityDistribution(...)` constructor call also appears incomplete.^[66]
- The code uses `random.seed(seed)`, which mutates global RNG state; for reproducible scientific simulation it would be better to use an instance-level RNG object.^[67]
- `summary_statistics()` recomputes the sample mean repeatedly inside the variance expression, which is inefficient.^[67]
- The `sample()` method adds an extra Gaussian noise term after inverse transform sampling, which distorts the intended distribution and makes the analytic exceedance function inconsistent with sampled output.^[67]
- `return_period_intensity()` assumes a fixed `storms_per_year=20`, but there is no event-count model or basin-specific frequency logic behind that constant.^[67]

That last point is especially important: your analytic exceedance curve and your Monte Carlo samples are not fully aligned because of the added post-sampling jitter.^[67]

What I would add next

The minimum next upgrades I'd suggest are:

1. Define the intensity metric explicitly: max sustained wind, central pressure, gust, or normalized loss-driving wind index.^[68] ^[67]
2. Add conditioning dimensions: basin, month, distance to landfall, and environmental regime.^[68] ^[67]
3. Separate annual frequency from event severity with a Poisson or overdispersed event-count model.^[67]
4. Add a lifecycle model so intensity evolves over time rather than being one unconditional draw.^[67]
5. Add radius / footprint variables if the end goal is insurance loss, because scalar intensity alone is not enough.^[67]
6. Add calibration and backtesting tools.^[66] ^[67]
7. Fix code structure and unit-test the distribution identities: monotone exceedance, inverse consistency, finite moments, capped support, and reproducible seeded sampling.^[66] ^[67]

My assessment is that you have a decent starting **severity-distribution module**, but what is missing is the event-generation structure around it and some important internal consistency

and code-quality fixes.[\[66\]](#) [\[68\]](#) [\[67\]](#)

If you want, I can do the next pass as a line-by-line code review and identify the exact bugs plus a proposed refactor into: frequency model, conditional intensity model, and hazard-footprint module.

✱✱

Let me offload the other models. Here is hydrograph. Does this change your view. I am thinking of creating a typhoon as a separate storm or an extension of storm.

Yes — this does change my view somewhat. With the hydrograph files, the system now looks much more like an event-to-impact flood module built around pulse generation, antecedent saturation, superposition, travel-time lag, and property-level depth calculation, rather than just an isolated storm-intensity sampler.[\[69\]](#) [\[70\]](#) [\[71\]](#) [\[72\]](#)

What the architecture now looks like

You now appear to have two different layers:

- An intensity/severity layer in `distribution.py` and `parameters.py`, which samples a storm-level scalar intensity distribution.[\[73\]](#) [\[74\]](#)
- A hydrograph transformation layer in `pulse.py`, `gamma.py`, `saturation.py`, and `compound.py`, which converts one or more pulse peaks into a time series at gauge and then property level.[\[70\]](#) [\[71\]](#) [\[72\]](#) [\[69\]](#)

That is a sensible modular decomposition for insurance modelling, because event severity and flood response are related but not identical objects.[\[74\]](#) [\[69\]](#)

Does this change the “missing” assessment?

It improves the picture, but it does not fully change the earlier conclusion. You now have the beginnings of a temporal hazard-response engine, but the meteorological event model still seems under-specified.[\[72\]](#) [\[69\]](#) [\[74\]](#)

What is now present:

- Pulse hydrograph shape via a gamma-like template.[\[71\]](#) [\[70\]](#)
- Multi-pulse event sequencing.[\[69\]](#) [\[72\]](#)
- Antecedent precipitation saturation amplification.[\[72\]](#) [\[69\]](#)
- Travel-time lag and retention to propagate gauge conditions to property depth.[\[69\]](#)

What still seems missing at the storm level:

- A principled mapping from meteorological storm characteristics to `peak_m`, `duration_hours`, `start_hour`, `precip_mm`, and `sequence_type`.^[71] ^[69]
- Joint dependence between wind, pressure, translation speed, rain rate, storm size, and the hydrograph parameters.^[74] ^[69]
- Basin/storm-type conditioning so that typhoon-like events produce different pulse families and temporal signatures than ordinary storms.^[73] ^[69]

So the hydrograph module helps a lot, but it mainly strengthens the **downstream response side** rather than resolving the **upstream storm-generation side**.^[72] ^[69]

Typhoon: separate class or extension?

My view is: make typhoon a specialization of a more general storm event, not a completely separate top-level concept. In meteorological terms, a typhoon is a regional subtype of tropical cyclone, and in your codebase the more important distinction is likely not the label itself but the parameter regime it implies for intensity, duration, pulse count, rainfall loading, and temporal structure.^[74] ^[69]

So architecturally I would suggest:

- `StormEvent` as the generic event object.
- Subtypes or parameterized families such as `ConvectiveStorm`, `ExtratropicalStorm`, `TropicalCyclone`, and then `Typhoon` as either:
 - a subclass of `TropicalCyclone`, or
 - a `storm_type="typhoon"` configuration of `TropicalCyclone`.^[73] ^[69]

For your use case, I would lean toward a parameterized extension rather than a fully separate code branch, because most of the hydrograph machinery is reusable and the real difference is in sampled parameters and coupling rules, not in the propagation logic.^[70] ^[69]
^[72]

Why extension is better here

A typhoon should probably differ from a generic storm in:

- Higher peak intensity distribution or fatter tail.^[73] ^[74]
- Longer duration and possibly slower decay / broader pulse shape.^[70] ^[71]
- More precipitation and stronger antecedent saturation effects across sequential pulses.
^[69] ^[72]
- Potentially larger number of pulses or a different `sequence_type` prior.^[69]

Those differences can be handled cleanly by storm-family parameters without duplicating the hydrograph engine.^[71] ^[72] ^[69]

What I would add next

To make this robust, I'd define a single event schema feeding the hydrograph module, for example:

- `storm_family`
- `intensity`
- `precip_total`
- `duration_hours`
- `translation_speed`
- `n_pulses`
- `pulse_spacing_hours`
- `peak_stage`
- `sequence_type`
- `antecedent_wetness` [\[72\]](#) [\[74\]](#) [\[69\]](#)

Then write one mapping layer from storm family to hydrograph parameters:

- `intensity` → peak stage,
- `precipitation` → pulse amplitudes and saturation,
- `duration / speed / size` → gamma alpha, pulse width, and number of pulses. [\[70\]](#) [\[71\]](#) [\[72\]](#) [\[69\]](#)

That would let typhoon be an extension of storm with different priors, while keeping one common event-to-hydrograph pipeline. [\[72\]](#) [\[69\]](#)

Additional concerns

I still see coding and design issues in the attached snippets:

- Several imports appear syntactically incomplete in `compound.py`, `saturation.py`, and `infiltration.py`. [\[75\]](#) [\[69\]](#) [\[72\]](#)
- `compound.py` imports `apply_infiltration` but then explicitly bypasses infiltration, which suggests unresolved model-governance around whether retention already absorbs infiltration losses. [\[75\]](#) [\[69\]](#)
- `wse_m` reconstruction in `compound.py` looks brittle and may not preserve a clean physical interpretation outside flooded states. [\[69\]](#)
- `sequence_type` → `alpha` is a useful shortcut, but probably too compressed for typhoon modelling unless you also vary pulse count, duration and spacing distributions. [\[70\]](#) [\[69\]](#)

So overall: yes, the hydrograph files make the project look more coherent and more insurance-relevant, but they reinforce the idea that typhoon should be an **extension of a generic storm-event framework**, not a wholly separate object. [\[71\]](#) [\[70\]](#) [\[72\]](#) [\[69\]](#)

The next most useful step would be to sketch the domain model — classes or dataclasses for `StormEvent`, `TropicalCycloneEvent`, `Pulse`, and `HydrographResult` — before changing more code.

Here is the rest for floodrisk

This makes the overall picture much clearer: you now have a fairly complete flood risk engine around the hydrograph core, with spatial interpolation, propagation, depth-damage, portfolio analytics, and visualization layered on top.[\[77\]](#) [\[78\]](#) [\[79\]](#) [\[80\]](#) [\[81\]](#) [\[82\]](#)

Revised view

My view is now more positive on the flood side: the codebase is no longer just a toy storm-intensity sampler; it is becoming a modular flood hazard-to-loss framework.[\[79\]](#) [\[80\]](#) [\[81\]](#) [\[82\]](#) [\[77\]](#)
The weakest point is now even more clearly the upstream meteorological event generator and the consistency between modules, rather than the downstream risk analytics stack.[\[82\]](#) [\[83\]](#) [\[77\]](#)

What now looks strong

You have the major downstream components one would expect in an insurance-style flood module:

- Property hydrograph construction with pulse superposition, saturation, lag and retention.[\[80\]](#) [\[84\]](#) [\[85\]](#) [\[82\]](#)
- Spatial support with distance functions, polyline projection, IDW interpolation and correlation scaffolding.[\[79\]](#)
- Vulnerability / loss with depth-damage and BRI-based adjustment.[\[81\]](#)
- Portfolio analytics with concentration metrics, VaR/ES style outputs, and clustering.[\[77\]](#)
- Visualization hooks for both interactive and static outputs.[\[78\]](#)

That is a coherent hazard → vulnerability → portfolio workflow.[\[78\]](#) [\[81\]](#) [\[82\]](#) [\[77\]](#)

What still looks missing

The key missing layer is still the event-definition contract. Right now, several modules assume flood-event quantities such as peak WSE, pulse peaks, severe threshold exceedance, radius, center coordinates, and gauge readings, but I do not yet see one canonical event object that guarantees consistent semantics across all modules.[\[80\]](#) [\[81\]](#) [\[82\]](#) [\[78\]](#)

I would still say the main missing items are:

- A formal event schema, e.g. StormEvent, FloodEvent, Pulse, GaugeSeries, PropertyImpact.[\[82\]](#) [\[77\]](#) [\[80\]](#)
- A single upstream mapping from storm characteristics to flood forcing variables.[\[83\]](#) [\[82\]](#)
- Unit consistency and semantic consistency around stage, WSE, depth, exceedance above severe level, and floor-relative depth.[\[81\]](#) [\[79\]](#) [\[80\]](#) [\[82\]](#)

- Calibration and validation framework tying the simplified forms to observed hydrographs and losses.^{[77] [81] [82]}

Typhoon design decision

Given the full floodrisk stack, I would now be even more confident that **typhoon** should be an extension of **storm**, not a separate **parallel model family**. The floodrisk code is mostly hazard-agnostic downstream; what changes for a typhoon is the forcing distribution and pulse structure, not the spatial interpolation, velocity, retention, or damage modules.^{[79] [80] [81] [82] [77]}

So the clean design is:

- `StormEvent` Or `MeteorologicalEvent` base object.
- `TropicalCycloneEvent` as a subtype.
- `TyphoonEvent` as either a named subtype of tropical cyclone or a parameter regime flag within it.
- A translator layer that converts the event into one or more flood pulses and gauge/property hydrographs.^{[84] [85] [80] [82]}

That keeps all floodrisk modules reusable.^{[81] [82] [79]}

Main technical concerns

A few concerns still stand out:

- Several files appear syntactically incomplete in the attachment summaries, especially import blocks and some constructor calls, so I would not yet assume the package runs cleanly as shown.^{[86] [85] [80] [82] [77] [81]}
- `simulate_portfolio_impact()` perturbs direct impacts with Gaussian shocks scaled by a fixed 20 percent factor, which is convenient but not yet a physically grounded secondary uncertainty model.^[77]
- `build_correlation_matrix()` appears to use Cartesian distances on lon/lat degrees scaled by a constant, which may be acceptable locally but should be replaced with projected coordinates or geodesic distances if used more broadly.^[79]
- There are still unresolved semantics around infiltration vs retention, because infiltration is implemented separately yet explicitly bypassed in `compound.py`.^{[86] [82]}
- `build_property_hydrograph()` and `build_compound_property_hydrograph()` look like two overlapping pathways to property hydrographs, which risks duplicate logic and inconsistent results unless one becomes canonical.^{[80] [82]}

Best next step

The most valuable next move is not more code, but a domain model and interface cleanup. I would define one canonical pipeline:

1. Meteorological event generation.^[83]
2. Conversion to pulse/gauge forcing.^{[85] [84] [82]}
3. Propagation to property hydrograph.^{[82] [80]}
4. Spatial assignment/interpolation.^[79]
5. Vulnerability and loss.^[81]
6. Portfolio analytics and visualization.^{[78] [77]}

At this point, I'd describe the project as a promising modular flood catastrophe prototype with a reasonably strong downstream architecture, but still needing a formal upstream event model and tighter cross-module contracts.^{[80] [82] [77] [81] [79]}

The next review I'd recommend is a package-level architecture review: identify the canonical data structures, remove duplicate hydrograph paths, and define how a typhoon event populates pulse peaks, duration, precipitation, and spatial footprint.



here are the hazard models, will keep on pushing models until you have a full coverage

This is now enough to see the intended hazard layer much more clearly: you have a gauge-response model, extreme-value fitting, hazard-curve construction, I/O, and PRS pricing hooks, so the architecture is starting to span event generation through to tradable risk outputs.^{[88] [89] [90] [91] [92] [93] [94] [95]}

What these files add

The hazard package now looks like a distinct upstream block feeding the floodrisk stack: `response_model.py` maps storm attributes into gauge peak levels, `gev.py` fits GEV or Gumbel tails and builds term structures, and `builder.py` assembles these into per-gauge hazard curves.^{[90] [94] [88]}

`io.py` then operationalizes that pipeline from `storm_sequences.json` and `gauge.json` into `gaugehc.json` plus per-gauge storm response files, which makes the package look like a production-oriented workflow rather than just a set of helper functions.^[91]

Architectural reading

At this point, the end-to-end concept appears to be:

1. Generate storms and storm sequences.^{[96] [91]}
2. Translate each storm into gauge water-level responses.^[94]

3. Fit extreme value curves and derive annual exceedance probabilities and return levels.

[\[88\]](#) [\[90\]](#)

4. Use those hazard outputs to drive property hydrographs, damage, portfolio analytics, and PRS-style pricing. [\[92\]](#) [\[93\]](#) [\[95\]](#) [\[97\]](#) [\[98\]](#) [\[99\]](#)

That is a coherent vertical slice, and it means your model family is no longer “missing hazard models” in a broad sense; it now has a recognizable hazard engine, albeit a simplified one. [\[91\]](#) [\[94\]](#) [\[88\]](#)

Main strengths

The strongest improvement is the appearance of explicit typed data structures such as `GaugeResponse`, `HazardCurvePoint`, `TermStructurePoint`, and `GaugeHazardCurve`, because that begins to create the canonical contracts I said were missing earlier. [\[89\]](#)

I also like that the hazard builder is separated from pricing, with both `QuantLib`-based and analytical PRS approaches present, and that the terrain-grid repricing layer shows how hazard rates can flow into interactive property pricing. [\[93\]](#) [\[95\]](#) [\[92\]](#) [\[88\]](#)

Main concerns

The largest modelling weakness is still the response model itself: `GaugeResponseModel` derives “normal level” and response coefficients partly from random draws and simple precipitation-duration scaling, so the gauge physics and catchment hydrology are still stylized rather than calibrated. [\[94\]](#)

The second issue is that `builder.py` appears to build threshold points as `loc + threshold`, effectively anchoring hazard thresholds to the fitted GEV location parameter rather than to clearly defined physical water-level thresholds, while also capping exceedance probabilities below 1 percent to a 100-year floor; both choices may distort interpretation of the resulting hazard curve. [\[88\]](#)

A few additional concerns remain visible:

- `compute_term_structure()` treats annual hazard as a Poisson intensity for “at least one flood,” which may be fine for PRS-style pricing but is not the same thing as a physically independent annual event process unless that assumption is explicit. [\[90\]](#)
- `terrain_grid.py` prices spreads directly from EA zone rates multiplied by retention and elevation factors, which is useful operationally but is more a pricing surrogate than a hazard model in the scientific sense. [\[95\]](#)
- Some files still look syntactically truncated or incomplete in the attachment output, especially import blocks and dictionary/list constructions in `io.py`, `response_model.py`, and `terrain_grid.py`, so I still would not assume the package is execution-ready without cleanup. [\[95\]](#) [\[91\]](#) [\[94\]](#)

Coverage status

I'd now say you have broad conceptual coverage of the stack:

- Event generation and storm distributions.^[100] ^[96]
- Gauge hazard transformation and extreme value fitting.^[90] ^[94] ^[88]
- Flood propagation to property.^[97] ^[101]
- Vulnerability and loss.^[99]
- Portfolio analytics and visualization.^[98] ^[102]
- Pricing and repricing outputs.^[92] ^[93] ^[95]

What is still missing is not coverage so much as **hardening**: calibration, internal consistency, deterministic interfaces, and a cleaner separation between physical hazard, statistical hazard, and pricing approximations.^[93] ^[94] ^[95] ^[88] ^[90]

Design implication for typhoon

These hazard files reinforce the earlier conclusion that typhoon should still sit as a specialized storm-generation and response regime, not as a separate downstream flood framework. The gauge-response, GEV, hydrograph, damage, and pricing layers all look reusable if a typhoon changes only the storm parameter distribution, sequencing, spatial footprint, and perhaps the response-model calibration by basin or catchment.^[97] ^[96] ^[94] ^[88]

At this point, the most valuable next tranche for review would be the files that define storm sequence generation, parameter calibration, and any catchment-specific mappings from meteorology to runoff or gauge behavior, because that is now the dominant uncertainty in the architecture.^[96] ^[94] ^[88]

*
**

I like the simplified bit, this is one person against an entire industry. OK remember this is to simulate how such storms are created not to be the definitive solution. So the question is what I need to do to focus on typhoon wind speeds. Last set of models under stormgauge.

Yes — and with this last stormgauge set, the answer becomes much sharper: if your goal is to simulate how typhoon storms are created, not build a definitive full-industry cat model, then you should stop trying to perfect flood response first and instead make wind speed the primary state variable of the storm generator. The current stormgauge package is already close to the right abstraction for that, because it represents a storm as track + time-varying intensity + footprint + decay kernel, then maps that into gauge-level responses.^[103] ^[104] ^[105]

What to change

Right now your storm objects still treat intensity as a generic 0–100 scalar, with synthetic track profiles such as triangular, Gaussian, or beta envelopes, and the downstream model converts that abstract intensity into gauge effects. [\[104\]](#) [\[105\]](#) [\[103\]](#)

For typhoon wind-speed work, the key move is to replace that abstract intensity with **physically interpretable wind metrics**, so the `Storm` and `TrackPoint` objects should carry quantities like maximum sustained wind, central pressure, radius of maximum winds, translation speed, and perhaps wind-field asymmetry rather than just an intensity score. [\[103\]](#) [\[104\]](#)

In practical terms, the first refactor should be:

- `TrackPoint.intensity` becomes something like `max_wind_ms` or `vmax_10min_ms`. [\[103\]](#)
- `Storm.peak_intensity` becomes `peak_wind_ms`. [\[105\]](#) [\[103\]](#)
- `footprint_km` becomes a wind-field parameter set, for example `rmax_km`, `r34_km`, or a generic outer wind radius. [\[104\]](#) [\[103\]](#)
- `decay_kernel` stays, but now represents radial wind decay from the storm core rather than generic “storm influence.” [\[104\]](#) [\[103\]](#)

Best simplified architecture

For your purposes, I would separate the typhoon model into three simple layers:

Layer	What it should do	What to avoid
Storm generator	Create track, forward speed, peak wind, size, pressure, lifecycle. [105] [106]	Do not jump straight to river gauge levels. [104]
Wind-field model	Convert storm center + intensity + radius into local wind at any point. [104] [103]	Do not hide wind inside a 0–100 score. [103]
Impact translators	Map local wind into gauge response, surge proxy, rainfall proxy, or damage inputs. [104] [107]	Do not force one intensity scale to serve every peril. [107] [104]

That gives you a clean path where typhoon wind is the core hazard, and flood or gauge response becomes one optional downstream translation. [\[107\]](#) [\[104\]](#)

Why stormgauge matters

The important thing in `forward_model.py` is not the current equations, but the architecture: it already evaluates local hazard as a function of storm track proximity, storm state at time t , and spatial decay from the centerline. That is exactly the structure you want for a simplified typhoon wind model. [\[104\]](#)

What is wrong for typhoons is only the variable semantics: `compute_intensity_at_gauge()` currently interpolates a unitless track intensity and decays it with distance, then `intensity_to_level()` converts that into water level. For typhoon winds, you want the first part to output **local wind speed at the gauge**, and only afterward decide whether to convert that into water level, rainfall, or structural damage. [\[104\]](#)

So the main conceptual rewrite is:

1. `compute_intensity_at_gauge()` becomes `compute_wind_at_location()`.^[104]
2. `intensity_timeseries` becomes `wind_timeseries`.^{[103] [104]}
3. Gauge response becomes a downstream module, not the core storm model.^{[107] [104]}

Minimal typhoon model

You do **not** need a full Holland, Emanuel, or WRF-like framework to do this usefully. Given your stated goal, a reduced-form typhoon model is enough if it captures five things well:

- Track geometry.^[105]
- Peak wind speed over time.^{[105] [103]}
- Storm size or radial decay.^{[103] [104]}
- Forward speed and asymmetry, because moving storms are windier on one side.^{[105] [103]}
- Landfall weakening / extratropical decay.^{[106] [105]}

That means your storm factory should generate:

- Genesis location.
- Track polyline.
- Time-varying central wind.
- Time-varying storm size.
- Time-varying translation speed.
- A simple decay regime after landfall.^{[105] [103]}

What to build first

If the target is typhoon wind speeds, I would build in this exact order:

1. Canonical typhoon dataclass

Extend `Storm` and `TrackPoint` with wind-speed semantics and radius information.^[103]

2. Local wind field function

Replace the generic distance-decay intensity with a radial wind model such as: $\text{local wind} = \text{storm max wind} \times \text{radial decay} \times \text{asymmetry factor} \times \text{land/terrain modifier}$. The existing decay-kernel framework is already a good starting point for this.^[104]

3. Storm factory for typhoons

Update `create_storm()` so it samples peak wind, radius, speed, and lifecycle from typhoon-specific distributions rather than generic profiles.^{[108] [106] [105]}

4. Wind-only outputs first

Produce timeseries and peak wind footprints at locations before trying to infer gauge levels. This keeps the physics chain clearer.^[104]

5. Only then add translators

Add optional modules for rainfall proxy, surge proxy, gauge response, and damage

response.^[107] ^[104]

Specific recommendation

If I were narrowing your focus now, I would say:

- Keep `storm_factory.py`, but make it produce **typhoon parameters**, not generic storms.^[105]
- Keep `forward_model.py`, but redefine it as a **wind-field engine** rather than a gauge-level engine.^[104]
- Keep `data_structures.py`, but rename fields from “intensity” to “wind” and add radius and pressure metadata.^[103]
- Demote `response_model.py` to a downstream translator used only when you want flood/gauge outputs.^[107]

That gives you a much cleaner story: *simulate typhoon creation first, then derive local wind speeds, then optionally derive flood or pricing consequences.*^[105] ^[103] ^[104]

The single biggest step

The single biggest step you need is to stop using a universal 0–100 intensity scale as the model’s core latent state. It is acceptable for early experimentation, but once you say “focus on typhoon wind speeds,” the model needs a physically named state variable at its center, otherwise calibration, validation, and interpretation all stay fuzzy.^[106] ^[103] ^[105] ^[104]

So the short answer is: make wind speed the native state of the storm model, and make gauge/flood response a downstream consequence rather than the organizing principle.^[107] ^[104]

The next files I’d want to see after this are anything that defines landfall decay, track generation, basin assumptions, or historical typhoon parameter calibration, because that is where the typhoon specialization really begins.



OK, so the gauge measures is not relevant for typhoon as we can focus on the wind aspect of typhoon as an adjunct to the storm model.

Yes — that is the right simplification. For a typhoon extension, the gauge layer is not central; the storm model should produce wind fields first, and gauges should be treated as optional downstream consumers, or dropped entirely for the typhoon module.^[110] ^[111]

New focus

Your current `stormgauge` design mixes two ideas: a storm field model and a gauge-response model. The forward model already contains the useful part for typhoons — track interpolation, spatial decay, and time evolution — while the water-level conversion is the part you no longer need for a wind-focused adjunct.^[110]

So the clean move is:

- Keep Storm, TrackPoint, spatial decay, and storm factory concepts.^{[112] [111]}
- Drop GaugeConfig, GaugeResponse, and intensity_to_level() from the typhoon core.^{[111] [110]}
- Rename the forward model into something like TyphoonWindModel or StormWindFieldModel.^[110]

What the adjunct should output

Instead of outputting gauge peak levels, the typhoon adjunct should output local wind quantities at arbitrary coordinates over time. Your current model already computes storm influence at a point from track position, intensity, footprint, and decay, which is exactly the structure needed for this.^[110]

The primary outputs should be:

- Peak local wind speed.
- Wind-speed time series at a location.
- Distance from storm core at peak.
- Duration above wind thresholds.
- Optional footprint rasters or contour bands.^{[112] [110]}

Refactor path

The minimum refactor is straightforward:

1. Replace intensity with wind_speed in TrackPoint and Storm.^[111]
2. Replace compute_intensity_at_gauge() with compute_wind_at_location().^[110]
3. Replace IntensityDistribution with a wind-speed distribution, even if the mathematics initially stay the same.^{[113] [114]}
4. Move all gauge-specific dataclasses into a separate flood or river-response adapter module.^{[111] [110]}

That leaves the typhoon module as a clean adjunct to the storm model rather than a special case of gauge flooding.^{[112] [111] [110]}

Practical target

If you want a usable simplified typhoon model quickly, define the core typhoon state as:

- Track.
- Maximum sustained wind.
- Storm size / wind radius.
- Forward speed.

- Optional decay after landfall.^[112] ^[111]

Then let the model answer one central question: “Given storm parameters and location, what wind speed is experienced over time?” That is a much better primary objective than “what would a gauge read,” and it fits your stated aim of simulating storm creation rather than reproducing the whole industry stack.^[112] ^[110]

Recommended boundary

I’d define the boundary this way:

- Storm model core: genesis, track, lifecycle, wind distribution, wind field.^[114] ^[113] ^[111] ^[112]
- Optional adapters: flood gauge, rainfall proxy, surge proxy, damage module, pricing module.^[110]

So yes: the gauge measure is not relevant to the typhoon core, and treating typhoon wind as an adjunct hazard module attached to the broader storm framework is the cleanest design choice.^[112] ^[110]



OK, so we have a storm that has a centre point and fast windspeeds away from the centre, how would you consider this.

I would treat it as a moving radial wind field, not just a point with decay. Your current data structures already support a center track, a footprint size, and a decay kernel, so the natural interpretation is: at each time step the storm has a center, a peak wind near that center, and a wind-speed profile that falls away with distance from the center.^[115] ^[116] ^[117]

Conceptual model

The key idea is that the center point is only the storm’s location, not the place where the largest experienced wind is necessarily represented in a simplistic way. In your current framework, `Storm` has a `track`, `footprint_km`, and a `DecayKernel`, while the forward model already computes distance from a location to the storm center and applies spatial decay, which is exactly the scaffold for a simplified typhoon wind field.^[117] ^[115]

So I would define the local wind as something like:

$$V_{\text{local}}(t, x) = V_{\text{max}}(t) \times f(r) \times g(\theta) \times h(\text{land})$$

where $V_{\text{max}}(t)$ is storm-center wind over time, $f(r)$ is radial decay with distance from the center, $g(\theta)$ is an optional directional asymmetry term, and $h(\text{land})$ is an optional land weakening factor. The existing `DecayKernel` concept is already a first-pass version of $f(r)$.^[115] ^[117]

Simplified interpretation

For a first useful typhoon model, I would not put maximum wind exactly at the center, because the calm eye and eyewall structure matter conceptually even in a reduced model. Instead, I would treat the center as the reference location of the storm and define a radius of strongest winds around it, then let wind decay outside that radius.^[117] ^[115]

That suggests these parameters:

- Storm center: longitude, latitude, time.^[116] ^[115]
- Peak sustained wind $V_{\max}$.^[116] ^[115]
- Radius of maximum wind $R_{\max}$, which you do not yet have explicitly.^[115]
- Outer footprint or decay scale, which `footprint_km` can approximate for now.^[117] ^[115]
- Translation speed, already approximated by `speed_kmh`.^[116] ^[115]

Practical first formula

A very good simplified first version is:

- Inside a small eye radius: lower winds.
- Around $R_{\max}$: peak winds.
- Beyond $R_{\max}$: monotone decay using Gaussian, exponential, or power-law falloff.

In your current code terms, I would evolve `compute_decay()` from “distance from track gives intensity factor” into “distance from storm center gives wind factor,” with `footprint_km` acting as the outer decay scale and a new `rmax_km` field defining where the strongest winds sit.^[117] ^[115]

For example:

- $r < r_{\text{eye}}$: wind reduced.
- $r_{\text{eye}} \leq r \leq r_{\max}$: wind ramps up.
- $r > r_{\max}$: wind decays by kernel.

That is still simple, but much more typhoon-like than a center-point-only decay.^[117] ^[115]

Best design choice

So, if you say “a storm has a centre point and fast windspeeds away from the centre,” I would model it as an **annular intensity structure** riding along the storm track. The track gives the moving center, the time profile gives $V_{\max}(t)$, and the radial profile gives winds away from the center.^[116] ^[115] ^[117]

Concretely, I would update your `Storm` and `TrackPoint` schema to include:

- `max_wind_ms`
- `rmax_km`
- `outer_radius_km`

- optional `asymmetry_factor`
- optional `land_decay_factor` [\[115\]](#)

Recommended next step

The next coding step is to replace `intensity` in `TrackPoint` with a wind variable and split the current single decay profile into:

1. a temporal wind-evolution profile along the track, and
2. a radial wind profile around the center. [\[116\]](#) [\[117\]](#) [\[115\]](#)

That would turn the model from “storm center plus generic influence blob” into “moving typhoon wind field,” which is the right abstraction for what you are trying to build. [\[117\]](#) [\[115\]](#)

✱

Typhoon Wind Model Specification

Purpose

This specification defines a simplified typhoon wind model to be added as an adjunct to the existing storm model. The objective is to simulate how typhoon-like storms are created and how their wind fields evolve over time and space, without attempting to be a definitive full-industry tropical cyclone model. The current codebase already contains a storm track abstraction, a storm factory, and a spatial decay framework, but these are presently expressed through a generic intensity scale and a gauge-response workflow rather than a native wind-speed formulation. [\[118\]](#) [\[119\]](#) [\[120\]](#)

The new model should therefore shift the core representation from abstract intensity to physically named wind variables, and it should make local wind speed the primary model output. Gauge response, flood response, surge proxy, rainfall proxy, and damage response should be treated as optional downstream adapters rather than part of the typhoon core. [\[119\]](#) [\[118\]](#)

Scope

The typhoon wind module should answer one central question: given a storm track and storm state, what wind speed is experienced at a specified location over time? It should support both event simulation and scenario generation, including stress scenarios driven by different tail assumptions in the storm distribution layer. [\[121\]](#) [\[122\]](#) [\[119\]](#)

The module should support the following use cases:

- Generate synthetic typhoon events with realistic track geometry and lifecycle structure. [\[120\]](#)
- Compute local wind-speed time series at arbitrary coordinates. [\[119\]](#)
- Compute peak gust or sustained wind metrics at locations or on a grid. [\[119\]](#)

- Produce storm footprints and exceedance summaries suitable for later adapters such as damage or pricing.^[119]
- Remain deliberately simplified and calibratable, rather than attempting full dynamical atmospheric simulation.^{[118] [119]}

The module should not, in its first implementation, attempt to solve full fluid dynamics, model full eyewall thermodynamics, or produce definitive rainfall and flood outputs. Those should remain optional later translations built on top of the wind field.^[119]

Design principles

The design should follow five principles.

1. Wind speed is the native state variable of the typhoon model, not a generic 0–100 intensity score.^{[120] [118] [119]}
2. The storm center is a moving reference point, not the only place where maximum experienced wind is defined.^{[118] [119]}
3. The local wind field should be represented as a moving radial structure around the center, with optional asymmetry from storm translation.^[119]
4. The model should separate temporal evolution along the track from spatial decay away from the center.^{[120] [118] [119]}
5. Optional downstream modules may translate wind into flood, gauge, surge, or damage quantities, but those translations must not be embedded in the core wind model.^[119]

New model components

The coding team should build the following new or refactored model components.

Component	Purpose	Status relative to current code
TyphoonEvent model	Canonical event-level representation of a typhoon using wind variables rather than abstract intensity	New model required; current Storm is too generic. ^[118]
TyphoonTrackPoint model	Time-stamped storm center state including maximum wind, pressure, radius, and translation speed	Refactor of TrackPoint required. ^{[118] [120]}
TyphoonWindFieldModel	Computes local wind speed at any location and time	Refactor of StormGaugeModel core logic required. ^[119]
TyphoonFactory	Generates synthetic typhoon scenarios and track paths using wind-centric parameters	Refactor of create_storm () required. ^[120]

Component	Purpose	Status relative to current code
WindDistribution or TyphoonIntensityDistribution	Samples peak wind or lifecycle parameters, replacing the generic intensity distribution as the core hazard driver	Refactor of IntensityDistribution and DistributionParameters required. [121] [122]
WindFootprintExporter	Optional output layer for gridded footprints, thresholds, and event summaries	New output model recommended. [119]
Impact adapters	Optional converters from wind to damage, surge proxy, rainfall proxy, or other downstream measures	New downstream modules, outside typhoon core. [119]

Canonical event data model

The current Storm and TrackPoint classes provide a useful starting point because they already represent a moving center, a time axis, a footprint scale, and a decay kernel. However, they must be refactored so that the event is expressed in meteorological terms. [\[118\]](#) [\[120\]](#)

Required event-level fields

A new TyphoonEvent model should include at least:

- event_id
- name
- start_time
- duration_hours
- track_points
- basin
- peak_wind_ms Or peak_wind_10min_ms
- minimum_central_pressure_hpa (optional but strongly recommended)
- outer_radius_km
- landfall_segments or land-interaction metadata
- metadata

Required track-point fields

A new TyphoonTrackPoint model should include at least:

- longitude
- latitude

- `time_hours`
- `max_wind_ms`
- `central_pressure_hpa` (optional in first version)
- `rmax_km` (radius of maximum wind)
- `outer_radius_km`
- `translation_speed_kmh`
- `bearing_degrees`
- `land_fraction` OR `over_land` flag

This is a deliberate replacement for the current `TrackPoint.intensity` field and the current `Storm.peak_intensity` field.^[120] ^[118]

Mathematical structure

The typhoon wind model should separate temporal storm evolution from spatial wind-field structure.

Core wind field equation

At a location x and time t , local wind speed should be modelled as:

$$V_{\text{local}}(x, t) = V_{\text{max}}(t) \cdot f_r(r, t) \cdot f_a(\theta, t) \cdot f_l(t) \cdot f_s(x)$$

Equation (1)

Where:

- $V_{\text{max}}(t)$ is the maximum sustained wind associated with the storm at time t .
- $f_r(r, t)$ is the radial wind profile as a function of distance r from the storm center.
- $f_a(\theta, t)$ is an asymmetry factor that can increase winds on the forward-right side or the storm-motion side.
- $f_l(t)$ is a land weakening factor.
- $f_s(x)$ is an optional static surface or terrain adjustment, which may initially be set to 1.0.

The first implementation may simplify this by omitting f_s and using a basic form of f_a , but the interfaces should still reserve a place for them.^[119]

Temporal evolution model

The current storm factory already creates a track and an intensity profile using triangular, Gaussian, or beta-shaped temporal envelopes. The typhoon model should retain the idea of an intensity profile, but it should apply it to maximum wind rather than abstract intensity.^[118] ^[120]

For a track point sequence indexed by time, define:

$$V_{\text{max}}(t) = V_{\text{peak}} \cdot p(t)$$

Equation (2)

Where $p(t)$ is a normalized lifecycle profile with values in $[0,1]$.

Recommended profile options:

- Triangular: rapid intensification and then decay.
- Gaussian: smooth rise and fall around peak.
- Beta-like: asymmetric rise and decay if the team wants a more realistic lifecycle shape.^[120]

The implementation should preserve the profile abstraction already present in `IntensityProfile`, but rename or reinterpret it as a wind lifecycle profile rather than a generic intensity profile.^{[118] [120]}

Spatial wind-field structure

The current model uses a single decay kernel from the storm center to a location. For typhoon winds, this should be expanded into an inner-core and outer-core structure.^[119]

Required radial zones

The radial profile should distinguish at least three regions:

1. Eye or inner-core region, where winds are reduced relative to the eyewall.
2. Radius of maximum wind region, where local wind is highest.
3. Outer decay region, where wind declines with distance.

Recommended simplified radial profile

A practical first version is a piecewise radial function:

$$f_r(r, t) = \begin{cases} \alpha_{\text{eye}} + (1 - \alpha_{\text{eye}}) \frac{r}{R_{\max}(t)} & 0 \leq r < R_{\max}(t) \\ \exp \left(- \left(\frac{r - R_{\max}(t)}{L(t)} \right)^p \right) & r \geq R_{\max}(t) \end{cases}$$

Equation (3)

Where:

- $R_{\max}(t)$ is the radius of maximum wind.
- $L(t)$ is an outer decay length scale.
- p controls how sharply winds decay.
- α_{eye} is the inner-core floor ratio, for example between 0.1 and 0.4.

This formulation gives lower wind near the center, highest wind near the eyewall, and smooth decay outside. It is a substantial improvement over a pure center-point blob while remaining much simpler than a full parametric cyclone model.^[119]

Alternative kernel approach

If the team wants to stay closer to the existing `DecayKernel` abstraction, the implementation may define:

$$f_r(r, t) = K \left(\frac{r}{R_{\text{scale}}(t)} \right)$$

Equation (4)

Where K is one of Gaussian, exponential, or linear kernels. However, this should only be used for the outer region, not as the whole radial structure, because typhoon winds are not best represented as simply peaking at the center.^{[118] [119]}

Translation asymmetry

The existing model already computes storm track position over time and knows the average storm speed. This should be extended into a directional wind asymmetry term.^{[120] [119]}

A simple first-order asymmetry factor is:

$$f_a(\theta, t) = 1 + a(t) \cos(\theta - \theta_m(t))$$

Equation (5)

Where:

- θ is the bearing from the storm center to the target location.
- $\theta_m(t)$ is the direction of storm motion.
- $a(t)$ is an asymmetry coefficient, usually increasing with translation speed.

A very simple parameterisation is:

$$a(t) = \min \left(a_{\text{max}}, c_a \cdot \frac{U(t)}{V_{\text{max}}(t) + \epsilon} \right)$$

Equation (6)

Where $U(t)$ is the translation speed and c_a is a calibration constant.

This keeps the model simple while allowing stronger winds on the forward side of motion, which is important for realistic typhoon footprints.^{[120] [119]}

Land weakening

The storm factory currently computes tracks without explicit land interaction. The typhoon model should include a simple land weakening function once a storm spends time over land.^[120]

A suitable first-order model is exponential decay:

$$V_{\max}(t + \Delta t) = V_{\max}(t) \exp(-k_{\text{land}}\Delta t)$$

Equation (7)

When the storm center is over land, or when a land-fraction indicator exceeds a threshold, the decay rate k_{land} should be applied. The same logic may optionally increase the outer decay rate or shrink the outer wind radius over land.^[120]

Event generation requirements

The current `create_storm()` function generates straight-line tracks and lifecycle profiles. The typhoon factory should preserve the simplicity of that design but extend it to generate wind-centric scenarios.^[120]

Inputs to event generation

The new `TyphoonFactory` should accept or generate:

- Basin or region.
- Genesis point.
- Initial bearing.
- Track curvature or waypoints.
- Duration.
- Peak wind speed.
- Radius of maximum wind.
- Outer radius or decay scale.
- Translation speed.
- Lifecycle profile type.
- Landfall timing or land mask interaction.

Minimum generation workflow

1. Generate a track polyline in space and time.^[120]
2. Generate a lifecycle curve for maximum wind along the track.^[120]
3. Generate size evolution, at `minimum_rmax_km` and `outer_radius_km` over time.
4. Generate translation speed and bearing over time, either implied by the track or explicitly stored.^[120]
5. Apply land weakening after landfall or partial land interaction.
6. Export the event in a canonical serializable typhoon format.

Distribution model requirements

The current distribution module uses a hybrid truncated-normal plus Pareto-tail structure on an abstract intensity scale. That is acceptable as a starting mathematical idea, but the variable should be renamed and reinterpreted as wind speed.^{[122] [121]}

The team should build a wind-centric distribution layer with at least the following uses:

- Sample peak sustained wind.
- Support stress scenarios by changing tail thickness.
- Support return-period calculations for event severity.^{[121] [122]}

Recommended distribution abstraction

Replace the current generic intensity variable I with peak wind V_{peak} . The same hybrid logic may be retained in the first version:

- Below threshold: truncated normal baseline.
- Above threshold: Pareto tail.^[121]

Formally:

$$P(V > v) = \begin{cases} 1 - \Phi\left(\frac{v - \mu}{\sigma}\right), & v \leq v_T \\ P(V > v_T) \left(\frac{v_T}{v}\right)^\alpha, & v > v_T \end{cases}$$

Equation (8)

Where:

- μ is baseline mean wind.
- σ is baseline standard deviation.
- v_T is the tail threshold.
- α is the Pareto tail index.^{[122] [121]}

The distribution parameters should therefore be renamed from generic intensity terms to wind-severity terms, but the scenario-family concept may be retained because it is already useful for stress testing.^[122]

Outputs required from the wind model

The typhoon core should expose outputs at two levels.

Event-level outputs

- Peak storm wind.
- Minimum central pressure, if modeled.
- Peak storm size metrics such as `rmax_km` and `outer_radius_km`.
- Track geometry and timing.^[118] ^[120]

Location-level outputs

For any queried location, the model should produce:

- Wind-speed time series.
- Peak sustained wind.
- Time of peak wind.
- Duration above user-defined thresholds.
- Distance from center at peak.
- Optional directional metadata, such as storm-relative quadrant.^[119]

Grid-level outputs

For a regular grid or polygon set, the model should optionally produce:

- Peak wind footprint.
- Threshold exceedance masks.
- Event-integrated wind severity metric.
- Footprint summaries for downstream risk models.^[119]

Downstream adapters

The typhoon core should not convert wind directly into gauge levels. Instead, the system should define optional downstream adapters.^[119]

Recommended adapters include:

- Wind-to-damage adapter.
- Wind-to-rainfall proxy adapter.
- Wind-to-surge proxy adapter.
- Wind-to-flood forcing adapter.
- Wind-to-pricing event summary adapter.

Each adapter should take typhoon outputs as input and should be implemented outside the core wind model boundary.^[119]

Refactoring requirements against current files

`data_structures.py`

Required changes:

- Replace `TrackPoint.intensity` with `max_wind_ms` or equivalent. [\[118\]](#)
- Replace `Storm.peak_intensity` with `peak_wind_ms`. [\[118\]](#)
- Add `rmax_km`, `outer_radius_km`, and optional central pressure fields to track points and/or the event. [\[118\]](#)
- Remove gauge-only assumptions from the typhoon core data model. [\[118\]](#)

`forward_model.py`

Required changes:

- Rename `StormGaugeModel` to `TyphoonWindFieldModel` or similar. [\[119\]](#)
- Replace `compute_intensity_at_gauge()` with `compute_wind_at_location()`. [\[119\]](#)
- Retain the current position interpolation logic but change the output variable from generic intensity to local wind speed. [\[119\]](#)
- Remove `intensity_to_level()` from the typhoon core. [\[119\]](#)
- Add radial wind-field logic with `rmax_km` and outer decay. [\[119\]](#)
- Add optional asymmetry using translation direction and speed. [\[119\]](#)

`storm_factory.py`

Required changes:

- Replace `peak_intensity` with `peak_wind_ms`. [\[120\]](#)
- Generate time-varying wind and size fields, not just generic intensity along track. [\[120\]](#)
- Preserve profile choices but reinterpret them as lifecycle profiles for peak wind. [\[120\]](#)
- Add landfall-aware decay hooks. [\[120\]](#)

`distribution.py` **and** `parameters.py`

Required changes:

- Rename the abstraction from intensity distribution to wind-severity distribution. [\[121\]](#) [\[122\]](#)
- Preserve scenario families and tail controls as a stress-testing mechanism. [\[122\]](#) [\[121\]](#)
- Define units and semantics explicitly, for example 10-minute sustained wind in m/s or km/h. [\[122\]](#)

Calibration requirements

The first implementation should be explicitly labelled as simplified but calibratable. Calibration should occur in layers.

1. Peak wind distribution calibration: fit μ, σ, v_T, α to a chosen basin dataset or expert judgment.^{[121] [122]}
2. Lifecycle calibration: fit duration and profile shape parameters to representative typhoon events.^[120]
3. Wind-field calibration: fit r_{max_km} , outer decay, and asymmetry parameters to observed footprint behavior or benchmark storm cases.^[119]
4. Landfall decay calibration: fit k_{land} to a simple weakening rule after landfall.^[120]

The coding team should ensure these are configuration-driven rather than hard-coded wherever possible.^[122]

Validation requirements

Validation should be simple, interpretable, and aligned with the reduced-form goal.

The team should validate:

- Peak local wind at selected observation points for benchmark storms.
- Plausibility of footprint shape under different decay kernels.^{[118] [119]}
- Sensitivity to storm size, peak wind, and translation speed.^{[120] [119]}
- Return-period severity outputs under the wind distribution scenarios.^{[121] [122]}
- Stability of outputs under different time resolutions and track-point densities.^{[120] [119]}

Non-goals

The following items are explicitly out of scope for the first release:

- Full atmospheric dynamics.
- Full pressure-wind coupled cyclone models.
- Full surge hydrodynamics.
- Full rainfall-runoff coupling.
- Automatic gauge water-level outputs in the typhoon core.^[119]

These may be added later as optional layers, but they should not define the first architecture.^[119]

Implementation summary

The coding team should treat the typhoon extension as a new wind-field model built on the existing storm abstractions. The existing code already provides useful building blocks: a moving storm center, lifecycle profiles, a factory pattern, and spatial decay logic. The required architectural shift is to replace abstract intensity with wind-speed variables, replace gauge-centered outputs with location-centered wind outputs, and introduce a radial wind profile with optional translation asymmetry and land weakening.^{[118] [120] [119]}

This should be considered the minimum viable scientific architecture for a simplified typhoon model: physically named state variables, explicit radial structure, explicit lifecycle structure, and a clean separation between the typhoon core and downstream impact translations.^{[118] [120] [119]}

✱

how does the typhoon move forward throughout the time horizon. Looking for a Bayesian progression.

A good Bayesian progression model treats the typhoon's forward motion as a **state-space process**: at each time step, the storm has an unobserved state such as position, heading, translation speed, wind and size, and that state is updated sequentially using priors plus new information. Bayesian tropical cyclone work commonly uses track-pattern clustering, Bayesian regression, Bayesian model averaging, or Bayesian interval prediction to express the storm path as a probability distribution rather than a single deterministic line.^{[123] [124] [125] [126]}

State model

For your simplified simulator, the cleanest setup is to define the latent state at time t as:

$$s_t = (\lambda_t, \phi_t, u_t, \psi_t, V_t, R_t)$$

Equation (1)

where λ_t, ϕ_t are longitude and latitude, u_t is forward speed, ψ_t is heading, V_t is peak wind and R_t is storm size. This matches your existing architecture well, because you already have a time-indexed track, storm speed, and footprint concept, but they are currently handled in a more deterministic way.^{[127] [128] [129]}

A first-order Markov evolution is then:

$$p(s_t \mid s_{t-1}, z_t, \theta)$$

Equation (2)

where z_t is environmental forcing such as steering flow, SST, wind shear, or land interaction, and θ is a parameter set. In Bayesian terms, each new step updates the posterior distribution

of the next storm state rather than fixing one path in advance.^{[124] [125] [130]}

Forward movement

The simplest forward motion model is to update heading and speed separately, then advect the center. Your propagation step can be:

$$u_t \sim \mathcal{N}(\mu_u(s_{t-1}, z_t), \sigma_u^2)$$
$$\psi_t \sim \mathcal{N}(\mu_\psi(s_{t-1}, z_t), \sigma_\psi^2)$$

Equation (3)

and then move the center by:

$$\Delta x_t = u_t \Delta t \cos(\psi_t), \quad \Delta y_t = u_t \Delta t \sin(\psi_t)$$

Equation (4)

with appropriate spherical conversion from km to lat/lon. This gives a stochastic track model in which forward motion is uncertain but still centered on physically plausible steering behavior.^{[125] [126]}

In practice, μ_u and μ_ψ should depend on:

- Previous speed and heading.
- Track type or regime.
- Land or sea flag.
- Latitude-dependent recurvature tendency.
- Optional large-scale steering variables.^{[131] [130] [124]}

Bayesian update

The Bayesian part is that you do not just sample from fixed rules; you update beliefs about the next state from prior plus likelihood:

$$p(s_t \mid y_{1:t}) \propto p(y_t \mid s_t) p(s_t \mid y_{1:t-1})$$

Equation (5)

where y_t is whatever information you condition on at time t , such as observed position, forecast guidance, wind field data, or simply the internally generated prior constraints. This is the core idea behind Bayesian sequential forecasting and interval prediction for tropical storms.^{[123] [125]}

For your simulator, if you are not assimilating real-time observations, then y_t can still be a “pseudo-observation” layer consisting of soft constraints:

- Storm should not jump unrealistically.

- Heading changes should be smooth.
- Landfall should weaken and slow the storm.
- Certain track clusters are more likely in certain regions.^[130] ^[124]

Track regimes

A very useful Bayesian simplification is to introduce a hidden regime variable k , representing track class. Bayesian typhoon studies have clustered historical tracks into a finite number of patterns and then fit separate Bayesian models by cluster, which is a strong fit for your use case because it keeps the model interpretable.^[132] ^[124] ^[130]

That gives:

$$p(s_t | s_{t-1}) = \sum_k p(s_t | s_{t-1}, k) p(k | \text{history})$$

Equation (6)

So instead of one universal movement rule, you can have regimes such as:

- Westward straight runner.
- Northwest recurver.
- Slow looper.
- Landfall decay track.^[124] ^[130]

This is probably the best “Bayesian progression” for a coding team because it is much easier to implement than a full dynamical Bayesian cyclone model, while still producing probabilistic path evolution.^[123] ^[124]

Recommended implementation

For your codebase, I would recommend this progression model:

1. Add a hidden `track_regime` variable to the typhoon event.
2. At $t = 0$, sample regime probabilities from basin/location priors.
3. For each step, update speed and heading from regime-specific conditional distributions.
4. Move the center forward using the sampled speed and heading.
5. Update wind and radius jointly with land/ocean effects.
6. Carry uncertainty by simulating many particles or paths rather than one track.^[129] ^[125] ^[127]
^[124]

A practical particle version would be:

- Start with N possible storm states.
- Propagate each one forward with noisy speed/heading updates.
- Reweight by plausibility or any observations.

- Resample.
- Use the resulting cloud as the posterior track distribution.^{[133] [125]}

Best spec direction

So the spec for “how the typhoon moves forward” should describe it as a **Bayesian sequential state-space model with regime-conditioned motion**. That is more robust than a single deterministic track generator and much more aligned with probabilistic tropical cyclone forecasting literature than simply drawing a line between start and end points.^{[126] [125] [124] [123]}

The most implementation-friendly version is:

- Latent state: position, speed, heading, wind, size.^[127]
- Regime prior: clustered track family.^{[130] [124]}
- Transition model: stochastic update for heading and speed.^{[125] [126]}
- Update method: particle filter or sequential Monte Carlo.^[125]
- Output: posterior distribution of future center points and associated wind footprints.^{[123] [125]}

Would you like me to turn this into a second spec document for the coding team, focused specifically on Bayesian typhoon track progression?

✱

Bayesian Typhoon Track Progression Specification

Purpose

This specification defines a Bayesian progression model for advancing a typhoon through time. The objective is to provide the coding team with a practical probabilistic architecture for simulating forward typhoon motion across a time horizon, with uncertainty represented explicitly rather than through a single deterministic path.^{[141] [142] [143] [144]}

The model is intended for a simplified simulation framework rather than a definitive operational forecast system. It should therefore be interpretable, modular, easy to calibrate, and compatible with the existing storm abstractions already present in the codebase, especially the time-indexed track representation, storm speed metadata, and footprint logic.^{[145] [146] [147]}

Objective

The model should answer the following question:

Given the storm state at time $t-1$, and any environmental or observational information available at time t , what is the posterior distribution of the typhoon state at time t ?^{[142] [143] [144]}

The model should produce a probabilistic sequence of storm center locations, headings, forward speeds, peak winds, and size metrics over the chosen time horizon. It should support both free simulation and sequential updating when new information becomes available.^[144]
^[148]

Design principles

The Bayesian progression model should satisfy the following principles.

1. Storm movement should be represented as a stochastic state process, not a fixed line segment between start and end points.^[148] ^[144]
2. The model should update the storm state sequentially over time using Bayes' rule.^[142]
^[144]
3. Forward motion should be driven by physically meaningful state variables, including position, heading, translation speed, wind intensity, and storm size.^[146] ^[145]
4. The model should allow uncertainty in both motion and structure, not only in one-dimensional track position.^[144] ^[148]
5. The first release should prioritize interpretability and calibration over full meteorological completeness.^[143] ^[149]

Model overview

The recommended design is a Bayesian state-space model. The typhoon is described by an unobserved latent state that evolves over time, while optional data or soft constraints provide likelihood information for updating the posterior state distribution.^[148] ^[142] ^[144]

At each time step the workflow is:

1. Start with the posterior distribution from the previous step.
2. Propagate that state forward using a transition model.
3. Evaluate any new information or plausibility constraints.
4. Update the state distribution using Bayes' rule.
5. Continue until the required time horizon is reached.^[142] ^[144]

State definition

The latent state at time t should be defined as:

$$s_t = (\lambda_t, \phi_t, u_t, \psi_t, V_t, R_t, k_t)$$

Equation (1)

Where:

- λ_t is longitude.
- ϕ_t is latitude.

- u_t is translation speed.
- ψ_t is heading or motion bearing.
- V_t is maximum sustained wind at the storm center.
- R_t is a size vector, at minimum including radius of maximum wind and outer wind radius.
- k_t is an optional track regime or hidden motion class.^{[147] [143] [145] [146]}

A more explicit size vector may be defined as:

$$R_t = (R_{\max,t}, R_{\text{outer},t})$$

Equation (2)

The state should be serializable and compatible with the typhoon event model proposed in the wind-field specification.^[145]

Regime variable

A discrete regime variable should be included because Bayesian typhoon work often benefits from track-pattern categorization. Historical storms tend to fall into recurring path families, and modeling motion conditional on regime is usually more stable and interpretable than using one global motion rule.^{[150] [151] [143]}

Suggested regimes include:

- Straight westward track.
- Northwest-moving recurver.
- Sharp recurvature track.
- Slow or stalled track.
- Landfall decay track.^{[143] [150]}

The regime may be fixed for an event or may evolve with low transition probability over time. The simplest first version should sample it once at storm genesis and hold it fixed unless evidence strongly suggests a transition.^{[150] [143]}

Bayesian formulation

The sequential Bayesian update is:

$$p(s_t \mid y_{1:t}) \propto p(y_t \mid s_t) p(s_t \mid y_{1:t-1})$$

Equation (3)

Where:

- s_t is the latent state at time t .
- y_t is information available at time t , including observations, environmental guidance, or synthetic plausibility constraints.

- $p(s_t \mid y_{1:t-1})$ is the predictive prior from the transition model.^{[144] [142]}

The predictive prior is obtained from:

$$p(s_t \mid y_{1:t-1}) = \int p(s_t \mid s_{t-1}) p(s_{t-1} \mid y_{1:t-1}) ds_{t-1}$$

Equation (4)

This is the canonical state-space formulation and should be the conceptual basis for the implementation.^{[148] [144]}

Transition model

The transition model governs how the typhoon moves from one step to the next. It should update motion, wind, and size jointly, or at least in coupled blocks.

Motion update

The recommended first-order motion block is:

$$\begin{aligned} u_t &\sim \mathcal{N}(\mu_u(s_{t-1}, z_t, k_t), \sigma_u^2(k_t)) \\ \psi_t &\sim \mathcal{N}(\mu_\psi(s_{t-1}, z_t, k_t), \sigma_\psi^2(k_t)) \end{aligned}$$

Equation (5)

Where:

- z_t is environmental forcing or steering information.
- k_t is the regime variable.^{[143] [144] [148]}

The conditional means should depend on at least:

- Previous speed and heading.
- Position and latitude.
- Regime class.
- Land or ocean interaction.
- Recurvature tendency at higher latitudes.^{[149] [150] [143]}

Position update

Given translation speed and heading, the center should be advanced using a local advection rule. In planar approximation:

$$\Delta x_t = u_t \Delta t \cos(\psi_t), \quad \Delta y_t = u_t \Delta t \sin(\psi_t)$$

Equation (6)

And then:

$$\lambda_t = \lambda_{t-1} + c_\lambda \Delta x_t, \quad \phi_t = \phi_{t-1} + c_\phi \Delta y_t$$

Equation (7)

Where c_λ and c_ϕ convert km displacement into longitude and latitude increments. The implementation may use Haversine or geodesic utilities for better geographic fidelity, as the current code already includes distance handling logic through Haversine calculations.^{[146] [147]}

Wind update

The wind state should evolve with persistence and environmental moderation:

$$V_t \sim \mathcal{N}(\mu_V(V_{t-1}, z_t, k_t), \sigma_V^2)$$

Equation (8)

The mean function may include:

- Persistence from previous wind.
- Intensification tendency over warm water.
- Weakening due to land interaction.
- Optional decay after recurvature or extratropical transition.

A simple landfall weakening rule may be:

$$V_t = V_{t-1} \exp(-k_{\text{land}} \Delta t)$$

Equation (9)

When the storm is over land or land fraction exceeds a threshold.^[147]

Size update

The storm size state should evolve stochastically but smoothly:

$$R_{\text{max},t} \sim \mathcal{N}(\mu_{R_{\text{max}}}(R_{\text{max},t-1}, V_t, z_t), \sigma_{R_{\text{max}}}^2)$$

$$R_{\text{outer},t} \sim \mathcal{N}(\mu_{R_{\text{outer}}}(R_{\text{outer},t-1}, V_t, z_t), \sigma_{R_{\text{outer}}}^2)$$

Equation (10)

This keeps the wind-field model structurally coupled to the motion model without requiring a full pressure-wind system.^{[145] [146]}

Regime-conditioned mixture model

The transition model should be regime-conditioned. Formally:

$$p(s_t | s_{t-1}) = \sum_k p(s_t | s_{t-1}, k) p(k | s_{t-1})$$

Equation (11)

In the simplest implementation, the event is assigned a regime at genesis:

$$k \sim \text{Categorical}(\pi_1, \dots, \pi_K)$$

Equation (12)

Where the prior weights π_k depend on genesis location, season, and perhaps initial heading.^{[151] [150] [143]}

This is strongly recommended because it provides a principled Bayesian way to encode broad path behavior without forcing the coding team into a highly nonlinear global model.^{[150] [143]}

Likelihood model

The likelihood $p(y_t | s_t)$ may be fed by one of three sources.

1. Real observations

If real-time observations are available, the likelihood may use observed storm center positions, observed headings, forecast advisory data, or wind estimates.^{[142] [144]}

2. External guidance

If external model guidance exists, the likelihood may represent agreement with a forecast ensemble or consensus track, following a Bayesian model averaging style approach.^[142]

3. Soft constraints for simulation mode

If the model is running as a synthetic simulator, the likelihood may be replaced by plausibility scores. Examples include:

- Penalize unrealistically sharp heading changes.
- Penalize implausible jumps in translation speed.
- Penalize motion over terrain inconsistent with the chosen regime.
- Penalize trajectories that violate basin boundaries or historical behavior.

This allows the same Bayesian machinery to be used even when there are no real observations to assimilate.^{[149] [143]}

Inference method

The recommended implementation method is Sequential Monte Carlo, also known as a particle filter. This is the most natural way to represent uncertainty in a nonlinear storm state process.^[144]

Particle representation

At time t , represent the posterior as a weighted set of particles:

$$p(s_t | y_{1:t}) \approx \sum_{i=1}^N w_t^{(i)} \delta(s_t - s_t^{(i)})$$

Equation (13)

Where:

- $s_t^{(i)}$ is particle i .
- $w_t^{(i)}$ is its normalized weight.^[144]

Particle filtering workflow

1. Initialize particles from the genesis prior.
2. Propagate each particle using the transition model.
3. Compute weights from the likelihood or plausibility function.
4. Normalize weights.
5. Resample if the effective sample size falls below a threshold.
6. Repeat for the next time step.^[144]

This should be the default implementation. A deterministic mean-path fallback may be added later, but the probabilistic particle version should be considered the canonical Bayesian model.^[144]

Initialization

At genesis time $t=0$, the prior should define distributions for:

- Genesis longitude and latitude.
- Initial heading.
- Initial translation speed.
- Initial maximum wind.
- Initial size.
- Regime class.^{[152] [153] [147] [143]}

The current distribution module already provides a scenario-family concept with adjustable tail thickness. That concept should be retained and reinterpreted so that peak wind, duration,

and perhaps initial translation speed can be sampled from scenario-conditioned priors.^[153]
^[152]

Environmental forcing

The state transition functions may accept an environmental vector $\mathbf{z}_t$. The first version may keep this simple.

Recommended optional forcing variables:

- Latitude band or recurvature proxy.
- Ocean versus land flag.
- Basin-specific steering tendency.
- Seasonal modifier.
- Optional shear or SST proxy.

If environmental data are unavailable, the model may still work by using regime-conditioned priors plus position-based heuristics.^[149] ^[143]

Output requirements

The Bayesian progression model should produce the following outputs.

Posterior trajectory outputs

- Mean or median storm center path.
- Credible bands or confidence envelopes for location over time.
- Ensemble of particle tracks.
- Posterior distributions for heading and translation speed.^[142] ^[144]

Wind-state outputs

- Posterior mean and quantiles for maximum wind over time.
- Posterior mean and quantiles for storm size metrics over time.
- Optional scenario-conditioned severity summaries.^[146] ^[145]

Event summaries

- Probability of landfall in each region.
- Probability of recurvature.
- Probability of exceeding wind thresholds at chosen horizons.
- Time-to-landfall or time-to-closest-approach distributions.^[142] ^[144]

Interface with the typhoon wind-field model

The Bayesian progression model should feed directly into the typhoon wind-field model from the earlier specification. Each posterior particle or posterior mean state at time t should be usable as an input to the radial wind-field module, which then computes location-specific winds. [\[145\]](#) [\[146\]](#)

The interface contract should therefore expose, at each time step:

- Center position.
- Translation speed.
- Heading.
- Maximum sustained wind.
- Radius of maximum wind.
- Outer wind radius.
- Land interaction flags.

This ensures the motion model and the wind-field model remain decoupled but interoperable. [\[146\]](#) [\[145\]](#)

Required new models and modules

Module	Purpose	Notes
BayesianTyphoonState	Canonical latent state container	Should include motion, wind, size, and regime. [145]
TrackRegimeModel	Regime priors and regime-conditioned transition parameters	Strongly recommended for interpretability. [143] [150]
MotionTransitionModel	Updates heading and translation speed	Core of forward progression. [144] [148]
IntensityTransitionModel	Updates maximum wind through time	Should support land weakening. [147]
SizeTransitionModel	Updates wind radii through time	Needed by wind-field model. [145] [146]
ObservationLikelihoodModel	Scores particles against observations or plausibility constraints	Must support observation-free simulation mode. [142] [144]
ParticleFilterEngine	Sequential Monte Carlo implementation	Recommended default inference engine. [144]
PosteriorExporter	Writes trajectories, credible regions, and summaries	Output-facing module

Refactoring guidance for current codebase

`storm_factory.py`

The current storm factory builds deterministic tracks and intensity profiles. It should be refactored into a prior generator that samples an initial state, regime, and track skeleton rather than a fully fixed event.^[147] ^[143]

`data_structures.py`

The current data structures are event-oriented and use generic intensity fields. They should be extended to support a Bayesian state object, regime variables, posterior particles, and explicit wind-based semantics.^[145]

`forward_model.py`

The current forward model interpolates deterministic state along a track and computes local decay from the center. This logic should remain available, but it should become the downstream consumer of posterior state sequences produced by the Bayesian progression engine.^[146]

Calibration requirements

Calibration should proceed in layers.

1. Regime identification from historical track families.^[151] ^[143] ^[150]
2. Conditional motion parameter fitting by regime for heading and speed updates.^[143] ^[144]
3. Wind persistence and land decay fitting.^[147]
4. Size evolution fitting, at least through heuristic rules initially.^[145] ^[146]
5. Likelihood function tuning for observation-free simulation plausibility.^[149] ^[143]

All parameters should be externalized into configuration files or parameter objects rather than embedded in code constants.^[153]

Validation requirements

The coding team should validate the Bayesian progression model against the following criteria.

- Plausible particle-track spread through time, widening with forecast horizon.^[142] ^[144]
- Realistic smoothness of heading and speed paths.^[143] ^[149]
- Reasonable distribution of recurvature and landfall behavior by regime.^[150] ^[143]
- Stable posterior behavior under different particle counts and time steps.^[144]
- Consistency of progression outputs with the downstream wind-field footprint generator.^[146]

Non-goals

The following are out of scope for the first implementation:

- Full atmospheric data assimilation.
- Full dynamical numerical weather prediction.
- Joint ocean-atmosphere coupling.
- Full pressure-wind structural models.
- Operational forecast-grade verification pipelines. [\[149\]](#) [\[144\]](#)

The focus should remain on a reduced-form Bayesian progression engine suitable for simulation, stress testing, and exploratory wind-risk modelling. [\[143\]](#) [\[144\]](#)

Implementation summary

The coding team should implement typhoon forward movement as a Bayesian sequential state-space process with regime-conditioned motion, stochastic updates to speed and heading, coupled updates to wind and size, and inference performed through a particle filter. [\[148\]](#) [\[143\]](#) [\[142\]](#) [\[144\]](#)

This is the recommended minimum viable Bayesian architecture because it captures uncertainty explicitly, remains interpretable, aligns with published Bayesian tropical cyclone track approaches, and integrates naturally with the existing track-based storm abstractions already present in the codebase. [\[147\]](#) [\[143\]](#) [\[145\]](#) [\[146\]](#) [\[144\]](#)



also define what the tail events look like for typhoons

For this model, typhoon “tail events” should mean rare, extreme-end realizations of the typhoon process: unusually intense storms, very large storms, long-lived storms, or unusual tracks that still obey the physics but sit in the far tail of your prior/scenario distributions. They are not a new physical phase after dissipation; they are extreme members of the same formation–intensification–decay lifecycle that tropical cyclones already follow. [\[154\]](#) [\[155\]](#) [\[156\]](#)

Conceptual definition

Within your Bayesian/cyclone framework, a tail event is a storm (or storm segment) that lies in a low-probability region of the joint distribution over:

- Peak wind V_{peak} (e.g. “super typhoon” levels).
- Size (large R_{max} and outer radius).
- Duration and translation (very long-lived or slow-moving).
- Track geometry (e.g. rare recurvature or unusual landfall corridor).
- Compound behavior (e.g. high intensity plus slow forward speed near land).

These should be treated as “daughter events” or scenario realizations from your tail-biased storm distribution and Bayesian track model, similar to how probabilistic catastrophe studies generate alternative realizations with varying track, forward speed, intensity and size. ^[157] ^[155] ^[156]

Tail dimensions to formalize

To make this precise for the coding/spec side, you want explicit “tail” definitions in at least three dimensions:

1. Intensity tail

- Use your hybrid normal–Pareto or similar distribution on peak wind V_{peak} , and define tail events by exceedance over high quantiles (e.g. V_{peak} above the 95th or 99th percentile). ^[155] ^[156] ^[158]
- In practice: scenarios with Category 4–5 equivalent winds in a given basin, or whatever threshold matches your calibration.

2. Size tail

- Define tails in R_{max} and outer wind radius: storms with very wide damaging-wind footprints despite similar peak winds. ^[156] ^[155]
- These are the events that drive large-area impact even if they are not the absolute strongest in peak wind.

3. Track / duration tail

- Rare track geometries: unusual landfall locations, large deviations from dominant regimes, or paths that graze densely exposed areas for long periods. ^[154] ^[157] ^[156]
- Slow-moving or stalling storms near land, producing prolonged winds and heavy rain, even as they decay. ^[159] ^[160] ^[154]

In your Bayesian progression model, tails will most naturally appear as low-probability combinations of these dimensions when you push scenario families and regime-conditioned transitions into their extremes.

Relation to lifecycle

Physically, even tail events still follow the same lifecycle:

- Formation over very warm water ($\gtrsim 27^\circ\text{C}$). ^[156] ^[154]
- Intensification supported by high humidity and favorable shear. ^[158] ^[154] ^[156]
- Mature stage with a well-formed eye and strong eyewall winds. ^[154] ^[156]
- Decay as the storm moves over land or cooler water and loses its ocean heat source, while still producing heavy rain, floods, and landslides as it weakens. ^[160] ^[159] ^[156] ^[154]

Tail events are those lifecycles that reach exceptional peak intensity, retain strength longer than usual, are unusually large, or take rare damaging tracks before they finally decay.

How to encode tails in your models

In your hazard engine, tail events can be explicitly defined and generated by:

- **Intensity distribution:** choose scenario families with higher mean, lower tail threshold, and fatter tails (lower α) for V_{peak} , so you deliberately sample rare high-wind events more often in stress testing.
- **Track regime priors:** assign small, non-zero prior probabilities to rare track regimes (e.g. unusual recurvature or landfall patterns), and let the Bayesian progression generate those paths under selected stress scenarios. [\[161\]](#) [\[162\]](#) [\[157\]](#)
- **Duration / speed:** bias translation-speed and decay parameters to slower/stalling behavior near land or over warm water, creating long-lived high-impact tails. [\[159\]](#) [\[156\]](#) [\[154\]](#)

From a spec perspective, you can then say: “Tail typhoon events are those simulations where one or more of $\{V_{\text{peak}}, R_{\text{max}}, R_{\text{outer}}, \text{duration near land, track rarity}\}$ exceed high quantile thresholds under the calibrated joint distribution,” and the code base generates them via the same Bayesian/Monte Carlo machinery, just under more extreme parameter families.

Do you want these tail definitions written into a short, separate spec section (e.g. “Typhoon Tail Event Definition and Stress Scenarios”) that we can append to the existing documents?



Bayesian Typhoon Track Progression Specification

Purpose

This specification defines a Bayesian progression model for advancing a typhoon through time. The objective is to provide the coding team with a practical probabilistic architecture for simulating forward typhoon motion across a time horizon, with uncertainty represented explicitly rather than through a single deterministic path. [\[171\]](#) [\[172\]](#) [\[173\]](#) [\[174\]](#)

The model is intended for a simplified simulation framework rather than a definitive operational forecast system. It should therefore be interpretable, modular, easy to calibrate, and compatible with the existing storm abstractions already present in the codebase, especially the time-indexed track representation, storm speed metadata, and footprint logic. [\[175\]](#) [\[176\]](#) [\[177\]](#)

Objective

The model should answer the following question:

Given the storm state at time $t-1$, and any environmental or observational information available at time t , what is the posterior distribution of the typhoon state at time t ? [\[172\]](#) [\[173\]](#) [\[174\]](#)

The model should produce a probabilistic sequence of storm center locations, headings, forward speeds, peak winds, and size metrics over the chosen time horizon. It should support both free simulation and sequential updating when new information becomes available.^[174]
^[178]

Design principles

The Bayesian progression model should satisfy the following principles.

1. Storm movement should be represented as a stochastic state process, not a fixed line segment between start and end points.^[178] ^[174]
2. The model should update the storm state sequentially over time using Bayes' rule.^[172] ^[174]
3. Forward motion should be driven by physically meaningful state variables, including position, heading, translation speed, wind intensity, and storm size.^[176] ^[175]
4. The model should allow uncertainty in both motion and structure, not only in one-dimensional track position.^[174] ^[178]
5. The first release should prioritize interpretability and calibration over full meteorological completeness.^[173] ^[179]

Model overview

The recommended design is a Bayesian state-space model. The typhoon is described by an unobserved latent state that evolves over time, while optional data or soft constraints provide likelihood information for updating the posterior state distribution.^[178] ^[172] ^[174]

At each time step the workflow is:

1. Start with the posterior distribution from the previous step.
2. Propagate that state forward using a transition model.
3. Evaluate any new information or plausibility constraints.
4. Update the state distribution using Bayes' rule.
5. Continue until the required time horizon is reached.^[172] ^[174]

State definition

The latent state at time t should be defined as:

$$s_t = (\lambda_t, \phi_t, u_t, \psi_t, V_t, R_t, k_t)$$

Equation (1)

Where:

- λ_t is longitude.
- ϕ_t is latitude.
- u_t is translation speed.

- ψ_t is heading or motion bearing.
- V_t is maximum sustained wind at the storm center.
- R_t is a size vector, at minimum including radius of maximum wind and outer wind radius.
- k_t is an optional track regime or hidden motion class.^{[177] [173] [175] [176]}

A more explicit size vector may be defined as:

$$R_t = (R_{\max,t}, R_{\text{outer},t})$$

Equation (2)

The state should be serializable and compatible with the typhoon event model proposed in the wind-field specification.^[175]

Regime variable

A discrete regime variable should be included because Bayesian typhoon work often benefits from track-pattern categorization. Historical storms tend to fall into recurring path families, and modeling motion conditional on regime is usually more stable and interpretable than using one global motion rule.^{[180] [181] [173]}

Suggested regimes include:

- Straight westward track.
- Northwest-moving recurver.
- Sharp recurvature track.
- Slow or stalled track.
- Landfall decay track.^{[173] [180]}

The regime may be fixed for an event or may evolve with low transition probability over time. The simplest first version should sample it once at storm genesis and hold it fixed unless evidence strongly suggests a transition.^{[180] [173]}

Bayesian formulation

The sequential Bayesian update is:

$$p(s_t \mid y_{1:t}) \propto p(y_t \mid s_t) p(s_t \mid y_{1:t-1})$$

Equation (3)

Where:

- s_t is the latent state at time t .
- y_t is information available at time t , including observations, environmental guidance, or synthetic plausibility constraints.
- $p(s_t \mid y_{1:t-1})$ is the predictive prior from the transition model.^{[174] [172]}

The predictive prior is obtained from:

$$p(s_t | y_{1:t-1}) = \int p(s_t | s_{t-1}) p(s_{t-1} | y_{1:t-1}) ds_{t-1}$$

Equation (4)

This is the canonical state-space formulation and should be the conceptual basis for the implementation.^{[178] [174]}

Transition model

The transition model governs how the typhoon moves from one step to the next. It should update motion, wind, and size jointly, or at least in coupled blocks.

Motion update

The recommended first-order motion block is:

$$\begin{aligned} u_t &\sim \mathcal{N}(\mu_u(s_{t-1}, z_t, k_t), \sigma_u^2(k_t)) \\ \psi_t &\sim \mathcal{N}(\mu_\psi(s_{t-1}, z_t, k_t), \sigma_\psi^2(k_t)) \end{aligned}$$

Equation (5)

Where:

- z_t is environmental forcing or steering information.
- k_t is the regime variable.^{[173] [174] [178]}

The conditional means should depend on at least:

- Previous speed and heading.
- Position and latitude.
- Regime class.
- Land or ocean interaction.
- Recurvature tendency at higher latitudes.^{[179] [180] [173]}

Position update

Given translation speed and heading, the center should be advanced using a local advection rule. In planar approximation:

$$\Delta x_t = u_t \Delta t \cos(\psi_t), \quad \Delta y_t = u_t \Delta t \sin(\psi_t)$$

Equation (6)

And then:

$$\lambda_t = \lambda_{t-1} + c_\lambda \Delta x_t, \quad \phi_t = \phi_{t-1} + c_\phi \Delta y_t$$

Equation (7)

Where λ and ϕ convert km displacement into longitude and latitude increments. The implementation may use Haversine or geodesic utilities for better geographic fidelity, as the current code already includes distance handling logic through Haversine calculations.^{[176] [177]}

Wind update

The wind state should evolve with persistence and environmental moderation:

$$V_t \sim \mathcal{N}(\mu_V(V_{t-1}, z_t, k_t), \sigma_V^2)$$

Equation (8)

The mean function may include:

- Persistence from previous wind.
- Intensification tendency over warm water.
- Weakening due to land interaction.
- Optional decay after recurvature or extratropical transition.

A simple landfall weakening rule may be:

$$V_t = V_{t-1} \exp(-k_{\text{land}} \Delta t)$$

Equation (9)

When the storm is over land or land fraction exceeds a threshold.^[177]

Size update

The storm size state should evolve stochastically but smoothly:

$$\begin{aligned} R_{\text{max},t} &\sim \mathcal{N}(\mu_{R_{\text{max}}}(R_{\text{max},t-1}, V_t, z_t), \sigma_{R_{\text{max}}}^2) \\ R_{\text{outer},t} &\sim \mathcal{N}(\mu_{R_{\text{outer}}}(R_{\text{outer},t-1}, V_t, z_t), \sigma_{R_{\text{outer}}}^2) \end{aligned}$$

Equation (10)

This keeps the wind-field model structurally coupled to the motion model without requiring a full pressure-wind system.^{[175] [176]}

Regime-conditioned mixture model

The transition model should be regime-conditioned. Formally:

$$p(s_t | s_{t-1}) = \sum_k p(s_t | s_{t-1}, k) p(k | s_{t-1})$$

Equation (11)

In the simplest implementation, the event is assigned a regime at genesis:

$$k \sim \text{Categorical}(\pi_1, \dots, \pi_K)$$

Equation (12)

Where the prior weights π_k depend on genesis location, season, and perhaps initial heading.^{[181] [180] [173]}

This is strongly recommended because it provides a principled Bayesian way to encode broad path behavior without forcing the coding team into a highly nonlinear global model.^{[180] [173]}

Likelihood model

The likelihood $p(y_t \mid s_t)$ may be fed by one of three sources.

1. Real observations

If real-time observations are available, the likelihood may use observed storm center positions, observed headings, forecast advisory data, or wind estimates.^{[172] [174]}

2. External guidance

If external model guidance exists, the likelihood may represent agreement with a forecast ensemble or consensus track, following a Bayesian model averaging style approach.^[172]

3. Soft constraints for simulation mode

If the model is running as a synthetic simulator, the likelihood may be replaced by plausibility scores. Examples include:

- Penalize unrealistically sharp heading changes.
- Penalize implausible jumps in translation speed.
- Penalize motion over terrain inconsistent with the chosen regime.
- Penalize trajectories that violate basin boundaries or historical behavior.

This allows the same Bayesian machinery to be used even when there are no real observations to assimilate.^{[179] [173]}

Inference method

The recommended implementation method is Sequential Monte Carlo, also known as a particle filter. This is the most natural way to represent uncertainty in a nonlinear storm state process.^[174]

Particle representation

At time t , represent the posterior as a weighted set of particles:

$$p(s_t | y_{1:t}) \approx \sum_{i=1}^N w_t^{(i)} \delta(s_t - s_t^{(i)})$$

Equation (13)

Where:

- $s_t^{(i)}$ is particle i .
- $w_t^{(i)}$ is its normalized weight. ^[174]

Particle filtering workflow

1. Initialize particles from the genesis prior.
2. Propagate each particle using the transition model.
3. Compute weights from the likelihood or plausibility function.
4. Normalize weights.
5. Resample if the effective sample size falls below a threshold.
6. Repeat for the next time step. ^[174]

This should be the default implementation. A deterministic mean-path fallback may be added later, but the probabilistic particle version should be considered the canonical Bayesian model. ^[174]

Initialization

At genesis time $t=0$, the prior should define distributions for:

- Genesis longitude and latitude.
- Initial heading.
- Initial translation speed.
- Initial maximum wind.
- Initial size.
- Regime class. ^{[182] [183] [177] [173]}

The current distribution module already provides a scenario-family concept with adjustable tail thickness. That concept should be retained and reinterpreted so that peak wind, duration, and perhaps initial translation speed can be sampled from scenario-conditioned priors. ^[183]
^[182]

Environmental forcing

The state transition functions may accept an environmental vector $\mathbf{z}_t$. The first version may keep this simple.

Recommended optional forcing variables:

- Latitude band or recurvature proxy.
- Ocean versus land flag.
- Basin-specific steering tendency.
- Seasonal modifier.
- Optional shear or SST proxy.

If environmental data are unavailable, the model may still work by using regime-conditioned priors plus position-based heuristics.^{[179] [173]}

Output requirements

The Bayesian progression model should produce the following outputs.

Posterior trajectory outputs

- Mean or median storm center path.
- Credible bands or confidence envelopes for location over time.
- Ensemble of particle tracks.
- Posterior distributions for heading and translation speed.^{[172] [174]}

Wind-state outputs

- Posterior mean and quantiles for maximum wind over time.
- Posterior mean and quantiles for storm size metrics over time.
- Optional scenario-conditioned severity summaries.^{[176] [175]}

Event summaries

- Probability of landfall in each region.
- Probability of recurvature.
- Probability of exceeding wind thresholds at chosen horizons.
- Time-to-landfall or time-to-closest-approach distributions.^{[172] [174]}

Interface with the typhoon wind-field model

The Bayesian progression model should feed directly into the typhoon wind-field model from the earlier specification. Each posterior particle or posterior mean state at time t should be usable as an input to the radial wind-field module, which then computes location-specific winds. ^[175] ^[176]

The interface contract should therefore expose, at each time step:

- Center position.
- Translation speed.
- Heading.
- Maximum sustained wind.
- Radius of maximum wind.
- Outer wind radius.
- Land interaction flags.

This ensures the motion model and the wind-field model remain decoupled but interoperable. ^[176] ^[175]

Required new models and modules

Module	Purpose	Notes
BayesianTyphoonState	Canonical latent state container	Should include motion, wind, size, and regime. ^[175]
TrackRegimeModel	Regime priors and regime-conditioned transition parameters	Strongly recommended for interpretability. ^[173] ^[180]
MotionTransitionModel	Updates heading and translation speed	Core of forward progression. ^[174] ^[178]
IntensityTransitionModel	Updates maximum wind through time	Should support land weakening. ^[177]
SizeTransitionModel	Updates wind radii through time	Needed by wind-field model. ^[175] ^[176]
ObservationLikelihoodModel	Scores particles against observations or plausibility constraints	Must support observation-free simulation mode. ^[172] ^[174]
ParticleFilterEngine	Sequential Monte Carlo implementation	Recommended default inference engine. ^[174]
PosteriorExporter	Writes trajectories, credible regions, and summaries	Output-facing module

Refactoring guidance for current codebase

`storm_factory.py`

The current storm factory builds deterministic tracks and intensity profiles. It should be refactored into a prior generator that samples an initial state, regime, and track skeleton rather than a fully fixed event.^[177] ^[173]

`data_structures.py`

The current data structures are event-oriented and use generic intensity fields. They should be extended to support a Bayesian state object, regime variables, posterior particles, and explicit wind-based semantics.^[175]

`forward_model.py`

The current forward model interpolates deterministic state along a track and computes local decay from the center. This logic should remain available, but it should become the downstream consumer of posterior state sequences produced by the Bayesian progression engine.^[176]

Calibration requirements

Calibration should proceed in layers.

1. Regime identification from historical track families.^[181] ^[173] ^[180]
2. Conditional motion parameter fitting by regime for heading and speed updates.^[173] ^[174]
3. Wind persistence and land decay fitting.^[177]
4. Size evolution fitting, at least through heuristic rules initially.^[175] ^[176]
5. Likelihood function tuning for observation-free simulation plausibility.^[179] ^[173]

All parameters should be externalized into configuration files or parameter objects rather than embedded in code constants.^[183]

Validation requirements

The coding team should validate the Bayesian progression model against the following criteria.

- Plausible particle-track spread through time, widening with forecast horizon.^[172] ^[174]
- Realistic smoothness of heading and speed paths.^[173] ^[179]
- Reasonable distribution of recurvature and landfall behavior by regime.^[180] ^[173]
- Stable posterior behavior under different particle counts and time steps.^[174]
- Consistency of progression outputs with the downstream wind-field footprint generator.^[176]

Non-goals

The following are out of scope for the first implementation:

- Full atmospheric data assimilation.
- Full dynamical numerical weather prediction.
- Joint ocean-atmosphere coupling.
- Full pressure-wind structural models.
- Operational forecast-grade verification pipelines.^{[179] [174]}

The focus should remain on a reduced-form Bayesian progression engine suitable for simulation, stress testing, and exploratory wind-risk modelling.^{[173] [174]}

Implementation summary

The coding team should implement typhoon forward movement as a Bayesian sequential state-space process with regime-conditioned motion, stochastic updates to speed and heading, coupled updates to wind and size, and inference performed through a particle filter.^{[178] [173] [172] [174]}

This is the recommended minimum viable Bayesian architecture because it captures uncertainty explicitly, remains interpretable, aligns with published Bayesian tropical cyclone track approaches, and integrates naturally with the existing track-based storm abstractions already present in the codebase.^{[177] [173] [175] [176] [174]}

Tail events and intensity skew

The Bayesian typhoon progression framework must explicitly represent tail events, meaning low-probability but high-severity typhoon realizations. In this specification, a tail event is not a separate post-storm physical phase. Instead, it is an extreme realization drawn from the same probabilistic lifecycle model, lying in the far right tail of one or more state dimensions such as peak wind, storm size, duration, translation behavior, or track rarity.^{[184] [185] [186]}

This distinction is important for architecture. The model should not create a separate “tail-event engine.” It should instead generate tail events naturally from skewed or fat-tailed prior and transition distributions, so that extreme storms remain consistent with the same Bayesian state-space machinery used for ordinary events.^{[186] [173] [174]}

Tail-event dimensions

Tail behavior should be represented in at least five dimensions.

1. **Intensity tail:** unusually high peak sustained wind, including the equivalent of the most severe typhoon classes in the chosen basin.^{[185] [187]}
2. **Size tail:** unusually large radius of maximum wind or outer wind radius, creating broad damaging-wind footprints even when peak wind is not uniquely extreme.^{[187] [185]}
3. **Duration tail:** unusually long-lived storms or storms that maintain damaging winds over long periods.^{[184] [185]}

4. **Track tail:** low-probability path geometries, such as unusual recurvature, unexpected landfall corridors, or tracks that remain close to exposed coastlines for extended periods.^{[186] [180] [173]}
5. **Compound tail:** combinations of high intensity, large size, slow translation, and vulnerable landfall geometry occurring together.^{[187] [186]}

The first implementation may focus mainly on intensity, size, and track tails, but the interfaces should reserve capacity for compound tail scenarios because these often dominate practical risk outcomes.^{[186] [187]}

Intensity skew requirement

Peak wind intensity should not be modeled with a symmetric light-tailed distribution. The codebase already contains a hybrid distribution concept in which ordinary values are modeled with a truncated normal structure and extreme values are modeled with a Pareto tail. That logic should be retained and reinterpreted as a peak-wind severity distribution.^{[182] [183]}

The exceedance distribution for peak wind V_{peak} should be defined as:

$$P(V > v) = \begin{cases} 1 - \Phi\left(\frac{v - \mu}{\sigma}\right), & v \leq v_T \\ P(V > v_T) \left(\frac{v_T}{v}\right)^\alpha, & v > v_T \end{cases}$$

Equation (14)

Where:

- μ is the baseline mean peak wind.
- σ is the baseline standard deviation.
- v_T is the threshold above which tail behavior dominates.
- α is the Pareto tail index, with smaller values representing fatter tails.^{[182] [183]}

This specification requires the coding team to preserve positive skew in the peak-wind distribution. That skew is the mechanism by which the model can generate rare but plausible extreme typhoons for stress testing and tail-risk analysis.^{[183] [182] [186]}

Tail-event classification

The model should support explicit tail-event tagging. A typhoon realization should be classified as a tail event if one or more calibrated state variables exceed predefined quantile thresholds under the reference climate or scenario family.

A recommended first version is:

- **Moderate tail event:** any of V_{peak} , R_{outer} , event duration, or landfall exposure metric exceeds the 95th percentile.^{[182] [183]}
- **Severe tail event:** any key metric exceeds the 99th percentile, or a joint-event score exceeds its 97.5th percentile.^{[183] [182]}

- **Compound tail event:** two or more tail dimensions simultaneously exceed chosen high-quantile thresholds.^{[187] [186]}

The quantiles should be configurable by basin and scenario family, not hard-coded globally.^[183]

Joint tail score

To support risk ranking and filtering, the model should optionally compute a joint tail severity score. A practical first formulation is:

$$S_{\text{tail}} = w_V q_V + w_R q_R + w_D q_D + w_T q_T + w_C q_C$$

Equation (15)

Where:

- q_V is the normalized upper-tail score for peak wind.
- q_R is the normalized upper-tail score for size.
- q_D is the normalized upper-tail score for duration.
- q_T is the normalized rarity score for track geometry.
- q_C is the normalized coastal or landfall exposure score.
- w_* are configurable weights.

This score is not a substitute for the underlying probabilistic model. It is a convenience metric for identifying and ranking extreme realizations drawn from the posterior or scenario ensemble.^{[186] [187]}

Tail behavior in Bayesian progression

Tail events should emerge naturally in the Bayesian progression model through three mechanisms.

1. **Tail-prior sampling:** initial peak wind, size, or duration are sampled from skewed prior distributions with fat-tail support.^{[182] [183]}
2. **Regime-conditioned rare tracks:** low-probability regimes or low-probability path updates are retained with non-zero posterior mass, allowing unusual but plausible trajectories.^{[180] [173]}
3. **Persistence of extremes:** high wind, large size, or slow translation should have persistence structure, so once an event enters an extreme state it does not collapse unrealistically in the next time step unless land or adverse forcing justifies that change.^{[177] [174]}

This means the transition model should not be too mean-reverting in the extreme tail. It should allow rare events to remain extreme over multiple steps when environmental conditions support persistence.^{[186] [174]}

Stress-scenario families

The scenario-family design already present in the codebase should be preserved and extended for typhoon tails. At minimum, the following family types should be supported:^[183]

Scenario family	Intended effect on tail behavior
Historical	Matches observed or reference-climate tail thickness. ^[183]
Baseline	Slightly conservative central estimate with realistic upper tail. ^[183]
Moderate stress	Higher mean peak wind, lower tail threshold, moderately fatter tail. ^[183]
Severe stress	Higher mean, stronger persistence, fatter extreme tail, more frequent large storms. ^[183]
Extreme	Strong positive skew, very fat upper tail, elevated probability of compound extremes. ^[183]

In implementation terms, scenario families should influence at least:

- Peak-wind distribution parameters μ, σ, v_T, α .^[183]
- Size distribution parameters for $R_{\max}$ and R_{outer} .
- Regime priors for rare track behavior.^{[180] [173]}
- Persistence or decay parameters in the transition model.

Tail outputs

The model should produce explicit outputs for tail-event analysis. These should include:

- Tail classification flag.
- Quantile rank for peak wind, size, duration, and track rarity.
- Joint tail score.^{[187] [186]}
- Scenario-family identifier.^[183]
- Exceedance probabilities for user-defined severity thresholds.^{[182] [183]}

These outputs should be available both at event level and, where relevant, at location level after coupling to the wind-field module.^{[175] [176]}

Validation of tail behavior

The coding team should validate not only mean behavior but also the upper tail. Validation should include:

- Stability of high quantiles under repeated Monte Carlo simulation.^{[182] [183]}
- Sensitivity of tail frequency to the Pareto index and threshold.^{[182] [183]}
- Plausibility of rare track realizations under regime priors.^{[173] [180]}
- Persistence of extreme states through time under favorable conditions.^[174]
- Sensible separation between ordinary, severe, and compound tail scenarios.^{[187] [186]}

Implementation note

The coding team should treat tail events as first-class outputs of the model rather than ad hoc exceptions. The architecture should therefore expose the skew and fat-tail settings explicitly in configuration, scenario definitions, posterior summaries, and event metadata.^[186]
^[183]

✱

Techniques for handling multivariate correlations in extreme weather modeling

Key techniques fall into three broad families: multivariate extreme value theory (EVT), copula and dependence modelling, and high-dimensional machine-learning or generative approaches, often combined with bias correction or post-processing. These are now being used to handle joint tails of variables like wind, precipitation, pressure and temperature in extreme weather studies.^[188] ^[189] ^[190] ^[191] ^[192]

1. Multivariate EVT and tail dependence

Classical EVT has been extended from univariate GEV/GPD to multivariate settings where the focus is on tail dependence rather than covariance.^[189] ^[191]

Key techniques:

- **Multivariate peaks-over-threshold (POT) frameworks:** fit GPD-type tails jointly to variables (e.g. wind + rain + surge), often with specialized dependence structures for exceedances.^[191] ^[193]
- **Transformed-linear methods:** replace covariance with a *tail pairwise dependence matrix (TPDM)* and build “extreme analogues” of linear tools (PCA, ARMA, factor models) that operate in the upper tail.^[189]
- **Spectral / angular models:** normalize extremes to unit radius and model the angular distribution to capture how variables co-explode.^[191] ^[189]

When to use: medium dimensions, strong need for theoretically sound tail extrapolation and interpretable tail dependence (e.g. joint storm drivers, multivariate attribution).

2. Copulas and “bulk-and-tail” constructions

Copula-based approaches model marginals and dependence separately, which is practical for environmental extremes.

Typical workflow:

- **Fit bulk-and-tails marginals:** non-extreme part with a flexible parametric or semi-parametric model; extremes with GPD or GEV.^[193] ^[191]
- **Choose a dependence structure:**

- Elliptical or Archimedean copulas for moderate dimension / simpler dependence.
- Vine copulas for higher dimensions with complex pairwise structure.
- **Emphasize tail dependence coefficients and tail-focused copulas when extremes are the priority.**

Use cases: risk portfolios where you want clean control over each marginal (e.g. wind, wave, surge) and an explicit joint tail dependence layer.

3. High-dimensional and AI-based approaches

For gridded fields and multi-hazard combinations, statistical EVT alone becomes unwieldy. Recent work uses **deep generative models** and other ML methods, often hybridized with EVT. [\[190\]](#) [\[194\]](#) [\[192\]](#)

Notable techniques:

- **GANs / VAEs with EVT constraints:** train generative models on transformed data that emphasize extremes, then enforce correct tail behaviour and extremal correlation via EVT-based loss terms or transformations. [\[192\]](#)
- **Hybrid GAN+EVT frameworks:** learn the spatial and multivariate extremal dependence structure for large fields (e.g. wind, rain, pressure on a grid) and compare against classical multivariate extremes models. [\[192\]](#)
- **AI correction of dynamical models:** ML “nudging” layers correct coarse climate model dynamics so that joint extremes (e.g. wind, temperature, humidity) have more realistic frequency and structure when downscaled into storm/impact models. [\[194\]](#) [\[190\]](#)

Use cases: thousands of correlated variables (e.g. full storm fields), multi-hazard simulations, or where you need realistic synthetic events for risk simulation.

4. Multivariate bias correction and post-processing

When using climate or NWP outputs for extremes, **multivariate bias correction** and post-processing are critical to preserve inter-variable structure. [\[195\]](#) [\[188\]](#)

Techniques:

- **Multivariate bias-correction schemes** (e.g. R2D2 and related methods): designed to preserve intervariable relationships (e.g. precip–temperature correlation) better than univariate methods, which often distort joint extremes. [\[188\]](#)
- **Multivariate post-processing for seasonal/ensemble forecasts:** methods that correct spatial and inter-variable error patterns, often using dependence-aware transformations or Gaussian-copula frameworks. [\[195\]](#)

Use cases: feeding corrected fields into event sets, catastrophe models, and Bayesian track/wind frameworks without destroying joint structure.

5. Practical strategy for your setting

For a cyclone/typhoon risk stack like yours, a pragmatic design is:

- Use **bulk-and-tail marginals + hybrid GPD/Pareto tails** for key scalar quantities (peak wind, radius, duration, track metrics).^{[193] [191]}
- Use **copulas or regime-conditional dependence** for low- to mid-dimensional joint drivers (e.g. peak wind, size, translation speed, landfall geometry), potentially augmented with a tail-pairwise dependence matrix to make the dependence explicitly tail-focused.^{[189] [191]}
- For high-dimensional fields (wind/pressure/rain on a grid, multi-hazard footprints), use a **hybrid EVT + deep generative** approach in line with recent multi-hazard GAN work, ensuring extremal correlation is validated against a classical multivariate extremes baseline.^[192]
- Where you rely on GCM/NWP, add a **multivariate bias-correction/post-processing layer** that preserves key cross-dependencies before driving the hazard models.^{[194] [188] [195]}

If you tell me the main variable set you care about (e.g. wind + rain + surge vs. wind + track metrics), I can sketch a concrete modelling stack tailored to that specific multivariate extreme problem.



I would say wind/surge and rain are the three variables I need to capture in a correlated way.

For wind, surge, and rain, I'd model them as a **compound hazard triad** with shared storm-level drivers and explicit residual dependence, rather than trying to correlate the three outputs directly with one static correlation matrix. The literature on compound flooding and multivariate storm hazards shows that surge and precipitation often co-occur under the same meteorological systems, and joint-risk assessment is more robust when it reflects the storm process rather than only pairwise linear correlation.^{[197] [198] [199] [200] [201]}

Recommended structure

The cleanest architecture is a **hierarchical conditional model**:

1. Sample storm-level latent variables: track, forward speed, peak wind, size, pressure proxy, translation asymmetry, duration.
2. Condition wind, surge, and rain on those common latent drivers.
3. Add a residual dependence layer for the remaining co-movement not explained by the storm state.^{[200] [202] [203]}

That gives you:

- **Wind** mainly from peak intensity, size, and local storm-relative geometry.
- **Surge** mainly from coastal wind stress, pressure deficit proxy, track angle, bathymetry/coastal geometry, and storm size.

- Rain mainly from moisture availability, translation speed, convergence, topography, and storm structure. [\[199\]](#) [\[204\]](#) [\[205\]](#)

Why not plain correlation

A single Gaussian correlation matrix is too weak for this problem because:

- Dependence is **nonlinear**.
- Joint behavior is strongest in the **upper tail**.
- Dependence can be **lagged** or near-simultaneous, especially for surge and rainfall in compound flooding. [\[206\]](#) [\[207\]](#) [\[197\]](#) [\[199\]](#)

So I'd recommend:

- Fit marginals for each variable, with EVT tails for extremes. [\[208\]](#) [\[200\]](#)
- Use a **copula** or **conditional multivariate EVT** layer for the residual joint dependence. [\[200\]](#) [\[208\]](#)
- Allow the dependence parameters to vary by **storm regime** or coastal segment. [\[198\]](#) [\[202\]](#) [\[203\]](#)

Practical model stack

A good practical stack for your case is:

Layer	What it does	Suggested technique
Storm generator	Creates track, wind, size, duration, heading	Bayesian state-space / particle model from your spec [209] [210]
Wind module	Produces local wind field	Radial wind model conditioned on storm state [211] [212]
Surge module	Produces surge proxy or coastal surge level	Conditional regression + EVT tail + coastal modifiers [199] [204] [205]
Rain module	Produces rain intensity/accumulation	Conditional model on storm state + speed/topography [197] [198] [204]
Dependence layer	Preserves leftover joint structure	Vine copula, tail copula, or conditional extremes [200] [208]

This is better than directly sampling (W, S, R) jointly in one black-box model, because it preserves interpretability and lets you stress one mechanism without breaking the others. [\[202\]](#) [\[203\]](#) [\[200\]](#)

Best statistical choices

For your variables, I'd prioritize one of these two approaches:

- **Conditional extremes / hierarchical copula approach**: best if you want interpretable tail dependence between wind, surge, and rain for event simulation. [\[208\]](#) [\[200\]](#)

- **Weather-type / storm-regime emulator:** best if you want to simulate many events by storm class, with different joint dependence structures under different regimes. The multivariate wave-surge emulator literature is very aligned with this idea.^{[203] [202]}

A particularly good structure is:

$$P(W, S, R) = \int P(W | Z) P(S | Z) P(R | Z) P_{\text{res}}(W, S, R | Z) P(Z) dZ$$

where Z is the latent storm state and P_{res} is the residual dependence layer. That lets most of the correlation come from common drivers, with the copula only handling what remains.^{[202] [200]}

What to include in your spec

For the coding team, I would explicitly require:

- Shared latent storm drivers for all three hazards.
- Hazard-specific conditional response models for wind, surge, and rain.
- A tail-aware joint dependence layer, not simple Pearson correlation.
- Regime-specific dependence parameters, because compound flooding behavior changes by storm type and geography.^{[198] [203] [200] [202]}

For example:

- Fast intense compact storm: high wind, moderate rain, localized surge.
- Large slow-moving storm near coast: moderate-high wind, very high surge, very high rain.
- Recurving offshore storm: moderate surge and wind, lower inland rain.^{[204] [205] [199]}

That kind of scenario logic is exactly what a hierarchical multivariate extremes model captures well.

I can turn this into a formal spec section for the team — “Joint dependence model for wind, surge, and rain” — and append it to your existing document.

✱

I like the hierarchy approach.

That makes sense. A **hierarchical model** is the right choice here because most of the dependence between wind, surge, and rain should come from shared storm physics, while a smaller residual layer handles what the latent storm state does not explain.^{[216] [217] [218] [219]}

Structure

I would define the hierarchy as:

1. **Storm state layer:** track, central pressure proxy, peak wind, size, translation speed, heading, duration, land interaction.^{[220] [221] [222]}

2. Hazard response layer: conditional models for local wind, surge, and rain given the storm state.^{[218] [223] [224]}

3. Residual dependence layer: copula or conditional-extremes layer for any remaining upper-tail dependence after conditioning on storm state.^{[219] [225]}

This is cleaner than direct joint sampling because it preserves interpretation and aligns with your preference for a formal model hierarchy rather than a monolithic black-box dependence model.^[226]

Recommended equations

A compact formulation is:

$$P(W, S, R) = \int P(W | Z) P(S | Z) P(R | Z) C_{\text{res}}(W, S, R | Z) P(Z) dZ$$

Equation (1)

where Z is the latent storm state, W is wind, S is surge, R is rain, and C_{res} is the residual dependence structure. This means most correlation is induced through the shared storm generator, while the copula layer only captures leftover co-movement, especially in the tail.^{[227] [228] [219]}

A practical hazard decomposition is:

- $W | Z$: radial wind-field model.^[221]
- $S | Z$: surge proxy model driven by coastal wind stress, pressure proxy, size, heading, and local coastal modifiers.^{[223] [218]}
- $R | Z$: rain model driven by storm moisture proxy, translation speed, persistence, size, and orographic/coastal exposure.^{[217] [224] [223]}

Best implementation choice

For your use case, I would recommend:

- Stage 1: hierarchical conditional model only.
- Stage 2: add a tail-aware residual copula, probably vine copula or conditional extremes, once you can measure unexplained dependence in the residuals.^{[225] [219]}
- Stage 3: allow regime-specific dependence, so a slow large landfalling storm uses a different residual structure from a fast compact offshore storm.^{[228] [217] [227]}

That gives you a modular build path and fits the broader architecture you've been shaping for typhoon progression, tail events, and institutional model hierarchy.^[226]

Next step

The next spec addition should be a dedicated section called something like “Hierarchical joint hazard model for wind, surge, and rain”, with:

- latent storm variables,
- conditional hazard equations,
- residual dependence design,
- tail treatment,
- calibration and validation requirements.^{[219] [225]}

I can append that section into the existing Bayesian typhoon spec so the coding team gets one integrated document.

✱

Bayesian Typhoon Track Progression Specification

Purpose

This specification defines a Bayesian progression model for advancing a typhoon through time. The objective is to provide the coding team with a practical probabilistic architecture for simulating forward typhoon motion across a time horizon, with uncertainty represented explicitly rather than through a single deterministic path.^{[229] [230] [231] [232]}

The model is intended for a simplified simulation framework rather than a definitive operational forecast system. It should therefore be interpretable, modular, easy to calibrate, and compatible with the existing storm abstractions already present in the codebase, especially the time-indexed track representation, storm speed metadata, and footprint logic.^{[233] [234] [235]}

Objective

The model should answer the following question:

Given the storm state at time $t-1$, and any environmental or observational information available at time t , what is the posterior distribution of the typhoon state at time t ?^{[230] [231] [232]}

The model should produce a probabilistic sequence of storm center locations, headings, forward speeds, peak winds, and size metrics over the chosen time horizon. It should support both free simulation and sequential updating when new information becomes available.^{[232] [236]}

Design principles

The Bayesian progression model should satisfy the following principles.

1. Storm movement should be represented as a stochastic state process, not a fixed line segment between start and end points.^{[236] [232]}
2. The model should update the storm state sequentially over time using Bayes' rule.^{[230] [232]}
3. Forward motion should be driven by physically meaningful state variables, including position, heading, translation speed, wind intensity, and storm size.^{[234] [233]}
4. The model should allow uncertainty in both motion and structure, not only in one-dimensional track position.^{[232] [236]}
5. The first release should prioritize interpretability and calibration over full meteorological completeness.^{[231] [237]}

Model overview

The recommended design is a Bayesian state-space model. The typhoon is described by an unobserved latent state that evolves over time, while optional data or soft constraints provide likelihood information for updating the posterior state distribution.^{[236] [230] [232]}

At each time step the workflow is:

1. Start with the posterior distribution from the previous step.
2. Propagate that state forward using a transition model.
3. Evaluate any new information or plausibility constraints.
4. Update the state distribution using Bayes' rule.
5. Continue until the required time horizon is reached.^{[230] [232]}

State definition

The latent state at time t should be defined as:

$$s_t = (\lambda_t, \phi_t, u_t, \psi_t, V_t, R_t, k_t)$$

Equation (1)

Where:

- λ_t is longitude.
- ϕ_t is latitude.
- u_t is translation speed.
- ψ_t is heading or motion bearing.
- V_t is maximum sustained wind at the storm center.

- R_t is a size vector, at minimum including radius of maximum wind and outer wind radius.
- k_t is an optional track regime or hidden motion class. [\[235\]](#) [\[231\]](#) [\[233\]](#) [\[234\]](#)

A more explicit size vector may be defined as:

$$R_t = (R_{\max,t}, R_{\text{outer},t})$$

Equation (2)

The state should be serializable and compatible with the typhoon event model proposed in the wind-field specification. [\[233\]](#)

Regime variable

A discrete regime variable should be included because Bayesian typhoon work often benefits from track-pattern categorization. Historical storms tend to fall into recurring path families, and modeling motion conditional on regime is usually more stable and interpretable than using one global motion rule. [\[238\]](#) [\[239\]](#) [\[231\]](#)

Suggested regimes include:

- Straight westward track.
- Northwest-moving recurver.
- Sharp recurvature track.
- Slow or stalled track.
- Landfall decay track. [\[231\]](#) [\[238\]](#)

The regime may be fixed for an event or may evolve with low transition probability over time. The simplest first version should sample it once at storm genesis and hold it fixed unless evidence strongly suggests a transition. [\[238\]](#) [\[231\]](#)

Bayesian formulation

The sequential Bayesian update is:

$$p(s_t \mid y_{1:t}) \propto p(y_t \mid s_t) p(s_t \mid y_{1:t-1})$$

Equation (3)

Where:

- s_t is the latent state at time t .
- y_t is information available at time t , including observations, environmental guidance, or synthetic plausibility constraints.
- $p(s_t \mid y_{1:t-1})$ is the predictive prior from the transition model. [\[232\]](#) [\[230\]](#)

The predictive prior is obtained from:

$$p(s_t | y_{1:t-1}) = \int p(s_t | s_{t-1}) p(s_{t-1} | y_{1:t-1}) ds_{t-1}$$

Equation (4)

This is the canonical state-space formulation and should be the conceptual basis for the implementation.^{[236] [232]}

Transition model

The transition model governs how the typhoon moves from one step to the next. It should update motion, wind, and size jointly, or at least in coupled blocks.

Motion update

The recommended first-order motion block is:

$$\begin{aligned} u_t &\sim \mathcal{N}(\mu_u(s_{t-1}, z_t, k_t), \sigma_u^2(k_t)) \\ \psi_t &\sim \mathcal{N}(\mu_\psi(s_{t-1}, z_t, k_t), \sigma_\psi^2(k_t)) \end{aligned}$$

Equation (5)

Where:

- z_t is environmental forcing or steering information.
- k_t is the regime variable.^{[231] [232] [236]}

The conditional means should depend on at least:

- Previous speed and heading.
- Position and latitude.
- Regime class.
- Land or ocean interaction.
- Recurvature tendency at higher latitudes.^{[237] [238] [231]}

Position update

Given translation speed and heading, the center should be advanced using a local advection rule. In planar approximation:

$$\Delta x_t = u_t \Delta t \cos(\psi_t), \quad \Delta y_t = u_t \Delta t \sin(\psi_t)$$

Equation (6)

And then:

$$\lambda_t = \lambda_{t-1} + c_\lambda \Delta x_t, \quad \phi_t = \phi_{t-1} + c_\phi \Delta y_t$$

Equation (7)

Where λ and ϕ convert km displacement into longitude and latitude increments. The implementation may use Haversine or geodesic utilities for better geographic fidelity, as the current code already includes distance handling logic through Haversine calculations.^{[234] [235]}

Wind update

The wind state should evolve with persistence and environmental moderation:

$$V_t \sim \mathcal{N}(\mu_V(V_{t-1}, z_t, k_t), \sigma_V^2)$$

Equation (8)

The mean function may include:

- Persistence from previous wind.
- Intensification tendency over warm water.
- Weakening due to land interaction.
- Optional decay after recurvature or extratropical transition.

A simple landfall weakening rule may be:

$$V_t = V_{t-1} \exp(-k_{\text{land}} \Delta t)$$

Equation (9)

When the storm is over land or land fraction exceeds a threshold.^[235]

Size update

The storm size state should evolve stochastically but smoothly:

$$\begin{aligned} R_{\text{max},t} &\sim \mathcal{N}(\mu_{R_{\text{max}}}(R_{\text{max},t-1}, V_t, z_t), \sigma_{R_{\text{max}}}^2) \\ R_{\text{outer},t} &\sim \mathcal{N}(\mu_{R_{\text{outer}}}(R_{\text{outer},t-1}, V_t, z_t), \sigma_{R_{\text{outer}}}^2) \end{aligned}$$

Equation (10)

This keeps the wind-field model structurally coupled to the motion model without requiring a full pressure-wind system.^{[233] [234]}

Regime-conditioned mixture model

The transition model should be regime-conditioned. Formally:

$$p(s_t | s_{t-1}) = \sum_k p(s_t | s_{t-1}, k) p(k | s_{t-1})$$

Equation (11)

In the simplest implementation, the event is assigned a regime at genesis:

$$k \sim \text{Categorical}(\pi_1, \dots, \pi_K)$$

Equation (12)

Where the prior weights π_k depend on genesis location, season, and perhaps initial heading.^{[239] [238] [231]}

This is strongly recommended because it provides a principled Bayesian way to encode broad path behavior without forcing the coding team into a highly nonlinear global model.^{[238] [231]}

Likelihood model

The likelihood $p(y_t \mid s_t)$ may be fed by one of three sources.

1. Real observations

If real-time observations are available, the likelihood may use observed storm center positions, observed headings, forecast advisory data, or wind estimates.^{[230] [232]}

2. External guidance

If external model guidance exists, the likelihood may represent agreement with a forecast ensemble or consensus track, following a Bayesian model averaging style approach.^[230]

3. Soft constraints for simulation mode

If the model is running as a synthetic simulator, the likelihood may be replaced by plausibility scores. Examples include:

- Penalize unrealistically sharp heading changes.
- Penalize implausible jumps in translation speed.
- Penalize motion over terrain inconsistent with the chosen regime.
- Penalize trajectories that violate basin boundaries or historical behavior.

This allows the same Bayesian machinery to be used even when there are no real observations to assimilate.^{[237] [231]}

Inference method

The recommended implementation method is Sequential Monte Carlo, also known as a particle filter. This is the most natural way to represent uncertainty in a nonlinear storm state process.^[232]

Particle representation

At time t , represent the posterior as a weighted set of particles:

$$p(s_t \mid y_{1:t}) \approx \sum_{i=1}^N w_t^{(i)} \delta(s_t - s_t^{(i)})$$

Equation (13)

Where:

- $s_t^{(i)}$ is particle i .
- $w_t^{(i)}$ is its normalized weight.^[232]

Particle filtering workflow

1. Initialize particles from the genesis prior.
2. Propagate each particle using the transition model.
3. Compute weights from the likelihood or plausibility function.
4. Normalize weights.
5. Resample if the effective sample size falls below a threshold.
6. Repeat for the next time step.^[232]

This should be the default implementation. A deterministic mean-path fallback may be added later, but the probabilistic particle version should be considered the canonical Bayesian model.^[232]

Initialization

At genesis time $t=0$, the prior should define distributions for:

- Genesis longitude and latitude.
- Initial heading.
- Initial translation speed.
- Initial maximum wind.
- Initial size.
- Regime class.^{[240] [241] [235] [231]}

The current distribution module already provides a scenario-family concept with adjustable tail thickness. That concept should be retained and reinterpreted so that peak wind, duration, and perhaps initial translation speed can be sampled from scenario-conditioned priors.^[241]
^[240]

Environmental forcing

The state transition functions may accept an environmental vector z_t . The first version may keep this simple.

Recommended optional forcing variables:

- Latitude band or recurvature proxy.
- Ocean versus land flag.
- Basin-specific steering tendency.

- Seasonal modifier.
- Optional shear or SST proxy.

If environmental data are unavailable, the model may still work by using regime-conditioned priors plus position-based heuristics.^{[237] [231]}

Output requirements

The Bayesian progression model should produce the following outputs.

Posterior trajectory outputs

- Mean or median storm center path.
- Credible bands or confidence envelopes for location over time.
- Ensemble of particle tracks.
- Posterior distributions for heading and translation speed.^{[230] [232]}

Wind-state outputs

- Posterior mean and quantiles for maximum wind over time.
- Posterior mean and quantiles for storm size metrics over time.
- Optional scenario-conditioned severity summaries.^{[234] [233]}

Event summaries

- Probability of landfall in each region.
- Probability of recurvature.
- Probability of exceeding wind thresholds at chosen horizons.
- Time-to-landfall or time-to-closest-approach distributions.^{[230] [232]}

Interface with the typhoon wind-field model

The Bayesian progression model should feed directly into the typhoon wind-field model from the earlier specification. Each posterior particle or posterior mean state at time t should be usable as an input to the radial wind-field module, which then computes location-specific winds.^{[233] [234]}

The interface contract should therefore expose, at each time step:

- Center position.
- Translation speed.
- Heading.
- Maximum sustained wind.
- Radius of maximum wind.
- Outer wind radius.

- Land interaction flags.

This ensures the motion model and the wind-field model remain decoupled but interoperable. [\[234\]](#) [\[233\]](#)

Required new models and modules

Module	Purpose	Notes
BayesianTyphoonState	Canonical latent state container	Should include motion, wind, size, and regime. [233]
TrackRegimeModel	Regime priors and regime-conditioned transition parameters	Strongly recommended for interpretability. [231] [238]
MotionTransitionModel	Updates heading and translation speed	Core of forward progression. [232] [236]
IntensityTransitionModel	Updates maximum wind through time	Should support land weakening. [235]
SizeTransitionModel	Updates wind radii through time	Needed by wind-field model. [233] [234]
ObservationLikelihoodModel	Scores particles against observations or plausibility constraints	Must support observation-free simulation mode. [230] [232]
ParticleFilterEngine	Sequential Monte Carlo implementation	Recommended default inference engine. [232]
PosteriorExporter	Writes trajectories, credible regions, and summaries	Output-facing module

Refactoring guidance for current codebase

`storm_factory.py`

The current storm factory builds deterministic tracks and intensity profiles. It should be refactored into a prior generator that samples an initial state, regime, and track skeleton rather than a fully fixed event. [\[235\]](#) [\[231\]](#)

`data_structures.py`

The current data structures are event-oriented and use generic intensity fields. They should be extended to support a Bayesian state object, regime variables, posterior particles, and explicit wind-based semantics. [\[233\]](#)

`forward_model.py`

The current forward model interpolates deterministic state along a track and computes local decay from the center. This logic should remain available, but it should become the downstream consumer of posterior state sequences produced by the Bayesian progression engine. [\[234\]](#)

Calibration requirements

Calibration should proceed in layers.

1. Regime identification from historical track families. [\[239\]](#) [\[231\]](#) [\[238\]](#)
2. Conditional motion parameter fitting by regime for heading and speed updates. [\[231\]](#) [\[232\]](#)
3. Wind persistence and land decay fitting. [\[235\]](#)
4. Size evolution fitting, at least through heuristic rules initially. [\[233\]](#) [\[234\]](#)
5. Likelihood function tuning for observation-free simulation plausibility. [\[237\]](#) [\[231\]](#)

All parameters should be externalized into configuration files or parameter objects rather than embedded in code constants. [\[241\]](#)

Validation requirements

The coding team should validate the Bayesian progression model against the following criteria.

- Plausible particle-track spread through time, widening with forecast horizon. [\[230\]](#) [\[232\]](#)
- Realistic smoothness of heading and speed paths. [\[231\]](#) [\[237\]](#)
- Reasonable distribution of recurvature and landfall behavior by regime. [\[238\]](#) [\[231\]](#)
- Stable posterior behavior under different particle counts and time steps. [\[232\]](#)
- Consistency of progression outputs with the downstream wind-field footprint generator. [\[234\]](#)

Non-goals

The following are out of scope for the first implementation:

- Full atmospheric data assimilation.
- Full dynamical numerical weather prediction.
- Joint ocean-atmosphere coupling.
- Full pressure-wind structural models.
- Operational forecast-grade verification pipelines. [\[237\]](#) [\[232\]](#)

The focus should remain on a reduced-form Bayesian progression engine suitable for simulation, stress testing, and exploratory wind-risk modelling. [\[231\]](#) [\[232\]](#)

Implementation summary

The coding team should implement typhoon forward movement as a Bayesian sequential state-space process with regime-conditioned motion, stochastic updates to speed and heading, coupled updates to wind and size, and inference performed through a particle filter. [\[236\]](#) [\[231\]](#) [\[230\]](#) [\[232\]](#)

This is the recommended minimum viable Bayesian architecture because it captures uncertainty explicitly, remains interpretable, aligns with published Bayesian tropical cyclone track approaches, and integrates naturally with the existing track-based storm abstractions already present in the codebase.^{[235] [231] [233] [234] [232]}

Tail events and intensity skew

The Bayesian typhoon progression framework must explicitly represent tail events, meaning low-probability but high-severity typhoon realizations. In this specification, a tail event is not a separate post-storm physical phase. Instead, it is an extreme realization drawn from the same probabilistic lifecycle model, lying in the far right tail of one or more state dimensions such as peak wind, storm size, duration, translation behavior, or track rarity.^{[242] [243] [244]}

This distinction is important for architecture. The model should not create a separate “tail-event engine.” It should instead generate tail events naturally from skewed or fat-tailed prior and transition distributions, so that extreme storms remain consistent with the same Bayesian state-space machinery used for ordinary events.^{[244] [231] [232]}

Tail-event dimensions

Tail behavior should be represented in at least five dimensions.

1. **Intensity tail:** unusually high peak sustained wind, including the equivalent of the most severe typhoon classes in the chosen basin.^{[243] [245]}
2. **Size tail:** unusually large radius of maximum wind or outer wind radius, creating broad damaging-wind footprints even when peak wind is not uniquely extreme.^{[245] [243]}
3. **Duration tail:** unusually long-lived storms or storms that maintain damaging winds over long periods.^{[242] [243]}
4. **Track tail:** low-probability path geometries, such as unusual recurvature, unexpected landfall corridors, or tracks that remain close to exposed coastlines for extended periods.^{[244] [238] [231]}
5. **Compound tail:** combinations of high intensity, large size, slow translation, and vulnerable landfall geometry occurring together.^{[245] [244]}

The first implementation may focus mainly on intensity, size, and track tails, but the interfaces should reserve capacity for compound tail scenarios because these often dominate practical risk outcomes.^{[244] [245]}

Intensity skew requirement

Peak wind intensity should not be modeled with a symmetric light-tailed distribution. The codebase already contains a hybrid distribution concept in which ordinary values are modeled with a truncated normal structure and extreme values are modeled with a Pareto tail. That logic should be retained and reinterpreted as a peak-wind severity distribution.^{[240] [241]}

The exceedance distribution for peak wind V_{peak} should be defined as:

$$P(V > v) = \begin{cases} 1 - \Phi\left(\frac{v - \mu}{\sigma}\right), & v \leq v_T \\ P(V > v_T) \left(\frac{v_T}{v}\right)^\alpha, & v > v_T \end{cases}$$

Equation (14)

Where:

- μ is the baseline mean peak wind.
- σ is the baseline standard deviation.
- v_T is the threshold above which tail behavior dominates.
- α is the Pareto tail index, with smaller values representing fatter tails. [\[240\]](#) [\[241\]](#)

This specification requires the coding team to preserve positive skew in the peak-wind distribution. That skew is the mechanism by which the model can generate rare but plausible extreme typhoons for stress testing and tail-risk analysis. [\[241\]](#) [\[240\]](#) [\[244\]](#)

Tail-event classification

The model should support explicit tail-event tagging. A typhoon realization should be classified as a tail event if one or more calibrated state variables exceed predefined quantile thresholds under the reference climate or scenario family.

A recommended first version is:

- **Moderate tail event:** any of V_{peak} , R_{outer} , event duration, or landfall exposure metric exceeds the 95th percentile. [\[240\]](#) [\[241\]](#)
- **Severe tail event:** any key metric exceeds the 99th percentile, or a joint-event score exceeds its 97.5th percentile. [\[241\]](#) [\[240\]](#)
- **Compound tail event:** two or more tail dimensions simultaneously exceed chosen high-quantile thresholds. [\[245\]](#) [\[244\]](#)

The quantiles should be configurable by basin and scenario family, not hard-coded globally. [\[241\]](#)

Joint tail score

To support risk ranking and filtering, the model should optionally compute a joint tail severity score. A practical first formulation is:

$$S_{\text{tail}} = w_V q_V + w_R q_R + w_D q_D + w_T q_T + w_C q_C$$

Equation (15)

Where:

- q_V is the normalized upper-tail score for peak wind.
- q_R is the normalized upper-tail score for size.

- q_D is the normalized upper-tail score for duration.
- q_T is the normalized rarity score for track geometry.
- q_C is the normalized coastal or landfall exposure score.
- w_i are configurable weights.

This score is not a substitute for the underlying probabilistic model. It is a convenience metric for identifying and ranking extreme realizations drawn from the posterior or scenario ensemble. [\[244\]](#) [\[245\]](#)

Tail behavior in Bayesian progression

Tail events should emerge naturally in the Bayesian progression model through three mechanisms.

1. **Tail-prior sampling:** initial peak wind, size, or duration are sampled from skewed prior distributions with fat-tail support. [\[240\]](#) [\[241\]](#)
2. **Regime-conditioned rare tracks:** low-probability regimes or low-probability path updates are retained with non-zero posterior mass, allowing unusual but plausible trajectories. [\[238\]](#) [\[231\]](#)
3. **Persistence of extremes:** high wind, large size, or slow translation should have persistence structure, so once an event enters an extreme state it does not collapse unrealistically in the next time step unless land or adverse forcing justifies that change. [\[235\]](#) [\[232\]](#)

This means the transition model should not be too mean-reverting in the extreme tail. It should allow rare events to remain extreme over multiple steps when environmental conditions support persistence. [\[244\]](#) [\[232\]](#)

Stress-scenario families

The scenario-family design already present in the codebase should be preserved and extended for typhoon tails. At minimum, the following family types should be supported:[\[241\]](#)

Scenario family	Intended effect on tail behavior
Historical	Matches observed or reference-climate tail thickness. [241]
Baseline	Slightly conservative central estimate with realistic upper tail. [241]
Moderate stress	Higher mean peak wind, lower tail threshold, moderately fatter tail. [241]
Severe stress	Higher mean, stronger persistence, fatter extreme tail, more frequent large storms. [241]
Extreme	Strong positive skew, very fat upper tail, elevated probability of compound extremes. [241]

In implementation terms, scenario families should influence at least:

- Peak-wind distribution parameters μ, σ, v_T, α . [\[241\]](#)
- Size distribution parameters for $R_{\max}$ and R_{outer} .
- Regime priors for rare track behavior. [\[238\]](#) [\[231\]](#)

- Persistence or decay parameters in the transition model.

Tail outputs

The model should produce explicit outputs for tail-event analysis. These should include:

- Tail classification flag.
- Quantile rank for peak wind, size, duration, and track rarity.
- Joint tail score.^{[245] [244]}
- Scenario-family identifier.^[241]
- Exceedance probabilities for user-defined severity thresholds.^{[240] [241]}

These outputs should be available both at event level and, where relevant, at location level after coupling to the wind-field module.^{[233] [234]}

Validation of tail behavior

The coding team should validate not only mean behavior but also the upper tail. Validation should include:

- Stability of high quantiles under repeated Monte Carlo simulation.^{[240] [241]}
- Sensitivity of tail frequency to the Pareto index and threshold.^{[240] [241]}
- Plausibility of rare track realizations under regime priors.^{[231] [238]}
- Persistence of extreme states through time under favorable conditions.^[232]
- Sensible separation between ordinary, severe, and compound tail scenarios.^{[245] [244]}

Implementation note

The coding team should treat tail events as first-class outputs of the model rather than ad hoc exceptions. The architecture should therefore expose the skew and fat-tail settings explicitly in configuration, scenario definitions, posterior summaries, and event metadata.^{[244] [241]}

Hierarchical joint hazard model for wind, surge, and rain

The Bayesian typhoon progression model should be extended with a hierarchical multivariate hazard layer for wind, surge, and rain. This is the recommended architecture for handling their dependence structure because most of the correlation between these variables should arise from common storm-level drivers rather than from a single undifferentiated correlation matrix.^{[246] [247] [248] [249]}

This specification considers the hierarchical approach to be at a sufficient level of sophistication for the intended platform. It is materially more realistic than a static Gaussian correlation model, but it remains modular, calibratable, and explainable enough for implementation, review, and model governance.^{[249] [250] [251]}

Conceptual structure

The model should be organized into three layers.

1. **Latent storm-state layer:** a Bayesian state-space representation of the storm, including position, heading, translation speed, peak wind, size, land interaction, and other core storm descriptors. [\[234\]](#) [\[235\]](#) [\[233\]](#)
2. **Hazard-response layer:** conditional models for local wind, surge, and rain given the latent storm state. [\[248\]](#) [\[252\]](#) [\[253\]](#)
3. **Residual dependence layer:** a tail-aware dependence model for the remaining co-movement not explained by the latent storm state. [\[254\]](#) [\[249\]](#)

This decomposition is the preferred implementation because it aligns with physical reasoning. Wind, surge, and rain are not independent, but they are not jointly driven by an arbitrary covariance structure either. They are linked because the same underlying storm state produces all three hazards. [\[247\]](#) [\[255\]](#) [\[248\]](#)

Joint distribution

The joint hazard structure should be written conceptually as:

$$P(W, S, R) = \int P(W | Z) P(S | Z) P(R | Z) C_{\text{res}}(W, S, R | Z) P(Z) dZ$$

Equation (16)

Where:

- Z is the latent storm state.
- W is local wind.
- S is local surge or surge proxy.
- R is local rainfall or rainfall proxy.
- C_{res} is the residual dependence structure. [\[250\]](#) [\[251\]](#) [\[249\]](#)

The coding team should interpret this as follows. Most cross-variable dependence should be induced by the common storm state Z . The residual dependence term should only capture the dependence that remains after conditioning on Z , especially in the upper tail. [\[249\]](#) [\[254\]](#)

Latent storm-state layer

The storm-state layer is already defined in the Bayesian progression model. For the joint hazard model, the key latent variables should include at least:

- Center location.
- Heading.
- Translation speed.
- Peak sustained wind.

- Radius of maximum wind.
- Outer wind radius.
- Land interaction indicator.
- Optional pressure deficit proxy.
- Optional moisture or convective efficiency proxy. [\[235\]](#) [\[233\]](#) [\[234\]](#)

These variables should be treated as the shared drivers of the three hazard processes. [\[252\]](#)
[\[248\]](#)

Hazard-response layer

The second layer should define conditional hazard models for wind, surge, and rain.

Wind model

The wind model should consume the storm state and generate local wind speed using the radial wind-field structure defined elsewhere in the specification. At minimum, local wind should depend on: [\[233\]](#) [\[234\]](#)

- Distance from storm center.
- Radius of maximum wind.
- Peak sustained wind.
- Motion asymmetry.
- Land interaction. [\[234\]](#) [\[233\]](#)

Conceptually:

$$W \sim P_W(\cdot \mid Z)$$

Equation (17)

Surge model

The surge model should be conditional on storm state plus local coastal exposure. Surge is driven not only by wind magnitude but also by storm size, pressure structure, heading, coastal orientation, and local bathymetric or geometric amplifiers. [\[253\]](#) [\[248\]](#) [\[252\]](#)

A simplified conditional form is:

$$S = g_S(Z, L_{\text{coast}}) + \varepsilon_S$$

Equation (18)

Where L_{coast} represents location-specific coastal modifiers. The first implementation may use a stylized surge proxy rather than a full hydrodynamic model, but it must remain structurally conditional on the storm state rather than merely correlated with wind. [\[248\]](#) [\[252\]](#)

Rain model

The rain model should be conditional on storm state plus local terrain or moisture modifiers. Rainfall should depend on at least storm size, translation speed, persistence, land interaction, and optional moisture/orographic proxies.^{[247] [252] [253]}

A simplified conditional form is:

$$R = g_R(Z, L_{\text{terrain}}) + \varepsilon_R$$

Equation (19)

Where L_{terrain} represents local terrain, coastal, or exposure modifiers. The coding team should note that slow translation speed may materially increase accumulated rainfall, even when peak wind is not at its most extreme.^{[252] [253] [247]}

Residual dependence layer

After conditioning on the latent storm state, some dependence may remain between wind, surge, and rain, especially in the upper tail. This should be handled with an explicit residual dependence layer, not by forcing the conditional models themselves to absorb all co-movement.^{[254] [249]}

The first implementation may use one of the following:

- Tail-aware copula.
- Vine copula for flexible pairwise dependence.
- Conditional extremes model for upper-tail co-exceedance.^{[249] [254]}

The residual layer should be weak relative to the shared storm-state layer in ordinary events, but it may become more important in the extreme tail or under specific storm regimes.^{[254] [249]}

Regime-specific dependence

Dependence parameters should be allowed to vary by storm regime, because a slow-moving large landfalling storm does not produce the same joint hazard structure as a fast compact offshore storm.^{[250] [247] [248]}

At minimum, the coding team should support regime-specific dependence profiles for:

- Compact high-wind storms.
- Large slow-moving storms.
- Coastal-grazing storms.
- Recurving offshore storms.^{[247] [248] [250]}

This requirement is important because compound hazard behavior is event-type dependent and cannot be captured adequately by one global tri-variate dependence model.^{[248] [247] [249]}

Tail treatment in the joint hazard model

The joint hazard layer must integrate with the tail-event framework already defined in this specification. In particular, the model should recognize that compound tail events may arise from:

- Extreme wind with moderate rain and high surge.
- Moderate-high wind with very high surge and very high rain.
- Long-duration moderate wind with exceptional rainfall accumulation.
- Large storms producing broad but not necessarily record peak winds. [\[252\]](#) [\[247\]](#) [\[248\]](#) [\[244\]](#)

This means tail classification should not rely only on peak wind. It should incorporate multi-hazard exceedance behavior and compound tail combinations. [\[245\]](#) [\[244\]](#)

Calibration requirements

Calibration should proceed in stages.

1. Calibrate the latent storm-state model separately. [\[231\]](#) [\[232\]](#)
2. Calibrate the wind module against wind-field expectations. [\[233\]](#) [\[234\]](#)
3. Calibrate the surge proxy or surge-response function against representative coastal behavior. [\[248\]](#) [\[252\]](#)
4. Calibrate the rainfall model against duration, speed, and size behavior. [\[253\]](#) [\[247\]](#) [\[252\]](#)
5. Estimate residual dependence on standardized residuals after conditioning on storm state. [\[249\]](#) [\[254\]](#)
6. Refit by regime if material structural differences are observed. [\[250\]](#) [\[247\]](#)

The coding team should not attempt to estimate the full joint tri-variate dependence in one step before the conditional hazard layers are stable. The hierarchy is the primary structural assumption and should be respected during calibration. [\[254\]](#) [\[249\]](#)

Validation requirements

Validation should include both marginal and joint behavior.

Required tests include:

- Realism of individual wind, surge, and rain marginals. [\[247\]](#) [\[252\]](#) [\[248\]](#)
- Realism of pairwise dependence: wind-surge, wind-rain, surge-rain. [\[256\]](#) [\[247\]](#) [\[248\]](#)
- Realism of tri-variate co-exceedance rates. [\[249\]](#) [\[254\]](#)
- Tail stability under Monte Carlo simulation. [\[240\]](#) [\[241\]](#)
- Regime sensitivity and explainability under scenario changes. [\[250\]](#) [\[247\]](#)

The team should pay particular attention to whether the model reproduces plausible compound flooding situations in which surge and heavy precipitation co-occur under the same storm system. [\[257\]](#) [\[247\]](#) [\[248\]](#)

Sufficiency statement

For the purposes of this platform, this hierarchical joint hazard design should be regarded as sufficiently sophisticated. It captures the dominant physical driver structure, permits explicit tail modeling, supports modular calibration, and provides an auditable dependence architecture suitable for scientific review and model governance. [\[251\]](#) [\[250\]](#) [\[249\]](#)

A more elaborate model, such as a fully coupled dynamical coastal-hydrology-atmospheric simulator, may be scientifically richer but is not necessary for the current objective. The current hierarchy is therefore the recommended stopping point for the first production-grade architecture. [\[253\]](#) [\[247\]](#) [\[248\]](#)

✱✱

Bayesian Typhoon Track Progression Specification

Purpose

This specification defines a Bayesian progression model for advancing a typhoon through time. The objective is to provide the coding team with a practical probabilistic architecture for simulating forward typhoon motion across a time horizon, with uncertainty represented explicitly rather than through a single deterministic path. [\[258\]](#) [\[259\]](#) [\[260\]](#) [\[261\]](#)

The model is intended for a simplified simulation framework rather than a definitive operational forecast system. It should therefore be interpretable, modular, easy to calibrate, and compatible with the existing storm abstractions already present in the codebase, especially the time-indexed track representation, storm speed metadata, and footprint logic. [\[262\]](#) [\[263\]](#) [\[264\]](#)

Objective

The model should answer the following question:

Given the storm state at time $t-1$, and any environmental or observational information available at time t , what is the posterior distribution of the typhoon state at time t ? [\[259\]](#) [\[260\]](#) [\[261\]](#)

The model should produce a probabilistic sequence of storm center locations, headings, forward speeds, peak winds, and size metrics over the chosen time horizon. It should support both free simulation and sequential updating when new information becomes available. [\[261\]](#) [\[265\]](#)

Design principles

The Bayesian progression model should satisfy the following principles.

1. Storm movement should be represented as a stochastic state process, not a fixed line segment between start and end points.^{[265] [261]}
2. The model should update the storm state sequentially over time using Bayes' rule.^{[259] [261]}
3. Forward motion should be driven by physically meaningful state variables, including position, heading, translation speed, wind intensity, and storm size.^{[263] [262]}
4. The model should allow uncertainty in both motion and structure, not only in one-dimensional track position.^{[261] [265]}
5. The first release should prioritize interpretability and calibration over full meteorological completeness.^{[260] [266]}

Model overview

The recommended design is a Bayesian state-space model. The typhoon is described by an unobserved latent state that evolves over time, while optional data or soft constraints provide likelihood information for updating the posterior state distribution.^{[265] [259] [261]}

At each time step the workflow is:

1. Start with the posterior distribution from the previous step.
2. Propagate that state forward using a transition model.
3. Evaluate any new information or plausibility constraints.
4. Update the state distribution using Bayes' rule.
5. Continue until the required time horizon is reached.^{[259] [261]}

State definition

The latent state at time t should be defined as:

$$s_t = (\lambda_t, \phi_t, u_t, \psi_t, V_t, R_t, k_t)$$

Equation (1)

Where:

- λ_t is longitude.
- ϕ_t is latitude.
- u_t is translation speed.
- ψ_t is heading or motion bearing.
- V_t is maximum sustained wind at the storm center.

- R_t is a size vector, at minimum including radius of maximum wind and outer wind radius.
- k_t is an optional track regime or hidden motion class. [\[264\]](#) [\[260\]](#) [\[262\]](#) [\[263\]](#)

A more explicit size vector may be defined as:

$$R_t = (R_{\max,t}, R_{\text{outer},t})$$

Equation (2)

The state should be serializable and compatible with the typhoon event model proposed in the wind-field specification. [\[262\]](#)

Regime variable

A discrete regime variable should be included because Bayesian typhoon work often benefits from track-pattern categorization. Historical storms tend to fall into recurring path families, and modeling motion conditional on regime is usually more stable and interpretable than using one global motion rule. [\[267\]](#) [\[268\]](#) [\[260\]](#)

Suggested regimes include:

- Straight westward track.
- Northwest-moving recurver.
- Sharp recurvature track.
- Slow or stalled track.
- Landfall decay track. [\[260\]](#) [\[267\]](#)

The regime may be fixed for an event or may evolve with low transition probability over time. The simplest first version should sample it once at storm genesis and hold it fixed unless evidence strongly suggests a transition. [\[267\]](#) [\[260\]](#)

Bayesian formulation

The sequential Bayesian update is:

$$p(s_t \mid y_{1:t}) \propto p(y_t \mid s_t) p(s_t \mid y_{1:t-1})$$

Equation (3)

Where:

- s_t is the latent state at time t .
- y_t is information available at time t , including observations, environmental guidance, or synthetic plausibility constraints.
- $p(s_t \mid y_{1:t-1})$ is the predictive prior from the transition model. [\[261\]](#) [\[259\]](#)

The predictive prior is obtained from:

$$p(s_t | y_{1:t-1}) = \int p(s_t | s_{t-1}) p(s_{t-1} | y_{1:t-1}) ds_{t-1}$$

Equation (4)

This is the canonical state-space formulation and should be the conceptual basis for the implementation.^{[265] [261]}

Transition model

The transition model governs how the typhoon moves from one step to the next. It should update motion, wind, and size jointly, or at least in coupled blocks.

Motion update

The recommended first-order motion block is:

$$\begin{aligned} u_t &\sim \mathcal{N}(\mu_u(s_{t-1}, z_t, k_t), \sigma_u^2(k_t)) \\ \psi_t &\sim \mathcal{N}(\mu_\psi(s_{t-1}, z_t, k_t), \sigma_\psi^2(k_t)) \end{aligned}$$

Equation (5)

Where:

- z_t is environmental forcing or steering information.
- k_t is the regime variable.^{[260] [261] [265]}

The conditional means should depend on at least:

- Previous speed and heading.
- Position and latitude.
- Regime class.
- Land or ocean interaction.
- Recurvature tendency at higher latitudes.^{[266] [267] [260]}

Position update

Given translation speed and heading, the center should be advanced using a local advection rule. In planar approximation:

$$\Delta x_t = u_t \Delta t \cos(\psi_t), \quad \Delta y_t = u_t \Delta t \sin(\psi_t)$$

Equation (6)

And then:

$$\lambda_t = \lambda_{t-1} + c_\lambda \Delta x_t, \quad \phi_t = \phi_{t-1} + c_\phi \Delta y_t$$

Equation (7)

Where λ and ϕ convert km displacement into longitude and latitude increments. The implementation may use Haversine or geodesic utilities for better geographic fidelity, as the current code already includes distance handling logic through Haversine calculations.^{[263] [264]}

Wind update

The wind state should evolve with persistence and environmental moderation:

$$V_t \sim \mathcal{N}(\mu_V(V_{t-1}, z_t, k_t), \sigma_V^2)$$

Equation (8)

The mean function may include:

- Persistence from previous wind.
- Intensification tendency over warm water.
- Weakening due to land interaction.
- Optional decay after recurvature or extratropical transition.

A simple landfall weakening rule may be:

$$V_t = V_{t-1} \exp(-k_{\text{land}} \Delta t)$$

Equation (9)

When the storm is over land or land fraction exceeds a threshold.^[264]

Size update

The storm size state should evolve stochastically but smoothly:

$$\begin{aligned} R_{\text{max},t} &\sim \mathcal{N}(\mu_{R_{\text{max}}}(R_{\text{max},t-1}, V_t, z_t), \sigma_{R_{\text{max}}}^2) \\ R_{\text{outer},t} &\sim \mathcal{N}(\mu_{R_{\text{outer}}}(R_{\text{outer},t-1}, V_t, z_t), \sigma_{R_{\text{outer}}}^2) \end{aligned}$$

Equation (10)

This keeps the wind-field model structurally coupled to the motion model without requiring a full pressure-wind system.^{[262] [263]}

Regime-conditioned mixture model

The transition model should be regime-conditioned. Formally:

$$p(s_t | s_{t-1}) = \sum_k p(s_t | s_{t-1}, k) p(k | s_{t-1})$$

Equation (11)

In the simplest implementation, the event is assigned a regime at genesis:

$$k \sim \text{Categorical}(\pi_1, \dots, \pi_K)$$

Equation (12)

Where the prior weights π_k depend on genesis location, season, and perhaps initial heading.^{[268] [267] [260]}

This is strongly recommended because it provides a principled Bayesian way to encode broad path behavior without forcing the coding team into a highly nonlinear global model.^{[267] [260]}

Likelihood model

The likelihood $p(y_t \mid s_t)$ may be fed by one of three sources.

1. Real observations

If real-time observations are available, the likelihood may use observed storm center positions, observed headings, forecast advisory data, or wind estimates.^{[259] [261]}

2. External guidance

If external model guidance exists, the likelihood may represent agreement with a forecast ensemble or consensus track, following a Bayesian model averaging style approach.^[259]

3. Soft constraints for simulation mode

If the model is running as a synthetic simulator, the likelihood may be replaced by plausibility scores. Examples include:

- Penalize unrealistically sharp heading changes.
- Penalize implausible jumps in translation speed.
- Penalize motion over terrain inconsistent with the chosen regime.
- Penalize trajectories that violate basin boundaries or historical behavior.

This allows the same Bayesian machinery to be used even when there are no real observations to assimilate.^{[266] [260]}

Inference method

The recommended implementation method is Sequential Monte Carlo, also known as a particle filter. This is the most natural way to represent uncertainty in a nonlinear storm state process.^[261]

Particle representation

At time t , represent the posterior as a weighted set of particles:

$$p(s_t \mid y_{1:t}) \approx \sum_{i=1}^N w_t^{(i)} \delta(s_t - s_t^{(i)})$$

Equation (13)

Where:

- $s_t^{(i)}$ is particle i .
- $w_t^{(i)}$ is its normalized weight.^[261]

Particle filtering workflow

1. Initialize particles from the genesis prior.
2. Propagate each particle using the transition model.
3. Compute weights from the likelihood or plausibility function.
4. Normalize weights.
5. Resample if the effective sample size falls below a threshold.
6. Repeat for the next time step.^[261]

This should be the default implementation. A deterministic mean-path fallback may be added later, but the probabilistic particle version should be considered the canonical Bayesian model.^[261]

Initialization

At genesis time $t=0$, the prior should define distributions for:

- Genesis longitude and latitude.
- Initial heading.
- Initial translation speed.
- Initial maximum wind.
- Initial size.
- Regime class.^{[269] [270] [264] [260]}

The current distribution module already provides a scenario-family concept with adjustable tail thickness. That concept should be retained and reinterpreted so that peak wind, duration, and perhaps initial translation speed can be sampled from scenario-conditioned priors.^[270]
^[269]

Environmental forcing

The state transition functions may accept an environmental vector z_t . The first version may keep this simple.

Recommended optional forcing variables:

- Latitude band or recurvature proxy.
- Ocean versus land flag.
- Basin-specific steering tendency.

- Seasonal modifier.
- Optional shear or SST proxy.

If environmental data are unavailable, the model may still work by using regime-conditioned priors plus position-based heuristics.^[266] ^[260]

Output requirements

The Bayesian progression model should produce the following outputs.

Posterior trajectory outputs

- Mean or median storm center path.
- Credible bands or confidence envelopes for location over time.
- Ensemble of particle tracks.
- Posterior distributions for heading and translation speed.^[259] ^[261]

Wind-state outputs

- Posterior mean and quantiles for maximum wind over time.
- Posterior mean and quantiles for storm size metrics over time.
- Optional scenario-conditioned severity summaries.^[263] ^[262]

Event summaries

- Probability of landfall in each region.
- Probability of recurvature.
- Probability of exceeding wind thresholds at chosen horizons.
- Time-to-landfall or time-to-closest-approach distributions.^[259] ^[261]

Interface with the typhoon wind-field model

The Bayesian progression model should feed directly into the typhoon wind-field model from the earlier specification. Each posterior particle or posterior mean state at time t should be usable as an input to the radial wind-field module, which then computes location-specific winds.^[262] ^[263]

The interface contract should therefore expose, at each time step:

- Center position.
- Translation speed.
- Heading.
- Maximum sustained wind.
- Radius of maximum wind.
- Outer wind radius.

- Land interaction flags.

This ensures the motion model and the wind-field model remain decoupled but interoperable. [\[263\]](#) [\[262\]](#)

Required new models and modules

Module	Purpose	Notes
BayesianTyphoonState	Canonical latent state container	Should include motion, wind, size, and regime. [262]
TrackRegimeModel	Regime priors and regime-conditioned transition parameters	Strongly recommended for interpretability. [260] [267]
MotionTransitionModel	Updates heading and translation speed	Core of forward progression. [261] [265]
IntensityTransitionModel	Updates maximum wind through time	Should support land weakening. [264]
SizeTransitionModel	Updates wind radii through time	Needed by wind-field model. [262] [263]
ObservationLikelihoodModel	Scores particles against observations or plausibility constraints	Must support observation-free simulation mode. [259] [261]
ParticleFilterEngine	Sequential Monte Carlo implementation	Recommended default inference engine. [261]
PosteriorExporter	Writes trajectories, credible regions, and summaries	Output-facing module

Refactoring guidance for current codebase

`storm_factory.py`

The current storm factory builds deterministic tracks and intensity profiles. It should be refactored into a prior generator that samples an initial state, regime, and track skeleton rather than a fully fixed event. [\[264\]](#) [\[260\]](#)

`data_structures.py`

The current data structures are event-oriented and use generic intensity fields. They should be extended to support a Bayesian state object, regime variables, posterior particles, and explicit wind-based semantics. [\[262\]](#)

`forward_model.py`

The current forward model interpolates deterministic state along a track and computes local decay from the center. This logic should remain available, but it should become the downstream consumer of posterior state sequences produced by the Bayesian progression engine. [\[263\]](#)

Calibration requirements

Calibration should proceed in layers.

1. Regime identification from historical track families. [\[268\]](#) [\[260\]](#) [\[267\]](#)
2. Conditional motion parameter fitting by regime for heading and speed updates. [\[260\]](#) [\[261\]](#)
3. Wind persistence and land decay fitting. [\[264\]](#)
4. Size evolution fitting, at least through heuristic rules initially. [\[262\]](#) [\[263\]](#)
5. Likelihood function tuning for observation-free simulation plausibility. [\[266\]](#) [\[260\]](#)

All parameters should be externalized into configuration files or parameter objects rather than embedded in code constants. [\[270\]](#)

Validation requirements

The coding team should validate the Bayesian progression model against the following criteria.

- Plausible particle-track spread through time, widening with forecast horizon. [\[259\]](#) [\[261\]](#)
- Realistic smoothness of heading and speed paths. [\[260\]](#) [\[266\]](#)
- Reasonable distribution of recurvature and landfall behavior by regime. [\[267\]](#) [\[260\]](#)
- Stable posterior behavior under different particle counts and time steps. [\[261\]](#)
- Consistency of progression outputs with the downstream wind-field footprint generator. [\[263\]](#)

Non-goals

The following are out of scope for the first implementation:

- Full atmospheric data assimilation.
- Full dynamical numerical weather prediction.
- Joint ocean-atmosphere coupling.
- Full pressure-wind structural models.
- Operational forecast-grade verification pipelines. [\[266\]](#) [\[261\]](#)

The focus should remain on a reduced-form Bayesian progression engine suitable for simulation, stress testing, and exploratory wind-risk modelling. [\[260\]](#) [\[261\]](#)

Implementation summary

The coding team should implement typhoon forward movement as a Bayesian sequential state-space process with regime-conditioned motion, stochastic updates to speed and heading, coupled updates to wind and size, and inference performed through a particle filter. [\[265\]](#) [\[260\]](#) [\[259\]](#) [\[261\]](#)

This is the recommended minimum viable Bayesian architecture because it captures uncertainty explicitly, remains interpretable, aligns with published Bayesian tropical cyclone track approaches, and integrates naturally with the existing track-based storm abstractions already present in the codebase.^{[264] [260] [262] [263] [261]}

Tail events and intensity skew

The Bayesian typhoon progression framework must explicitly represent tail events, meaning low-probability but high-severity typhoon realizations. In this specification, a tail event is not a separate post-storm physical phase. Instead, it is an extreme realization drawn from the same probabilistic lifecycle model, lying in the far right tail of one or more state dimensions such as peak wind, storm size, duration, translation behavior, or track rarity.^{[271] [272] [273]}

This distinction is important for architecture. The model should not create a separate “tail-event engine.” It should instead generate tail events naturally from skewed or fat-tailed prior and transition distributions, so that extreme storms remain consistent with the same Bayesian state-space machinery used for ordinary events.^{[273] [260] [261]}

Tail-event dimensions

Tail behavior should be represented in at least five dimensions.

1. **Intensity tail:** unusually high peak sustained wind, including the equivalent of the most severe typhoon classes in the chosen basin.^{[272] [274]}
2. **Size tail:** unusually large radius of maximum wind or outer wind radius, creating broad damaging-wind footprints even when peak wind is not uniquely extreme.^{[274] [272]}
3. **Duration tail:** unusually long-lived storms or storms that maintain damaging winds over long periods.^{[271] [272]}
4. **Track tail:** low-probability path geometries, such as unusual recurvature, unexpected landfall corridors, or tracks that remain close to exposed coastlines for extended periods.^{[273] [267] [260]}
5. **Compound tail:** combinations of high intensity, large size, slow translation, and vulnerable landfall geometry occurring together.^{[274] [273]}

The first implementation may focus mainly on intensity, size, and track tails, but the interfaces should reserve capacity for compound tail scenarios because these often dominate practical risk outcomes.^{[273] [274]}

Intensity skew requirement

Peak wind intensity should not be modeled with a symmetric light-tailed distribution. The codebase already contains a hybrid distribution concept in which ordinary values are modeled with a truncated normal structure and extreme values are modeled with a Pareto tail. That logic should be retained and reinterpreted as a peak-wind severity distribution.^{[269] [270]}

The exceedance distribution for peak wind V_{peak} should be defined as:

$$P(V > v) = \begin{cases} 1 - \Phi\left(\frac{v - \mu}{\sigma}\right), & v \leq v_T \\ P(V > v_T) \left(\frac{v_T}{v}\right)^\alpha, & v > v_T \end{cases}$$

Equation (14)

Where:

- μ is the baseline mean peak wind.
- σ is the baseline standard deviation.
- v_T is the threshold above which tail behavior dominates.
- α is the Pareto tail index, with smaller values representing fatter tails. [\[269\]](#) [\[270\]](#)

This specification requires the coding team to preserve positive skew in the peak-wind distribution. That skew is the mechanism by which the model can generate rare but plausible extreme typhoons for stress testing and tail-risk analysis. [\[270\]](#) [\[269\]](#) [\[273\]](#)

Tail-event classification

The model should support explicit tail-event tagging. A typhoon realization should be classified as a tail event if one or more calibrated state variables exceed predefined quantile thresholds under the reference climate or scenario family.

A recommended first version is:

- **Moderate tail event:** any of V_{peak} , R_{outer} , event duration, or landfall exposure metric exceeds the 95th percentile. [\[269\]](#) [\[270\]](#)
- **Severe tail event:** any key metric exceeds the 99th percentile, or a joint-event score exceeds its 97.5th percentile. [\[270\]](#) [\[269\]](#)
- **Compound tail event:** two or more tail dimensions simultaneously exceed chosen high-quantile thresholds. [\[274\]](#) [\[273\]](#)

The quantiles should be configurable by basin and scenario family, not hard-coded globally. [\[270\]](#)

Joint tail score

To support risk ranking and filtering, the model should optionally compute a joint tail severity score. A practical first formulation is:

$$S_{\text{tail}} = w_V q_V + w_R q_R + w_D q_D + w_T q_T + w_C q_C$$

Equation (15)

Where:

- q_V is the normalized upper-tail score for peak wind.
- q_R is the normalized upper-tail score for size.

- q_D is the normalized upper-tail score for duration.
- q_T is the normalized rarity score for track geometry.
- q_C is the normalized coastal or landfall exposure score.
- w_i are configurable weights.

This score is not a substitute for the underlying probabilistic model. It is a convenience metric for identifying and ranking extreme realizations drawn from the posterior or scenario ensemble. [\[273\]](#) [\[274\]](#)

Tail behavior in Bayesian progression

Tail events should emerge naturally in the Bayesian progression model through three mechanisms.

1. **Tail-prior sampling:** initial peak wind, size, or duration are sampled from skewed prior distributions with fat-tail support. [\[269\]](#) [\[270\]](#)
2. **Regime-conditioned rare tracks:** low-probability regimes or low-probability path updates are retained with non-zero posterior mass, allowing unusual but plausible trajectories. [\[267\]](#) [\[260\]](#)
3. **Persistence of extremes:** high wind, large size, or slow translation should have persistence structure, so once an event enters an extreme state it does not collapse unrealistically in the next time step unless land or adverse forcing justifies that change. [\[264\]](#) [\[261\]](#)

This means the transition model should not be too mean-reverting in the extreme tail. It should allow rare events to remain extreme over multiple steps when environmental conditions support persistence. [\[273\]](#) [\[261\]](#)

Stress-scenario families

The scenario-family design already present in the codebase should be preserved and extended for typhoon tails. At minimum, the following family types should be supported: [\[270\]](#)

Scenario family	Intended effect on tail behavior
Historical	Matches observed or reference-climate tail thickness. [270]
Baseline	Slightly conservative central estimate with realistic upper tail. [270]
Moderate stress	Higher mean peak wind, lower tail threshold, moderately fatter tail. [270]
Severe stress	Higher mean, stronger persistence, fatter extreme tail, more frequent large storms. [270]
Extreme	Strong positive skew, very fat upper tail, elevated probability of compound extremes. [270]

In implementation terms, scenario families should influence at least:

- Peak-wind distribution parameters μ, σ, v_T, α . [\[270\]](#)
- Size distribution parameters for $R_{\max}$ and R_{outer} .
- Regime priors for rare track behavior. [\[267\]](#) [\[260\]](#)

- Persistence or decay parameters in the transition model.

Tail outputs

The model should produce explicit outputs for tail-event analysis. These should include:

- Tail classification flag.
- Quantile rank for peak wind, size, duration, and track rarity.
- Joint tail score.^{[274] [273]}
- Scenario-family identifier.^[270]
- Exceedance probabilities for user-defined severity thresholds.^{[269] [270]}

These outputs should be available both at event level and, where relevant, at location level after coupling to the wind-field module.^{[262] [263]}

Validation of tail behavior

The coding team should validate not only mean behavior but also the upper tail. Validation should include:

- Stability of high quantiles under repeated Monte Carlo simulation.^{[269] [270]}
- Sensitivity of tail frequency to the Pareto index and threshold.^{[269] [270]}
- Plausibility of rare track realizations under regime priors.^{[260] [267]}
- Persistence of extreme states through time under favorable conditions.^[261]
- Sensible separation between ordinary, severe, and compound tail scenarios.^{[274] [273]}

Implementation note

The coding team should treat tail events as first-class outputs of the model rather than ad hoc exceptions. The architecture should therefore expose the skew and fat-tail settings explicitly in configuration, scenario definitions, posterior summaries, and event metadata.^{[273] [270]}

Hierarchical joint hazard model for wind, surge, and rain

The Bayesian typhoon progression model should be extended with a hierarchical multivariate hazard layer for wind, surge, and rain. This is the recommended architecture for handling their dependence structure because most of the correlation between these variables should arise from common storm-level drivers rather than from a single undifferentiated correlation matrix.^{[275] [276] [277] [278]}

This specification considers the hierarchical approach to be at a sufficient level of sophistication for the intended platform. It is materially more realistic than a static Gaussian correlation model, but it remains modular, calibratable, and explainable enough for implementation, review, and model governance.^{[278] [279] [280]}

Conceptual structure

The model should be organized into three layers.

1. **Latent storm-state layer:** a Bayesian state-space representation of the storm, including position, heading, translation speed, peak wind, size, land interaction, and other core storm descriptors. [\[263\]](#) [\[264\]](#) [\[262\]](#)
2. **Hazard-response layer:** conditional models for local wind, surge, and rain given the latent storm state. [\[277\]](#) [\[281\]](#) [\[282\]](#)
3. **Residual dependence layer:** a tail-aware dependence model for the remaining co-movement not explained by the latent storm state. [\[283\]](#) [\[278\]](#)

This decomposition is the preferred implementation because it aligns with physical reasoning. Wind, surge, and rain are not independent, but they are not jointly driven by an arbitrary covariance structure either. They are linked because the same underlying storm state produces all three hazards. [\[276\]](#) [\[284\]](#) [\[277\]](#)

Joint distribution

The joint hazard structure should be written conceptually as:

$$P(W, S, R) = \int P(W | Z) P(S | Z) P(R | Z) C_{\text{res}}(W, S, R | Z) P(Z) dZ$$

Equation (16)

Where:

- Z is the latent storm state.
- W is local wind.
- S is local surge or surge proxy.
- R is local rainfall or rainfall proxy.
- C_{res} is the residual dependence structure. [\[279\]](#) [\[280\]](#) [\[278\]](#)

The coding team should interpret this as follows. Most cross-variable dependence should be induced by the common storm state Z . The residual dependence term should only capture the dependence that remains after conditioning on Z , especially in the upper tail. [\[278\]](#) [\[283\]](#)

Latent storm-state layer

The storm-state layer is already defined in the Bayesian progression model. For the joint hazard model, the key latent variables should include at least:

- Center location.
- Heading.
- Translation speed.
- Peak sustained wind.

- Radius of maximum wind.
- Outer wind radius.
- Land interaction indicator.
- Optional pressure deficit proxy.
- Optional moisture or convective efficiency proxy. [\[264\]](#) [\[262\]](#) [\[263\]](#)

These variables should be treated as the shared drivers of the three hazard processes. [\[281\]](#)
[\[277\]](#)

Hazard-response layer

The second layer should define conditional hazard models for wind, surge, and rain.

Wind model

The wind model should consume the storm state and generate local wind speed using the radial wind-field structure defined elsewhere in the specification. At minimum, local wind should depend on: [\[262\]](#) [\[263\]](#)

- Distance from storm center.
- Radius of maximum wind.
- Peak sustained wind.
- Motion asymmetry.
- Land interaction. [\[263\]](#) [\[262\]](#)

Conceptually:

$$W \sim P_W(\cdot \mid Z)$$

Equation (17)

Surge model

The surge model should be conditional on storm state plus local coastal exposure. Surge is driven not only by wind magnitude but also by storm size, pressure structure, heading, coastal orientation, and local bathymetric or geometric amplifiers. [\[282\]](#) [\[277\]](#) [\[281\]](#)

A simplified conditional form is:

$$S = g_S(Z, L_{\text{coast}}) + \varepsilon_S$$

Equation (18)

Where L_{coast} represents location-specific coastal modifiers. The first implementation may use a stylized surge proxy rather than a full hydrodynamic model, but it must remain structurally conditional on the storm state rather than merely correlated with wind. [\[277\]](#) [\[281\]](#)

Rain model

The rain model should be conditional on storm state plus local terrain or moisture modifiers. Rainfall should depend on at least storm size, translation speed, persistence, land interaction, and optional moisture/orographic proxies.^{[276] [281] [282]}

A simplified conditional form is:

$$R = g_R(Z, L_{\text{terrain}}) + \varepsilon_R$$

Equation (19)

Where L_{terrain} represents local terrain, coastal, or exposure modifiers. The coding team should note that slow translation speed may materially increase accumulated rainfall, even when peak wind is not at its most extreme.^{[281] [282] [276]}

Residual dependence layer

After conditioning on the latent storm state, some dependence may remain between wind, surge, and rain, especially in the upper tail. This should be handled with an explicit residual dependence layer, not by forcing the conditional models themselves to absorb all co-movement.^{[283] [278]}

The first implementation may use one of the following:

- Tail-aware copula.
- Vine copula for flexible pairwise dependence.
- Conditional extremes model for upper-tail co-exceedance.^{[278] [283]}

The residual layer should be weak relative to the shared storm-state layer in ordinary events, but it may become more important in the extreme tail or under specific storm regimes.^{[283] [278]}

Regime-specific dependence

Dependence parameters should be allowed to vary by storm regime, because a slow-moving large landfalling storm does not produce the same joint hazard structure as a fast compact offshore storm.^{[279] [276] [277]}

At minimum, the coding team should support regime-specific dependence profiles for:

- Compact high-wind storms.
- Large slow-moving storms.
- Coastal-grazing storms.
- Recurving offshore storms.^{[276] [277] [279]}

This requirement is important because compound hazard behavior is event-type dependent and cannot be captured adequately by one global tri-variate dependence model.^{[277] [276] [278]}

Tail treatment in the joint hazard model

The joint hazard layer must integrate with the tail-event framework already defined in this specification. In particular, the model should recognize that compound tail events may arise from:

- Extreme wind with moderate rain and high surge.
- Moderate-high wind with very high surge and very high rain.
- Long-duration moderate wind with exceptional rainfall accumulation.
- Large storms producing broad but not necessarily record peak winds.^{[281] [276] [277] [273]}

This means tail classification should not rely only on peak wind. It should incorporate multi-hazard exceedance behavior and compound tail combinations.^{[274] [273]}

Calibration requirements

Calibration should proceed in stages.

1. Calibrate the latent storm-state model separately.^{[260] [261]}
2. Calibrate the wind module against wind-field expectations.^{[262] [263]}
3. Calibrate the surge proxy or surge-response function against representative coastal behavior.^{[277] [281]}
4. Calibrate the rainfall model against duration, speed, and size behavior.^{[282] [276] [281]}
5. Estimate residual dependence on standardized residuals after conditioning on storm state.^{[278] [283]}
6. Refit by regime if material structural differences are observed.^{[279] [276]}

The coding team should not attempt to estimate the full joint tri-variate dependence in one step before the conditional hazard layers are stable. The hierarchy is the primary structural assumption and should be respected during calibration.^{[283] [278]}

Validation requirements

Validation should include both marginal and joint behavior.

Required tests include:

- Realism of individual wind, surge, and rain marginals.^{[276] [281] [277]}
- Realism of pairwise dependence: wind-surge, wind-rain, surge-rain.^{[285] [276] [277]}
- Realism of tri-variate co-exceedance rates.^{[278] [283]}
- Tail stability under Monte Carlo simulation.^{[269] [270]}
- Regime sensitivity and explainability under scenario changes.^{[279] [276]}

The team should pay particular attention to whether the model reproduces plausible compound flooding situations in which surge and heavy precipitation co-occur under the same storm system.^{[286] [276] [277]}

Sufficiency statement

For the purposes of this platform, this hierarchical joint hazard design should be regarded as sufficiently sophisticated. It captures the dominant physical driver structure, permits explicit tail modeling, supports modular calibration, and provides an auditable dependence architecture suitable for scientific review and model governance. [\[280\]](#) [\[279\]](#) [\[278\]](#)

A more elaborate model, such as a fully coupled dynamical coastal-hydrology-atmospheric simulator, may be scientifically richer but is not necessary for the current objective. The current hierarchy is therefore the recommended stopping point for the first production-grade architecture. [\[282\]](#) [\[276\]](#) [\[277\]](#)

Typhoon wind-field model

The platform requires a dedicated typhoon wind-field model that converts the Bayesian storm-state trajectory into local wind speeds across space and time. This section defines the required structure, variables, equations, calibration expectations, and outputs for that model. Parametric tropical cyclone wind-field models are widely used in engineering and hazard applications, with radius of maximum wind, outer wind geometry, and asymmetry among the key drivers of accuracy. [\[287\]](#) [\[288\]](#) [\[289\]](#) [\[290\]](#)

This specification recommends a reduced-form parametric wind-field model rather than a full dynamical atmospheric solver. That choice is appropriate for the intended use because parametric wind-profile methods are standard for generating tropical cyclone wind and pressure fields in engineering applications, and published work shows that their reliability depends strongly on representing radius of maximum wind, outer wind structure, and asymmetry adequately. [\[288\]](#) [\[289\]](#) [\[291\]](#) [\[292\]](#)

Purpose

The wind-field model should answer the following question at any time t and location x : what sustained wind speed and, optionally, gust-equivalent wind speed are produced by the typhoon state at that point? [\[289\]](#) [\[290\]](#) [\[287\]](#)

The model should support:

- Point queries for local wind time series.
- Gridded wind-footprint generation.
- Threshold exceedance maps.
- Inputs to surge, damage, and compound hazard layers.
- Event summaries such as peak local wind, duration above thresholds, and area affected. [\[290\]](#) [\[292\]](#)

Model philosophy

The model should represent the storm as a moving center with a symmetric baseline wind structure plus an asymmetry correction. This is the standard practical decomposition in many parametric tropical cyclone wind-field formulations.^{[292] [293] [287]}

The wind-field architecture should therefore be:

1. A storm-center trajectory from the Bayesian progression engine.^{[264] [263] [262]}
2. A symmetric radial wind profile around the center.^{[291] [287] [292]}
3. An asymmetry correction linked to storm translation and orientation.^{[293] [294] [295]}
4. A surface conversion or reduction layer if needed to translate gradient-level winds to local sustained winds.^{[296] [289]}
5. An optional terrain or land-surface attenuation layer.^[289]

Required state inputs

At each time step, the wind-field model should consume the following inputs from the latent typhoon state:

- Center longitude and latitude.
- Heading.
- Translation speed.
- Maximum sustained wind.
- Radius of maximum wind $R_{\max}$.
- Outer wind radius R_{outer} , such as gale-force radius or equivalent footprint scale.
- Optional minimum central pressure or pressure deficit proxy.
- Land or surface interaction indicators.^{[288] [290] [291] [289]}

The coding team should explicitly store $R_{\max}$ and R_{outer} because published work identifies these geometry parameters as central to realistic parametric wind-field performance.^{[297] [298] [288]}

Coordinate geometry

For any location x at time t , define the storm-relative radius and azimuth:

$$r(x, t) = \text{distance}(x, c_t)$$

Equation (20)

$$\theta(x, t) = \text{bearing from center } c_t \text{ to location } x$$

Equation (21)

Where c_t is the storm center. The implementation may use Haversine or geodesic distance logic, consistent with the current codebase's existing geographic calculations.^{[288] [263]}

Baseline symmetric wind profile

The recommended first implementation should use a parametric radial profile whose peak occurs near $R_{\max}$ and decays smoothly outside that radius. The Holland family of profiles is a common benchmark in tropical cyclone wind modeling, and recent work continues to use Holland-style formulations or compare against them in engineering applications.^{[287] [291] [292] [296]}

A simplified symmetric profile suitable for this platform is:

$$V_{\text{sym}}(r, t) = V_{\text{max}}(t) \cdot f_r(r; R_{\text{max}}(t), L(t), p, \alpha_{\text{eye}})$$

Equation (22)

Where f_r is a normalized radial profile such that the maximum occurs near $R_{\max}$. A recommended piecewise first version is:

$$f_r(r) = \begin{cases} \alpha_{\text{eye}} + (1 - \alpha_{\text{eye}}) \frac{r}{R_{\text{max}}}, & 0 \leq r < R_{\text{max}} \\ \exp\left(-\left(\frac{r - R_{\text{max}}}{L}\right)^p\right), & r \geq R_{\text{max}} \end{cases}$$

Equation (23)

Where:

- α_{eye} is the inner-core wind floor ratio.
- L is the outer decay length scale.
- p controls the sharpness of outer decay.

This piecewise representation is intentionally simpler than a full pressure-wind derivation but preserves the critical engineering features of a reduced inner-core wind, a maximum near the eyewall radius, and a controlled outer decay.^{[292] [287] [288]}

Optional Holland-style pressure profile

If the coding team wants a more canonical parametric option, the model may support a Holland-style profile as an alternative selectable backend. Published work identifies the Holland wind-pressure profile as one of the most widely used benchmarks for tropical cyclone wind-field modeling, with $R_{\max}$ and the Holland B parameter governing wind profile shape.^{[291] [287] [292]}

A high-level representation is:

$$V_{\text{sym}}(r, t) = h(r; V_{\text{max}}(t), R_{\text{max}}(t), B(t), \Delta p(t))$$

Equation (24)

Where h denotes a Holland-type radial wind function. The specification does not require the first release to use the full Holland pressure derivation, but the interface should allow

such an option to be added later.^{[291] [292]}

Outer wind geometry

The wind-field model should not treat the storm footprint as a single arbitrary blob. It should include at least one explicit outer geometry parameter, such as gale-force wind radius or equivalent outer wind radius. Recent global work on tropical cyclone geometry shows that estimating radius of maximum winds and outer wind extent materially improves the realism of parametric wind fields.^{[298] [288]}

The model should therefore define an outer-radius constraint, for example:

$$V_{\text{sym}}(R_{\text{outer}}, t) = V_{\text{outer,ref}}$$

Equation (25)

Where $V_{\text{outer,ref}}$ is a chosen reference threshold such as gale-force wind. This provides a calibration anchor for the outer-decay parameter L or any equivalent shape parameter.^{[298] [288]}

Asymmetry correction

Tropical cyclone wind fields are not azimuthally symmetric in reality, and asymmetry linked to storm translation is a well-recognized feature in the literature. The wind-field model must therefore include an asymmetry term.^{[294] [295] [293]}

The recommended first-order asymmetry formulation is:

$$V(r, \theta, t) = V_{\text{sym}}(r, t) \cdot f_a(\theta, t)$$

Equation (26)

With:

$$f_a(\theta, t) = 1 + \epsilon(t) \cos(\theta - \phi(t))$$

Equation (27)

Where:

- $\epsilon(t)$ is the asymmetry strength.
- $\phi(t)$ is the azimuth of maximum asymmetry, typically aligned with or offset from the direction of storm motion.^{[293] [294]}

A practical first parameterization is:

$$\epsilon(t) = \min \left(\epsilon_{\text{max}}, c_{\epsilon} \cdot \frac{U(t)}{V_{\text{max}}(t) + \eta} \right)$$

Equation (28)

Where:

- $U(t)$ is translation speed.
- c_{ϵ} is a calibration constant.
- η is a small stabilizing constant.^{[295] [294] [293]}

This allows faster-moving storms to have stronger directional asymmetry while keeping the implementation simple and controllable.^{[294] [295]}

Surface reduction and terrain adjustment

Published wind-field work notes that some parametric models output gradient-level winds and require conversion to surface winds, while drag and land–sea differences also influence results. The first implementation should therefore include a configurable surface-adjustment layer.^{[296] [289]}

A simple multiplicative adjustment is:

$$V_{\text{surf}}(x, t) = \rho_{\text{surf}}(x, t) \cdot V(r, \theta, t)$$

Equation (29)

Where ρ_{surf} is a reduction factor depending on land or sea surface type and, optionally, terrain roughness. The first release may use a simple land-sea binary reduction; later versions may use a roughness or land-cover map.^{[289] [296]}

Land attenuation

Wind intensity near and after landfall should weaken relative to open-water conditions. This should be captured both in the latent storm-state update and optionally in the local surface layer. If a separate local attenuation is required, the model may apply:

$$V_{\text{final}}(x, t) = \rho_{\text{land}}(x, t) \cdot V_{\text{surf}}(x, t)$$

Equation (30)

Where ρ_{land} is a local attenuation factor. This should not replace the event-level land-decay logic already defined elsewhere; it is an optional additional exposure-layer adjustment.^{[289] [264]}

Gust conversion

If gust outputs are required, the model should expose a configurable gust factor rather than embedding gusts into the sustained-wind field. A practical interface is:

$$V_{\text{gust}}(x, t) = g_{\text{gust}}(x, t) \cdot V_{\text{final}}(x, t)$$

Equation (31)

Where g_{gust} is a configurable gust multiplier. The specification does not require a basin-specific gust law in the first release, but the interface should allow one to be added later.^{[290] [287]}

Required outputs

The wind-field model should produce the following outputs.

Point outputs

- Sustained wind time series at a specified location.
- Gust time series if enabled.
- Peak sustained wind.
- Time of peak wind.
- Duration above user-defined thresholds.
- Distance and azimuth relative to storm center at peak.^{[290] [292]}

Grid outputs

- Peak sustained wind footprint.
- Peak gust footprint if enabled.
- Threshold exceedance maps.
- Area exposed above each wind threshold.
- Time-slice wind fields for animation or replay.^[290]

Event outputs

- Maximum local wind within the study area.
- Area-weighted wind severity summaries.
- Wind-footprint metadata including $R_{\max}$, R_{outer} , asymmetry strength, and storm-motion azimuth over time.^{[288] [290]}

Model options and extensibility

The model should support at least two radial-profile backends:

- Simplified piecewise profile as the default first-release implementation.^{[287] [288]}
- Holland-style profile as an optional advanced implementation path.^{[292] [291]}

This allows the platform to start with a transparent and easily calibrated radial structure while preserving a route to a more canonical parametric wind model later.^{[287] [292]}

Calibration requirements

Calibration of the wind-field model should proceed in the following order.

1. Calibrate $R_{\max}$ and R_{outer} relationships by basin and storm class.^{[297] [298] [288]}
2. Calibrate the outer decay parameters so that winds at R_{outer} match the reference threshold.^{[298] [288]}

3. Calibrate asymmetry strength against storm translation speed and direction.^{[295] [293] [294]}
4. Calibrate land-sea reduction and optional terrain modifiers.^{[296] [289]}
5. Validate peak local wind and spatial footprint behavior against representative storms or benchmark cases.^{[292] [289] [290]}

Validation requirements

Validation should include:

- Radial realism, meaning peak winds occur near $R_{\max}$ rather than at the exact center.^{[291] [288]}
- Outer wind realism, meaning the storm footprint is neither unrealistically compact nor too diffuse.^{[298] [288]}
- Asymmetry realism for moving storms.^{[293] [294] [295]}
- Sensitivity to track, speed, and size changes.^{[289] [290]}
- Stable gridded and pointwise outputs under different time and space resolutions.^[290]

Integration with other modules

The wind-field model is a core dependency for multiple downstream modules. It must feed:

- The surge module, which should consume wind stress or wind severity metrics conditioned on coastal exposure.^{[281] [277]}
- The joint hazard model for wind, surge, and rain.^{[283] [278]}
- Any damage or loss module that uses local wind exceedance behavior.

The coding team should therefore treat the wind-field model as a canonical shared service within the typhoon architecture rather than as a narrow visualization layer.^{[292] [290]}

Sufficiency statement

For the purposes of this platform, a parametric wind-field model with explicit $R_{\max}$, outer wind geometry, asymmetry, and surface adjustment should be regarded as sufficiently sophisticated. Published research shows that these parameters are central to the practical performance of tropical cyclone parametric wind models, and they provide a strong basis for downstream compound-hazard and risk modeling without requiring a full atmospheric solver.^{[288] [291] [289] [292]}



1. <https://www.metoffice.gov.uk/blog/2025/what-are-typhoons-and-where-do-they-form>
2. <https://oceanservice.noaa.gov/facts/cyclone.html>
3. <https://www.battlbox.com/blogs/outdoors/what-causes-a-typhoon-understanding-the-dynamics-of-natures-powerful-storms>
4. <https://www.climateimpactstracker.com/how-do-typhoons-form/>

5. <https://www.seattletimes.com/nation-world/nation/what-makes-a-storm-a-typhoon-whats-a-super-typhoon/>
6. <https://www.scientificamerican.com/article/tropical-storm-typhoon-and-more-your-guide-to-hurricane-season-jargon/>
7. <https://study.com/academy/lesson/typhoon-facts-lesson-for-kids.html>
8. <https://en.wikipedia.org/wiki/Typhoon>
9. <https://www.youtube.com/watch?v=zMtL96eSzo0>
10. <https://www.vedantu.com/geography/typhoon>
11. https://en.wikipedia.org/wiki/Tropical_cyclone
12. <https://cyclone.wmo.int/pdf/Chapter-Four.pdf>
13. <https://www.ecmwf.int/sites/default/files/elibrary/1998/9281-ocean-interaction-effects-tropical-cyclones.pdf>
14. <https://pmc.ncbi.nlm.nih.gov/articles/PMC11156673/>
15. https://centaur.reading.ac.uk/109401/8/2023-03-23_Feng.pdf
16. https://texmex.mit.edu/pub/emanuel/PAPERS/Emanuel_Sobel_2013.pdf
17. https://www.meteo.physik.uni-muenchen.de/~roger/TCLeCs/Tropical_Cyclones_Observations.html
18. <https://www.youtube.com/watch?v=NDjjOfiiENs>
19. https://portal.meteo.go.tz/portal/storage/library/1726558987-A_Simple_Family_of_Tropical_Cyclone_Models.pdf
20. <https://hurricanesimulator.org>
21. <https://www.nrel.gov/docs/fy17osti/67445.pdf>
22. https://www.teachshare.com/gallery/tropical-cyclone-formation-AZwbJ84sc4C_tGQUSImOcw
23. <https://www.bbc.co.uk/bitesize/articles/zk89kty>
24. <https://www.youtube.com/watch?v=UKL9NixLIIE>
25. <https://www.gfdl.noaa.gov/global-modeling-of-tropical-cyclone-activity/>
26. <https://pmc.ncbi.nlm.nih.gov/articles/PMC11156673/>
27. <https://ssec.si.edu/storm-smart>
28. <https://www.mathematik.uni-ulm.de/stochastik/personal/schmidt/publications/cyclones.pdf>
29. <http://www.youtube.com/watch?v=RIKLuZEesqQ>
30. https://www.meteo.physik.uni-muenchen.de/~roger/Lectures/Tropical_Cyclones/060510_tropical_cyclones.pdf
31. <https://www.tynker.com/community/projects/play/hurricane-simulator/5dab036470b0026fb22e3b38/>
32. <https://par.nsf.gov/servlets/purl/10432494>
33. <https://apps.apple.com/us/app/hurricane-outbreak/id1492545081>
34. <https://oasislmf.org>
35. <https://oasislmf.github.io>
36. <https://oasislmf.org/our-modelling-platform>

37. <https://nhess.copernicus.org/articles/20/567/>
38. https://oasislmf.org/application/files/3217/1624/0849/Hazards_-_TC.pdf
39. <https://climate-adapt.eea.europa.eu/en/metadata/tools/oasis-loss-modeling-framework>
40. <https://www.international-climate-initiative.com/en/project/oasis-platform-for-climate-and-catastrophe-risk-assessment-asia-18-ii-165-asia-a-climate-and-catastrophe-risk-assessment/>
41. <https://www.lloyds.com/insights/lloyds-lab/programmes-and-initiatives/lloyds-lab-accelerator/alumni/oasis-loss-modelling-framework>
42. https://www.swissre.com/dam/jcr:08c7f2b0-89bf-43ad-a7af-54000e6c56e1/oasis_MarkPinkerton_Oasis.pdf
43. https://www.scor.com/sites/default/files/paul_nunn-oasis2.pdf
44. <https://agupubs.onlinelibrary.wiley.com/doi/full/10.1029/2023MS003686>
45. <https://github.com/oasislmf>
46. http://www.scielo.org.mx/scielo.php?script=sci_arttext&pid=S1405-55462022000301399
47. <https://www.sciencedirect.com/science/article/pii/S2212094725000027>
48. <https://www.youtube.com/watch?v=MWYCg5ptcdk>
49. <https://www.sciencedirect.com/science/article/abs/pii/S0377026522000379>
50. https://re.public.polimi.it/retrieve/b10f7fee-c1b4-480c-842c-b7d808a0cedf/Ascenso_2023_Environ._Res._Commun._5_021001.pdf
51. https://repository.library.noaa.gov/view/noaa/64695/noaa_64695_DS1.pdf
52. <https://ams.confex.com/ams/pdfpapers/109416.pdf>
53. <https://rmets.onlinelibrary.wiley.com/doi/10.1002/joc.6579>
54. <https://cyclone.wmo.int/pdf/Chapter-Four.pdf>
55. <https://www.sciencedirect.com/science/article/pii/S2212094721000669>
56. https://severeweather.wmo.int/TCFW/RAIV_Workshop2016/11_TC_Genesis_ToddKimberlain.pdf
57. <https://agupubs.onlinelibrary.wiley.com/doi/full/10.1002/2015MS000543>
58. https://en.wikipedia.org/wiki/Tropical_cyclogenesis
59. <https://adsabs.harvard.edu/full/2007TellA..59..428C>
60. <https://nhess.copernicus.org/articles/20/567/>
61. https://oasislmf.org/application/files/3217/1624/0849/Hazards_-_TC.pdf
62. <https://charts.ecmwf.int/products/medium-tc-genesis>
63. <https://www.sciencedirect.com/science/article/pii/S1674283425001837>
64. https://mountainscholar.org/bitstream/handle/10217/247/0234_Bluebook.pdf
65. https://re.public.polimi.it/retrieve/2934a2d2-6184-47b3-8303-2af846615667/Cavicchia2023_TCgenesis.pdf
66. [cli.py](#)
67. [distribution.py](#)
68. [parameters.py](#)

69. compound.py
70. gamma.py
71. pulse.py
72. saturation.py
73. parameters.py
74. distribution.py
75. infiltration.py
76. cli.py
77. risk_analytics.py
78. risk_visualization.py
79. spatial.py
80. velocity.py
81. depth_damage.py
82. compound.py
83. distribution.py
84. pulse.py
85. saturation.py
86. infiltration.py
87. gamma.py
88. builder.py
89. data_structures.py
90. gev.py
91. io.py
92. pricing.py
93. prs_analytical.py
94. response_model.py
95. terrain_grid.py
96. distribution.py
97. compound.py
98. risk_analytics.py
99. depth_damage.py
100. parameters.py
101. velocity.py
102. risk_visualization.py
103. data_structures.py
104. forward_model.py

105. storm_factory.py

106. distribution.py

107. response_model.py

108. parameters.py

109. builder.py

110. forward_model.py

111. data_structures.py

112. storm_factory.py

113. parameters.py

114. distribution.py

115. data_structures.py

116. storm_factory.py

117. forward_model.py

118. data_structures.py

119. forward_model.py

120. storm_factory.py

121. distribution.py

122. parameters.py

123. [https://hfip.psu.edu/Groupmeeting/20180523/BMA TC Track_20180522.pptx](https://hfip.psu.edu/Groupmeeting/20180523/BMA%20TC%20Track_20180522.pptx)

124. <https://scholars.lib.ntu.edu.tw/entities/publication/87334e58-0980-4232-b55e-a89492ec5d3d>

125. <https://www.climatechange.ai/papers/iclr2020/14>

126. <https://www.sciencedirect.com/science/article/pii/S2211675317302841>

127. data_structures.py

128. forward_model.py

129. storm_factory.py

130. <https://agris.fao.org/search/en/providers/122535/records/65dfd5147c7033e84bef2efb>

131. <https://www.dbpia.co.kr/journal/articleDetail?nodeId=NODE11401841>

132. <https://pure.ewha.ac.kr/en/publications/bayesian-forecasting-of-seasonal-typhoon-activity-a-track-pattern>

133. <https://www.jstor.org/stable/20868951>

134. <https://www.soest.hawaii.edu/MET/Hsco/publications/2011.1.pdf>

135. <https://www.sciencedirect.com/science/article/abs/pii/S0169809511002158>

136. https://d197for5662m48.cloudfront.net/documents/publicationstatus/232162/preprint_pdf/e6e3cef66661d9ddd892aaf39c40939c.pdf

137. <https://www.sciencedirect.com/science/article/abs/pii/S2211675317302841>

138. <https://ui.adsabs.harvard.edu/abs/2025AMS...10553786H/abstract>

139. <https://www.soest.hawaii.edu/MET/Hsco/Paper/2010JCLI3710.pdf>

140. https://www.linkedin.com/posts/jonathan-dekermanjian_how-can-we-use-probabilistic-modeling-to-activty-7346291900225605632-ljYf

141. <https://www.soest.hawaii.edu/MET/Hsco/publications/2011.1.pdf>

142. https://hfip.psu.edu/Groupmeeting/20180523/BMA TC Track_20180522.pptx

143. <https://scholars.lib.ntu.edu.tw/entities/publication/87334e58-0980-4232-b55e-a89492ec5d3d>

144. <https://www.climatechange.ai/papers/iclr2020/14>

145. data_structures.py

146. forward_model.py

147. storm_factory.py

148. <https://www.sciencedirect.com/science/article/pii/S2211675317302841>

149. <https://agris.fao.org/search/en/providers/122535/records/65dfd5147c7033e84bef2efb>

150. <https://pure.ewha.ac.kr/en/publications/bayesian-forecasting-of-seasonal-typhoon-activity-a-track-pattern>

151. <https://www.soest.hawaii.edu/MET/Hsco/Paper/2010JCLI3710.pdf>

152. distribution.py

153. parameters.py

154. <https://www.climateimpactstracker.com/how-do-typhoons-form/>

155. <https://disasterphilanthropy.org/cdp-resource/hurricanes-typhoons-and-cyclones/>

156. https://en.wikipedia.org/wiki/Tropical_cyclone

157. <https://www.swissre.com/risk-knowledge/mitigating-climate-risk/typhoon-japan.html>

158. <https://shelterbox.org/disasters-explained/hurricanes/>

159. <https://www.battlbox.com/blogs/outdoors/what-happens-during-a-typhoon-understanding-the-fury-of-nature>

160. <https://hurricanescience.org/science/science/hurricanecdecay/index.html>

161. <https://scholars.lib.ntu.edu.tw/entities/publication/87334e58-0980-4232-b55e-a89492ec5d3d>

162. <https://pure.ewha.ac.kr/en/publications/bayesian-forecasting-of-seasonal-typhoon-activity-a-track-pattern>

163. <https://www.eurofighter.com/the-aircraft/features>

164. <https://www.sciencedirect.com/science/article/pii/S2212094725001021>

165. https://en.wikipedia.org/wiki/Eurofighter_Typhoon

166. <https://hushkit.substack.com/p/10-mind-blowing-features-of-the-eurofighter>

167. https://trainstation.fandom.com/wiki/Typhoon_Tail

168. <https://www.facebook.com/scmp/posts/what-happens-when-a-typhoon-hits/1183889723787078/>

169. <http://www.benthamopenarchives.com/contents/pdf/TOASCJ/TOASCJ-7-14.pdf>

170. <https://emergency.fsu.edu/hazards/tropical-storms-hurricanes/tropical-storms-hurricanes-after>

171. <https://www.soest.hawaii.edu/MET/Hsco/publications/2011.1.pdf>

172. https://hfip.psu.edu/Groupmeeting/20180523/BMA TC Track_20180522.pptx

173. <https://scholars.lib.ntu.edu.tw/entities/publication/87334e58-0980-4232-b55e-a89492ec5d3d>

174. <https://www.climatechange.ai/papers/iclr2020/14>

175. data_structures.py

176. forward_model.py

177. storm_factory.py

178. <https://www.sciencedirect.com/science/article/pii/S2211675317302841>

179. <https://agris.fao.org/search/en/providers/122535/records/65dfd5147c7033e84bef2efb>

180. <https://pure.ewha.ac.kr/en/publications/bayesian-forecasting-of-seasonal-typhoon-activity-a-track-pattern>

181. <https://www.soest.hawaii.edu/MET/Hsco/Paper/2010JCLI3710.pdf>

182. distribution.py

183. parameters.py

184. <https://www.climateimpactstracker.com/how-do-typhoons-form/>

185. https://en.wikipedia.org/wiki/Tropical_cyclone

186. <https://www.swissre.com/risk-knowledge/mitigating-climate-risk/typhoon-japan.html>

187. <https://disasterphilanthropy.org/cdp-resource/hurricanes-typhoons-and-cyclones/>

188. <https://agupubs.onlinelibrary.wiley.com/doi/full/10.1029/2025WR041526>

189. <https://www.imsi.institute/videos/transformed-linear-methods-for-multivariate-extremes-and-application-to-climate/>

190. <https://www.nature.com/articles/s41467-025-56573-8>

191. <https://projecteuclid.org/journals/annals-of-applied-statistics/volume-14/issue-3/Climate-extreme-event-attribution-using-multivariate-peaks-over-thresholds-modeling/10.1214/20-AOAS1355.pdf>

192. <https://nhess.copernicus.org/articles/26/1663/2026/>

193. https://whitneyhuang83.github.io/WCE2023/Clemson_WCE2023_Program.pdf

194. <https://news.mit.edu/2024/mit-derived-algorithm-helps-forecast-frequency-extreme-weather-0326>

195. <https://www.bohrium.com/en/paper-details/multivariate-postprocessing-methods-for-high-dimensional-seasonal-weather-forecasts/812637422775959554-806>

196. <https://www.sciencedirect.com/science/article/abs/pii/S1051200425004464>

197. <https://www.sciencedirect.com/science/article/pii/S221209472400063X>

198. <https://hess.copernicus.org/articles/25/4403/>

199. <https://pmc.ncbi.nlm.nih.gov/articles/PMC7661409/>

200. [https://ascelibrary.org/doi/abs/10.1061/\(ASCE\)HE.1943-5584.0002154](https://ascelibrary.org/doi/abs/10.1061/(ASCE)HE.1943-5584.0002154)

201. <https://www.nature.com/nature-index/topics/l4/compound-flooding-dynamics-in-coastal-environments>

202. http://www.e-ala-kai-research.com/s/Rueda_om_2016.pdf

203. <https://www.sciencedirect.com/science/article/abs/pii/S1463500316300592>

204. https://repository.library.noaa.gov/view/noaa/48828/noaa_48828_DS1.pdf

205. <https://agupubs.onlinelibrary.wiley.com/doi/full/10.1029/2023WR037014>

206. https://digitalcommons.usf.edu/msc_facpub/539/

207. <https://livrepository.liverpool.ac.uk/3150427/1/fbuil-07-727294.pdf>

208. <https://www.hydrol-earth-syst-sci-discuss.net/7/7563/2010/hessd-7-7563-2010-print.pdf>

209. <https://scholars.lib.ntu.edu.tw/entities/publication/87334e58-0980-4232-b55e-a89492ec5d3d>

210. <https://www.climatechange.ai/papers/iclr2020/14>

211. data_structures.py

212. forward_model.py

213. <https://d-nb.info/1215304099/34>

214. https://www.uni-siegen.de/presse/press_releases/nat_clim_study.html?m=e

215. <https://lirias.kuleuven.be/server/api/core/bitstreams/2f85a5b5-9805-46e8-8255-b362eb9bb90a/content>

216. <https://www.sciencedirect.com/science/article/pii/S221209472400063X>

217. <https://hess.copernicus.org/articles/25/4403/>

218. <https://pmc.ncbi.nlm.nih.gov/articles/PMC7661409/>

219. [https://ascelibrary.org/doi/abs/10.1061/\(ASCE\)HE.1943-5584.0002154](https://ascelibrary.org/doi/abs/10.1061/(ASCE)HE.1943-5584.0002154)

220. data_structures.py

221. forward_model.py

222. storm_factory.py

223. https://repository.library.noaa.gov/view/noaa/48828/noaa_48828_DS1.pdf

224. <https://agupubs.onlinelibrary.wiley.com/doi/full/10.1029/2023WR037014>

225. <https://www.hydrol-earth-syst-sci-discuss.net/7/7563/2010/hessd-7-7563-2010-print.pdf>

226. <https://www.perplexity.ai/search/0c8b0ad2-5486-4380-ae60-ab1ef698581b>

227. http://www.e-ala-kai-research.com/s/Rueda_om_2016.pdf

228. <https://www.sciencedirect.com/science/article/abs/pii/S1463500316300592>

229. <https://www.soest.hawaii.edu/MET/Hsco/publications/2011.1.pdf>

230. https://hfip.psu.edu/Groupmeeting/20180523/BMA_TC_Track_20180522.pptx

231. <https://scholars.lib.ntu.edu.tw/entities/publication/87334e58-0980-4232-b55e-a89492ec5d3d>

232. <https://www.climatechange.ai/papers/iclr2020/14>

233. data_structures.py

234. forward_model.py

235. storm_factory.py

236. <https://www.sciencedirect.com/science/article/pii/S2211675317302841>

237. <https://agris.fao.org/search/en/providers/122535/records/65dfd5147c7033e84bef2efb>

238. <https://pure.ewha.ac.kr/en/publications/bayesian-forecasting-of-seasonal-typhoon-activity-a-track-pattern>

239. <https://www.soest.hawaii.edu/MET/Hsco/Paper/2010JCLI3710.pdf>

240. distribution.py

241. [parameters.py](#)

242. <https://www.climateimpactstracker.com/how-do-typhoons-form/>

243. https://en.wikipedia.org/wiki/Tropical_cyclone

244. <https://www.swissre.com/risk-knowledge/mitigating-climate-risk/typhoon-japan.html>

245. <https://disasterphilanthropy.org/cdp-resource/hurricanes-typhoons-and-cyclones/>

246. <https://www.sciencedirect.com/science/article/pii/S221209472400063X>

247. <https://hess.copernicus.org/articles/25/4403/>

248. <https://pmc.ncbi.nlm.nih.gov/articles/PMC7661409/>

249. [https://ascelibrary.org/doi/abs/10.1061/\(ASCE\)HE.1943-5584.0002154](https://ascelibrary.org/doi/abs/10.1061/(ASCE)HE.1943-5584.0002154)

250. http://www.e-ala-kai-research.com/s/Rueda_om_2016.pdf

251. <https://www.sciencedirect.com/science/article/abs/pii/S1463500316300592>

252. https://repository.library.noaa.gov/view/noaa/48828/noaa_48828_DS1.pdf

253. <https://agupubs.onlinelibrary.wiley.com/doi/full/10.1029/2023WR037014>

254. <https://www.hydrol-earth-syst-sci-discuss.net/7/7563/2010/hessd-7-7563-2010-print.pdf>

255. <https://www.nature.com/nature-index/topics/l4/compound-flooding-dynamics-in-coastal-environments>

256. <https://livrepository.liverpool.ac.uk/3150427/1/fbuil-07-727294.pdf>

257. https://digitalcommons.usf.edu/msc_facpub/539/

258. <https://www.soest.hawaii.edu/MET/Hsco/publications/2011.1.pdf>

259. https://hfip.psu.edu/Groupmeeting/20180523/BMA_TC_Track_20180522.pptx

260. <https://scholars.lib.ntu.edu.tw/entities/publication/87334e58-0980-4232-b55e-a89492ec5d3d>

261. <https://www.climatechange.ai/papers/iclr2020/14>

262. [data_structures.py](#)

263. [forward_model.py](#)

264. [storm_factory.py](#)

265. <https://www.sciencedirect.com/science/article/pii/S2211675317302841>

266. <https://agris.fao.org/search/en/providers/122535/records/65dfd5147c7033e84bef2efb>

267. <https://pure.ewha.ac.kr/en/publications/bayesian-forecasting-of-seasonal-typhoon-activity-a-track-pattern>

268. <https://www.soest.hawaii.edu/MET/Hsco/Paper/2010JCLI3710.pdf>

269. [distribution.py](#)

270. [parameters.py](#)

271. <https://www.climateimpactstracker.com/how-do-typhoons-form/>

272. https://en.wikipedia.org/wiki/Tropical_cyclone

273. <https://www.swissre.com/risk-knowledge/mitigating-climate-risk/typhoon-japan.html>

274. <https://disasterphilanthropy.org/cdp-resource/hurricanes-typhoons-and-cyclones/>

275. <https://www.sciencedirect.com/science/article/pii/S221209472400063X>

276. <https://hess.copernicus.org/articles/25/4403/>
277. <https://pmc.ncbi.nlm.nih.gov/articles/PMC7661409/>
278. [https://ascelibrary.org/doi/abs/10.1061/\(ASCE\)HE.1943-5584.0002154](https://ascelibrary.org/doi/abs/10.1061/(ASCE)HE.1943-5584.0002154)
279. http://www.e-ala-kai-research.com/s/Rueda_om_2016.pdf
280. <https://www.sciencedirect.com/science/article/abs/pii/S1463500316300592>
281. https://repository.library.noaa.gov/view/noaa/48828/noaa_48828_DS1.pdf
282. <https://agupubs.onlinelibrary.wiley.com/doi/full/10.1029/2023WR037014>
283. <https://www.hydrol-earth-syst-sci-discuss.net/7/7563/2010/hessd-7-7563-2010-print.pdf>
284. <https://www.nature.com/nature-index/topics/l4/compound-flooding-dynamics-in-coastal-environments>
285. <https://livrepository.liverpool.ac.uk/3150427/1/fbuil-07-727294.pdf>
286. https://digitalcommons.usf.edu/msc_facpub/539/
287. <https://www.sciencedirect.com/science/article/pii/S235248552200024X>
288. <https://nhess.copernicus.org/articles/19/2359/2019/>
289. <https://www.sciencedirect.com/science/article/pii/S0167610520300416>
290. <https://nhess.copernicus.org/articles/20/567/2020/nhess-20-567-2020.pdf>
291. <https://www.sciencedirect.com/science/article/abs/pii/S0167610524000138>
292. <https://agupubs.onlinelibrary.wiley.com/doi/abs/10.1029/2021JD036359>
293. <https://agupubs.onlinelibrary.wiley.com/doi/full/10.1002/2016JC012237>
294. <https://wes.copernicus.org/preprints/wes-2025-273/wes-2025-273.pdf>
295. https://www.hpc.msstate.edu/publications/docs/2014/04/12528fitz_asymmetry_ams_hurr_conf.pdf
296. <https://wes.copernicus.org/preprints/wes-2025-269/wes-2025-269.pdf>
297. <https://nhess.copernicus.org/articles/16/705/2016/nhess-16-705-2016.pdf>
298. <https://nhess.copernicus.org/articles/19/2359/2019/nhess-19-2359-2019.pdf>
299. https://repository.library.noaa.gov/view/noaa/60204/noaa_60204_DS1.pdf
300. <https://pdfs.semanticscholar.org/66f8/3b0a253a2294deb9ebc2c29cdd6d67fa49ea.pdf>
301. <https://www.meteo.physik.uni-muenchen.de/~roger/Publications/M6.pdf>